

Project Diary

Consolidated session logs, ordered by date and grouped by day, then by role (P1, P2, P3, P4) within each day.

Team

- **P1 — Sabrina** — Backend / Data Engineering
- **P2 — Emma** — Quant / Portfolio Optimization
- **P3 — Matteo** — ML / Risk Profiling
- **P4 — Elena** — Frontend / LLM / Docs

Note: the session-level logs begin on 27 April (Week 1, late April) and run through to mid-June. The per-week summary table at the end provides a consolidated milestone snapshot of the core deliverables (W1-W4).

Project phasing. From the start the team planned the work in two phases. **Weeks 1-4 (27 April - 24 May)** were dedicated to building the *core* of the project — the full pipeline end to end: data layer, ML risk profiler, HRP optimizer, LLM narrator + validator, API, frontend, deployment and academic documentation. By the end of Week 4 the system was feature-complete, tested and deployed (v1.0). The **subsequent weeks (Week 5 onwards, late May - June)** were intentionally reserved for *refinement*: UI/UX polish, bug fixes, theme work (Dark/Light), documentation cleanup and post-submission detail work. The per-week summary table below therefore reports the core deliverables (W1-W4) as the milestone snapshot; the dated entries continue into Weeks 5-7 with the refinement work.

27 April 2026 — Week 1 (Monday)

P3 — ML / Risk Profiling

Estimated duration: 1h30

What I did

- Defined the full questionnaire structure: 10 questions split into 3 sections (Who You Are Financially, How You Invest, How You React)
- Chose the Grable & Lytton (1999) methodology as the academic basis for the questions
- Defined the scoring system (0-30) with confidence zones and the Q7 override rule
- Produced docs/questionnaire_schema.md with questions, answer options, per-question rationale and bibliographic references
- Configured Git locally, cloned the repo, created branch feature/p3-questionnaire-schema
- Pushed the file and opened PR #1 toward main

How I did it

- Designed the questionnaire structure and the methodological choice myself, using Claude as a support tool

- Discussed each question and its mapping to the three profiles (CONSERVATIVE, MODERATE, AGGRESSIVE)
- Ran the Git commands from the terminal (Mac) for clone, branch, add, commit, push
- Opened the PR manually on GitHub

Difficulties

- Initial `git add` error because the file was not yet in the `docs/` folder — solved by copying it from Downloads with `cp`
- First-time understanding of the Git flow (branch, PR, main) — clarified during the session

Achievements / Key decisions

- **Questionnaire completed and committed** — first P3 deliverable on GitHub
- **PR #1 opened** on `feature/p3-questionnaire-schema` → `main`
- Key choice: Q7 has an override rule — if the user answers “safety net”, the profile is capped to CONSERVATIVE regardless of total score
- Key choice: Q9 placed last to reduce social-desirability bias — academically defensible
- Confidence zones defined: borderline at 8-9, 10-11, 18-19, 20-21 → `low_confidence_flag = True`

Next steps

- Wait for P1’s review on PR #1
- Start `backend/ml/profiler/rule_based.py` (Wed–Thu W1): scoring logic, Q7 override, output `profile_label + confidence + low_confidence_flag`
- Verify that `AGENTS.md` has been pushed by P4

Notes for the academic PDF

- The questionnaire follows the **Grable & Lytton (1999) Risk Tolerance Scale** — citation ready
- The behavioural questions (Q8, Q9) use first-person framing to reduce social-desirability bias — defensible motivation
- Q6 + Q5 together identify asymmetric profiles (people who know but never invested, or vice versa) — interesting point for the ML Risk Profiler section
- Bibliographic references already in the file: Grable & Lytton 1999, Guiso et al. 2018, Fed SCF 2022, MiFID II Art. 25

28 April 2026 — Week 1 (Tuesday)

P1 — Backend / Data Engineering

Estimated duration: ~1 hour

What I did

- Configured `ci.yml` in `.github/workflows/` — GitHub Actions with lint (ruff) + pytest on every push and PR
- Resolved the CI “collected 0 items” error by adding `tests/test_placeholder.py`
- Resolved the CI “E501 line too long” error by setting `line-length = 100` in `pyproject.toml`
- Reviewed and approved Emma’s (P2) PR #2 (`universe_config.py`): corrected `ASSET_WEIGHT_MIN` from 0.03 to 0.05 to align with design v3.1
- Merged PR #2 (`universe_config.py`) and PR #3 (`ci.yml`) into `main`
- Created `backend/data/schema.sql` — DB schema v3.1 with tables `users`, `recommendations`, `market_data_snapshots` and their indexes
- Created `backend/data/loader.py` — complete `ValidatedDataLoader` with NaN gate, ffill, SHA-256 hash, UCITS fallback logic, `DataQualityReport`
- Configured branch protection on `main`: require PR + 1 review + green CI before merge

How I did it

- All work done directly on GitHub (web interface), no local git
- CI configured with `astral-sh/setup-uv@v5` for dependency management via `uv`
- Reviewed Emma’s code against the canonical v3.1 design before approving the merge
- `ValidatedDataLoader` written per v3.1: `load()` returns `(pd.DataFrame, DataQualityReport)`, fallback ticker resolved before the main download, SHA-256 hash computed on `prices.to_csv()`
- Branch protection configured via Settings → Branches → Add ruleset

Difficulties

- CI failed with exit code 5 (zero tests found) — solved by adding `test_placeholder.py`
- CI failed with E501 (long lines in ETF rationale) — solved by raising `line-length` to 100
- GitHub navigation not immediate for someone without platform experience (branch switching, committing to a specific branch)
- `loader.py` already existed as an empty placeholder (initial commit) — modified rather than recreated

Achievements / Key decisions

- **W1 ~85% complete** in a single session
- **Green CI** on `main` — every future PR gets automatic feedback
- **Branch protection active** — professional process visible in the repo history
- `universe_config.py` **aligned to design v3.1** — `ASSET_WEIGHT_MIN` = 0.05, 8 ETFs, 4 clusters, 3 UCITS tickers, integrity assertions at import time
- **DB schema v3.1 complete** with all required fields: `ucits_tickers_used`, `fallback_tickers_applied`, `regulatory_context`, `etf_universe_version`, `market_data_hash`
- `ValidatedDataLoader` **scaffold** ready — complete interface, UCITS fallback logic implemented, `DataQualityReport` with `to_dict()` for DB serialization

Next steps

- `snapshots.py` — `market_data_snapshots` audit-trail logic (Fri W1)
- `test_data_loader.py` — at least 2 happy-path tests (Fri W1)

- FastAPI skeleton — 5 stub endpoints `/profile`, `/optimize`, `/compare`, `/advice`, `/backtest` (start W2)
- Rate limiting with `slowapi` + API key header auth (W2)
- ADR-001 — SQLite vs PostgreSQL document (W2)
- Verify P3 delivers an importable `rule_based.py` by Monday W2 — otherwise prepare a 3-cluster stub

Notes for the academic PDF

- The choice of `uv` as package manager is defensible as a modern, reproducible alternative to classic `pip` — install speed and deterministic lockfile
 - Branch protection with mandatory CI is an element of the agentic process documentable in the Lessons Learned section (Section 7)
 - The `market_data_hash` field (SHA-256 of `prices.to_csv()`) deserves a note in the DB section: it guarantees bit-for-bit reproducibility of recommendations even if `yfinance` retroactively adjusts historical data (splits, dividends)
 - The UCITS/US tension in `universe_config.py` (EFA, GLD, VNQ with no liquid UCITS equivalent) is direct material for the Limitations and Failure Modes section
-

P2 — Quant / Portfolio Optimization (session 1)

Estimated duration: ~1.5 hours

What I did

- Checked the shared repo state: `backend/data/` already initialized by P1, `universe_config.py` present but empty
- Cloned the repo locally (`git clone`)
- Created branch `feature/p2-universe-config`
- Pasted and committed the `universe_config.py` code on GitHub (first via browser, then synced locally)
- Ran the import test from the terminal (`get_primary_tickers()`)
- Opened Pull Request #2 toward `main` with a review request to P1 (Sabrina15072002)

How I did it

- Wrote the code with AI support (Claude) as an assistant, aligned to the canonical v3.1 design
- File structured with `dataclass(frozen=True)` for configuration immutability
- Helper functions implemented for direct compatibility with `ValidatedDataLoader` (P1) and `hrp.py` (P2 W2)
- Integrity assertions run at import time (`_validate_universe()`) to guard against accidental misconfiguration
- Git workflow: clone → branch → commit on GitHub browser → pull locally → test → PR

Difficulties

- First experience with Git and GitHub: browser vs terminal flow unclear at first

- Commit on GitHub via browser not saved the first time (missed clicking “Commit changes”)
- `cd robo-advisor` run twice by mistake (already inside the folder after clone)
- `git pull origin main` did not download the file because the commit was on a separate branch — solved with `git pull origin feature/p2-universe-config`

Achievements / Key decisions

- **W1 task #1 complete:** `universe_config.py` written, tested, PR opened
- **P1 dependency unblocked:** P1 can now implement `ValidatedDataLoader` with fallback logic
- **Design choice:** EFA keeps the same ticker as primary and fallback (no UCITS equivalent with adequate yfinance coverage) — documented in the `rationale` field
- **Design choice:** XEON.MI as EUR cash instead of BIL USD — more consistent for an EU investor, with BIL fallback if yfinance returns excessive NaNs
- **Design choice:** AGGH.MI as EUR-hedged aggregate bond instead of AGG USD — reduces FX risk for an EU investor, cluster `safe_haven`
- Import-time assertions verify: exactly 8 ETFs, no duplicates, 4 clusters present, ≥ 3 UCITS

Next steps

- W1 task #2: scaffold `backend/optimizer/hrp.py` with the `OptimizationResult` Typed-Dict/dataclass
- W1 task #3: stub `tests/test_optimizer.py` with at least 2-3 structural tests
- Start Ledoit-Wolf with `pypfopt.CovarianceShrinkage` on synthetic data
- Wait for P1 to merge the PR before importing `universe_config` into `hrp.py`

Notes for the academic PDF

- **Hybrid UCITS/US universe:** keeping primary UCITS and US fallback is motivated by MiFID II compliance for EU investors. Cite in Section 3 (Portfolio Optimization) as a conscious design — not technical — choice
- **AGGH.MI vs AGG:** the substitution introduces slightly reduced correlation with TLT (different denomination currency) — the HRP dendrogram will reflect this in the cluster C structure. Expected and didactically relevant result
- **Cluster D (cash):** minimum allocation guaranteed across all profiles via `ASSET_WEIGHT_MIN` — ensures a liquidity buffer. Mention as a risk-management choice in the guardrail section
- Limitation to cite: EFA has no UCITS equivalent with comparable liquidity and data coverage on yfinance — geographic gap of the chosen ETF universe

P2 — Quant / Portfolio Optimization (session 2)

Estimated duration: ~30 minutes

What I did

- Generated and pasted the `OptimizationResult TypedDict` in `backend/optimizer/hrp.py`
- Verified the import from terminal with `python3 -c "from backend.optimizer.hrp import OptimizationResult; print('OK')"` → OK
- Opened PR #4 toward main from branch `feature/p2-optimizer-scaffold`
- Requested review from Sabrina15072002 (P1)
- Fixed a ruff lint error in `hrp.py` (unordered imports)
- Fixed a ruff lint error in `backend/data/loader.py` (unused `Optional` import — P1's file)
- Green CI: "All checks have passed"

How I did it

- Wrote the `OptimizationResult` code with AI support (Claude) as an assistant, aligned to the canonical v3.1 design
- Structure: `TypedDict` with `Literal` for enum-like fields (`algorithm`, `solver_status`)
- File created directly from the GitHub browser editor to avoid local-branch problems
- Lint fix also done from the GitHub browser editor
- Import verification run from the local terminal after `git pull origin feature/p2-optimizer-scaffold`

Difficulties

- **VS Code would not save the file** — `Cmd+S` had no visible effect, the "M" (modified) marker stayed on the tab. Worked around by editing directly on the GitHub browser
- **code command unavailable in the terminal** — `zsh: command not found: code`. Worked around with the browser editor
- **Local branch not aligned with remote** — `git push` failed with "src refspec does not match any" because the branch was created first on the GitHub browser. Solved with `git checkout -b feature/p2-optimizer-scaffold + git pull`
- **CI failed on a P1 file** — `backend/data/loader.py` had an unused `from typing import Optional`. Fixed directly on the branch with a browser commit

Achievements / Key decisions

- **W1 task #2 complete:** `OptimizationResult TypedDict` written, verified, PR #4 opened with green CI
- **Fields included:** `algorithm`, `weights`, `expected_return`, `expected_volatility`, `sharpe_ratio`, `risk_contributions`, `optimizer_version`, `solver_status`, `ucits_tickers_used`, `fallback_tickers_applied`
- The `ucits_tickers_used` and `fallback_tickers_applied` fields are v3.1 additions — needed for the audit trail and the UI
- The `risk_contributions` field is P0 mandatory: consumed by the LLM narrator (P4) and the validator

Next steps

- W1 task #3: stub `tests/test_optimizer.py` with at least 2-3 structural tests
- Wait for Sabrina (P1) to merge PR #4 before importing `OptimizationResult` into other modules

- Install the VS Code command-line tools (Shell Command: Install 'code' command in PATH) to avoid future issues

Notes for the academic PDF

- `OptimizationResult` as an interface contract is a defensible design choice: it guarantees all modules (P1, P3, P4) receive structured, typed data, reducing integration errors
 - The `risk_contributions` field deserves mention in the Portfolio Optimization section: it is the direct link between the optimizer and the XAI/LLM layer
 - The UCITS fields are motivated by MiFID II compliance — citable in the EU Investor Note section
-

P3 — ML / Risk Profiling

Estimated duration: ~2.5 hours

What I did

- Reviewed the full W1 tasks and identified the state of progress
- Decided the canonical naming for `profile_label`: **CONSERVATIVE / MODERATE / AGGRESSIVE** (EN, UPPER) — to propagate across the whole codebase
- Wrote the complete `backend/ml/profiler/rule_based.py` (Phase A profiler)
- Applied two fixes from external code review:
 - Fix #1: “validate at the boundary” — extracted a private `_compute_score_unchecked` to avoid double validation in the `profile_user` → `compute_score` path
 - Fix #2: normalized `top_drivers` against the maximum **possible** deviation (constant 1.5) instead of the observed one — avoids inflated importance on uniformly lukewarm answers
- Ran a smoke test on all 14 scoring-table boundaries + Q7 override + all-equal-responses case
- Committed on branch `feature/p3-rule-based-profiler` and pushed
- Opened PR #6 toward `main`
- Identified a naming conflict in P1’s `schema.sql` (IT vs EN)
- Left a comment on PR #6 notifying P1 (@emmaerba) of the conflict

How I did it

- Wrote the code with Claude as a support tool, starting from the existing `questionnaire_schema.md v1.0`
- Approach: strict type hints, named constants (zero magic numbers), NumPy-style docstrings, pure functions with no side effects
- Used a second AI review to surface candidate fixes, which I critically evaluated before applying
- Smoke test run directly in Python before commit
- Git operations from the macOS terminal (zsh)
- PR opened manually on the GitHub browser

Difficulties

- Terminal initially opened in the home ~ instead of the repo folder — solved with `cd ~/robo-advisor`
- `compare/base` branches inverted in the GitHub UI on the first attempt — fixed manually
- `profile_label` naming conflict discovered while reading P1's `schema.sql` (IT vs EN) — flagged in the PR, awaiting P1's fix

Achievements / Key decisions

- `rule_based.py` **complete and committed** — PR #6 opened, awaiting P1 review
- **Canonical naming fixed**: CONSERVATIVE / MODERATE / AGGRESSIVE (EN, UPPER) — decision to propagate to P1 (`schema.sql`) and P4 (Ground Truth JSON)
- `ProfilerOutput` **schema stable**: identical to what the GBM will produce in W3, no downstream refactor needed
- **Q7 override documented as a hard MiFID II rule** (confidence = 1.0, not probabilistic) — academically relevant distinction for the PDF
- **Phase A** `top_drivers`: documented deterministic heuristic, schema identical to SHAP Phase B

Next steps

- Wait for P1's review/merge on PR #6 (must fix `schema.sql` naming IT→EN)
- Create the `backend/ml/profiler/scf_pipeline.py` scaffold (W1 priority)
- Create a draft `docs/adr/ADR-002-scf-preprocessing.md` (W1 priority, by Sunday)
- W2: write `tests/test_profiler.py` with ≥ 3 tests per label + the identified edge cases (boundaries 7/8, 9/10, 17/18, 21/22; Q7 override; all-equal responses)

Notes for the academic PDF

- **Q7 override**: describe it as a MiFID II Art. 25 regulatory constraint (suitability assessment), not an algorithmic choice. The “hard rule vs probabilistic estimate” distinction is relevant for the ML Risk Profiler section
 - **Phase A** `top_drivers`: document honestly as a deterministic heuristic (proxy for SHAP). The schema was designed to be identical to Phase B — this demonstrates architectural thinking, not a patch
 - **Naming decision**: could warrant a mini-ADR to document the EN vs IT choice — the kind of decision-trail documentation the professor values (coding-style criterion)
 - **Citations already used in code**: Grable & Lytton (1999), MiFID II Directive 2014/65/EU Art. 25 — to reuse verbatim in the LaTeX section
-

P4 — Frontend / LLM / Docs

Estimated duration: ~1h 30min

What I did

- Wrote `AGENTS.md`: definition of the project's agent roles (Code Review Agent, Test Generation Agent, Documentation Agent), description of the agentic workflow, plan for the

automated PR via GitHub Actions + Claude API, evidence log for the professor's criterion 5

- Reviewed and approved `frontend/app.py` (Streamlit scaffold with 4 pages: Questionnaire, Profile Result, Portfolio Dashboard, Chat Advisor)
- Added the `render_profile()` page with `profile_label`, `confidence` and a `top_drivers` placeholder
- Wrote the complete `README.md`: header + badges, project structure, installation, usage flow, API docs (3 endpoints with JSON examples), Technical Highlights table, EU Awareness section, disclaimer, academic documentation section
- Resolved a merge conflict on `backend/data/loader.py` (source: P1's parallel change)
- Fixed ruff F401: removed the unused `from typing import Optional` in `loader.py`
- Opened PR #5 `feature/p4-docs` → `main`, green CI, merge completed

How I did it

- VS Code for direct file editing
- Integrated terminal for `git fetch`, `git merge`, `py_compile`, `pip install ruff`, `ruff check --fix`
- GitHub Desktop / GitHub web for PR management and CI verification
- Used Claude as a technical support tool to verify consistency with design v3.1 and for step-by-step operational guidance

Difficulties

- Merge conflict on `backend/data/loader.py`: resolved keeping P1's version (their file)
- CI failing on an unused import (`typing.Optional`) flagged by ruff: solved with `ruff check --fix`
- `uv` unavailable in the local PATH: solved by activating the `venv` and using `pip install ruff` directly

Achievements / Key decisions

- W1 P4 closed with all deliverables foreseen by design v3.1
- `app.py` already includes HRP/Markowitz tabs, an EU Investor Note placeholder, `session_state` for the profile — structure ready for W2 without refactoring
- `README.md` covers all the professor's minimum requirements (installation, usage, API docs, user guide outline) — to update with the real URL and `docker-compose` once P1 completes them
- PR #5 merged into `main` with green CI: clean and traceable commit history

Next steps

- **W2 (4-10 May):** implement the complete questionnaire UI (7-10 Grable-Lytton questions), profile page with `confidence` and `top_drivers`, portfolio dashboard with weights and base metrics, connection with mock output or P1 API
- Update the `README.md` Docker section once `docker-compose.yml` is ready (P1)
- Verify with P1 that `agent_pr.yml` is planned — the professor's criterion 5, mandatory

Notes for the academic PDF

- The merge-conflict and ruff-linter resolution process is documentable in the Lessons Learned section as a concrete example of a collaborative GitHub workflow with active CI
 - Structuring `app.py` with autonomous mock data (no backend dependency) ensures the frontend is always demonstrable — the “Phase A always works” pattern, consistent with design v3.1
 - Using ruff as a CI-enforced linter guarantees a uniform coding style across the team (the professor’s criterion 4)
-

29 April 2026 — Week 1 (Wednesday)

P2 — Quant / Portfolio Optimization

Estimated duration: ~2 hours

What I did

- Analyzed PR #4 (define `OptimizationResult` interface) and replied to Sabrina’s (P1) comment on the ERC vs BL conflict in the `Literal`
- Corrected `Literal["HRP", "MV", "ERC"]` → `Literal["HRP", "MV", "BL"]` in `hrp.py` before the merge
- Wrote and posted a technical comment on GitHub PR #4 for Sabrina explaining the architectural choice (ERC = internal component, BL = standalone exposed algorithm)
- Merged PR #4 into `main` with a formal description
- Created branch `feature/p2-hrp-optimizer`
- Added the `compute_covariance` stub (Ledoit-Wolf, W1) in `hrp.py`
- Created `tests/test_optimizer.py` with 3 structural tests
- Resolved a CI ruff error (F821 missing imports `np`, `pd`)
- Resolved a CI ruff error (I001 unordered imports)
- Opened PR #5 on `feature/p2-hrp-optimizer`, awaiting review

How I did it

- All work via the GitHub web interface (edit file, commit on branch, PR)
- `compute_covariance` stub with defensive asserts on empty input, NaNs, and minimum number of assets
- Explicit `NotImplementedError` to signal that the real implementation is deferred to W2
- Tests written to cover the interface (`OptimizationResult` fields) and the stub behaviour (`AssertionError` on invalid input, `NotImplementedError` on valid input)
- Lint fix: ruff-compliant import order (`from __future__` → `from typing` → `import numpy` → `import pandas`)

Difficulties

- CI failed twice: first for missing imports (`np`, `pd`), then for ruff-non-compliant import order (I001)

- Risk of committing directly to `main` out of habit — avoided thanks to the branch-protection check

Achievements / Key decisions

- **Architectural decision confirmed:** ERC is an internal component (aggressive tilt + regime fallback), not an exposed algorithm. `Literal["HRP", "MV", "BL"]` is the correct contract for design v3.1
- **W1 P2 complete:** all 3 weekly tasks closed (`universe_config`, `OptimizationResult`, Ledoit-Wolf stub + tests)
- **Dependencies unblocked:** P1 has `OptimizationResult` on `main`, P3 and P4 can start integrating the interface
- **Green CI** on `feature/p2-hrp-optimizer` after the lint fixes

Next steps

- Wait for Sabrina to merge PR #5
- **W2 (from Monday):** implement the real `compute_covariance` with `CovarianceShrinkage(prices).led` from `PyPortfolioOpt`
- W2: complete `hrp.py` with log returns, Ward clustering, recursive bisection, profile tilt, box constraints
- W2: implement `risk_metrics.py` and `markowitz.py`
- W2: add ≥ 3 functional tests in `test_optimizer.py`

Notes for the academic PDF

- **ERC vs BL in the Literal:** the distinction between ERC as an internal component and BL as a standalone algorithm is a documentable architectural choice for the Portfolio Optimization section. ERC requires no μ estimate (consistent with the HRP philosophy), while BL is exposed as an explicit alternative with views derived from the profiler
- **Ledoit-Wolf shrinkage:** the stub is already documented with a reference to Ledoit & Wolf (2004). The academic motivation (reduction of covariance estimation error on finite samples) belongs in Section 3 of the PDF and in the Ledoit-Wolf ADR
- **Defensive assertions:** every public function opens with explicit preconditions — documentable as a software-engineering choice in the Lessons Learned section

P3 — ML / Risk Profiling

Estimated duration: ~1 hour

What I did

- Recovered the full project context at the start of the session: PR #6 state (`rule_based.py`, P1 review pending), IT/EN label conflict resolved and pushed in the previous session
- Produced `progetto_overview_narrativo.md` — an Italian document for personal orientation, useful for the presentation to the professor
- Created the complete `scf_pipeline.py` scaffold with the definitive structure: `load_scf()`, `select_features()`, `standardise_features()`, `build_pipeline()`. English type hints and

docstrings. `load_scf()` is a stub with `NotImplementedError` — real implementation deferred to W2

- Downloaded and inspected `SCFP2022.csv` directly from the Fed to verify the real column names. Discovered that `RISKSCALE` does not exist in the Summary Extract — replaced with `YESFINRISK` and `NOFINRISK`. Also corrected the allocation columns (`CASH` → `CASHLI`, `REAL` removed)
- Translated the whole file to English (docstrings, comments, error messages)
- Wrote `ADR-002-scf-preprocessing.md` in English, documenting 4 decisions: SCF 2022 version, `implicate=1`, feature selection with questionnaire mapping, mandatory use of `WGT`
- Committed and pushed both files on branch `feature/p3-scf-pipeline`
- Opened a PR: “feat: SCF pipeline scaffold + ADR-002 preprocessing decisions” — 3 commits, all checks passed, no conflicts
- Explored the GitHub connector and custom MCP server topic

How I did it

- Used Claude as a technical and academic support tool throughout the session. The flow was collaborative: I drafted the code and documents with Claude’s assistance, then verified the content against the real dataset (downloaded and inspected `SCFP2022.csv` from the Fed), and committed manually from the terminal on iPhone. The `RISKSCALE` correction emerged precisely from direct verification on the file — not from assumptions. I reviewed each choice before committing the code

Difficulties

- Initially did not know where the repo was (wrong terminal directory) — solved with `ls` and `cd robo-advisor`
- The GitHub connector shows as “Connected” in the Claude UI but exposes no interactive MCP tools — Claude cannot navigate the repo autonomously. The manual flow (`cp + git add/commit/push`) works fine anyway
- `RISKSCALE` does not exist in the SCF 2022 Summary Extract: discovered by directly checking the CSV. Corrected before the final commit

Achievements / Key decisions

- W1 fully closed: `scf_pipeline.py` + ADR-002 on a dedicated branch, PR opened and green
- Empirically verified the SCF 2022 dataset: 22,975 rows (4,595 households × 5 imputations), 357 columns. Key columns confirmed: `YESFINRISK`, `NOFINRISK`, `WGT`, `EQUITY`, `BOND`, `CASHLI`, `STOCKS`
- Understood and documented why `WGT` is mandatory: the SCF over-samples wealthy households, each row has a weight representing N real families (e.g. 3027.96 → ~3,028 families). Without `WGT` the model mainly learns from the behaviour of the wealthy
- Discussed the potential of a custom MCP server for Criterion 5 (AI Agents): an MCP server exposing GitHub tools would let Claude open PRs automatically — exactly the agentic workflow the professor wants documented in `AGENTS.md`. To explore next session

Next steps

- Wait for P1’s review on PR #6 (`rule_based.py`) before merging both PRs

- Verify that P1 resolved the IT/EN label conflict in `schema.sql`
- W2 (4–10 May): implement `load_scf()` with the real dataset, `clustering.py` with K-Means/GMM, label assignment on the clusters
- Place `SCFP2022.csv` in the repo's `data/scf/` folder (or handle it via `.gitignore` + README instructions if too large for GitHub)
- Explore building a custom GitHub MCP server next session — useful both for the dev workflow and for Criterion 5

Notes for the academic PDF

- The `implicate=1` choice is a simplification vs Rubin's Rules (5 imputations) — document it honestly in the Limitations section. The motivation is that 4,595 observations are sufficient for a robust GBM and the added complexity is not justified for this scope
 - `RISKSCALE` does not exist in the SCF 2022 Summary Extract. The SCF measures risk attitude via binary variables (`YESFINRISK`, `NOFINRISK`), not a continuous scale. Relevant for the ML section: the questionnaire-to-SCF-feature mapping is not always 1:1 — some variables must be adapted
 - The `WGT` value (e.g. 3027.96) has a concrete interpretation to cite in the PDF: each household in the sample represents thousands of real American families. Using the weights is not optional if the model is to be representative of the population, not just the sample
-

P4 — Frontend / LLM / Docs

Estimated duration: ~1 hour

What I did

- Verified the existence of `docs/` and `docs/adr/` in the local repo
- Verified the content of `frontend/app.py` (already complete with 3 pages + disclaimer)
- Created `docs/architecture.md` with data flow, component boundaries, LLM safety pipeline, failure modes and an ADR table
- Created empty placeholders for ADR-001, ADR-002, ADR-003, ADR-004
- Renamed `ADR-001-db-schema.md` (P1's, empty) to `ADR-005-db-schema.md` to avoid a numbering conflict
- Committed and pushed on `feature/p4-docs`
- Created branch `feature/p4-streamlit-ui` (empty — `app.py` was already on main)
- Opened a PR on `feature/p4-docs` with Sabrina (P1) as reviewer
- Left a note in the PR about the ADR rename

How I did it

- Terminal navigation with `git branch -a`, `ls`, `cat` to inspect the repo state
- Drafted the `architecture.md` content with AI support (Claude) and reviewed it manually
- Decision to differentiate `architecture.md` from `README.md` after directly comparing the two files
- Used `git add`, `git commit`, `git push` from the terminal
- Verified branch and PR state on GitHub

Difficulties

- code unavailable from the terminal (VS Code not in PATH) — worked around by opening files manually from VS Code
- feature/p4-streamlit-ui created but empty because app.py was already on main — no PR opened (no diff)
- First attempt at architecture.md was too similar to the README — rewritten in a complementary way

Achievements / Key decisions

- W1 P4 complete: README, AGENTS.md, app.py scaffold, docs/architecture.md, ADR placeholders
- architecture.md correctly differentiated from the README: it covers internal data flow, component boundaries, LLM safety pipeline, failure modes — content not present in the README
- ADR numbering convention established and communicated to the team via a PR comment
- PR feature/p4-docs opened with P1 as reviewer

Next steps

- Wait for review and merge of PR feature/p4-docs
- W2 (from Monday): complete questionnaire UI, profile page with profile_label / confidence / top_drivers, portfolio dashboard with weights and metrics, connection with mock output or P1 API
- Install code in the PATH to open files from the terminal
- Coordinate ADR numbering and the ADR-005 content with P1

Notes for the academic PDF

- Separating architecture.md from the README reflects a conscious design distinction: README for the external user, architecture for the internal developer. Citable in the Frontend/UX section as an example of structured documentation
 - The Component Boundaries table (architecture.md section 3) is directly reusable in the LLM Narrator section of the PDF to justify the narrator pattern: “the LLM must not create new numbers or recommendations”
 - The ADR-001 → ADR-005 rename and the team communication is a concrete example of agentic coordination documentable in the Lessons Learned section
-

1 May 2026 — Week 1 (Friday)

P4 — Frontend / LLM / Docs

Estimated duration: ~3 hours

What I did

- Researched and analyzed the canonical Ground Truth JSON schema from def_2 v3.1
- Created `backend/schemas/ground_truth.py`: complete Pydantic models for the entire GT JSON (Metadata, Profiler, Portfolio, RiskMetrics, ClusterStructure, StressScenarios, BacktestSummary, LLMConstraints, RegulatoryContext + GroundTruthPayload root)
- Created `backend/schemas/mock_data.py`: factory `get_mock_payload()` with realistic payloads for all 3 profiles (conservative / balanced / aggressive), Phase A compliant
- Created `backend/schemas/__init__.py`: package exports
- Fixed ruff CI errors (import sort + 3 E501 lines in `_BACKTEST`)
- Wrote `docs/adr/ADR-001-hrp-over-markowitz.md` in full: context, decision, 3-phase HRP math, Ledoit-Wolf, Ward linkage, per-profile tilt, guardrails, consequences, alternatives considered, bibliographic references
- Committed and pushed everything on branch `feature/p4-llm-narrator`

How I did it

- Verified alignment with def_2 v3.1 and wrote the schema content directly
- VS Code terminal for git, ruff, python tests
- Pydantic v2 for schema validation (model_validator, Field constraints)
- Manual testing with `python3 -c` to verify invariants (weights sum, UCITS tickers, allowed_numbers)
- GitHub Actions CI for automatic lint verification

Difficulties

- Profile labels: initially generated in Italian (from def_2), corrected to English for consistency with the team codebase
- Validator `currency_exposure_sums_to_one`: initially required $\text{USD} + \text{EUR} = 1.0$, but `CSPX.L` is listed in GBP — relaxed to $\text{USD} + \text{EUR} \leq 1.0$ with an explanatory comment
- Duplicate field `cluster_C_real_assets` in `ClusterStructure`: removed
- CI failed on first push for ruff E501 (long `_BACKTEST` lines) and I001 (unordered imports): fixed and recommitted

Achievements / Key decisions

- `backend/schemas/` is now the **single source of truth** for the Ground Truth JSON — all modules (narrator, validator, frontend) will import from here
- `allowed_numbers` auto-populated by `build_allowed_numbers()`: no manual maintenance of the LLM whitelist
- `expected_annual_return` and `sharpe_ratio` explicitly null with a comment: defensible design choice (HRP does not produce reliable expected returns)
- `RegulatoryContext` with `profiler_us_centric_caveat = True` triggers Rule 9 of the LLM system prompt
- ADR-001 complete and citable in the academic PDF (Portfolio Optimization section)
- W3 task (Ground Truth schema) brought forward by 2 weeks — W2 and W3 start ahead

Next steps

- **W2 (from Monday 4 May):**

- Open PR `feature/p4-llm-narrator` → main and request review
- Align `frontend/app.py` with the new mocks (`get_mock_payload()` instead of hardcoded data)
- Implement the complete questionnaire UI (7-10 questions)
- Profile page with `profile_label`, `confidence`, `top_drivers`
- Portfolio dashboard with weights and base metrics
- Connect the frontend with mock output or P1 API

Notes for the academic PDF

- The `expected_annual_return = null` choice is a deliberate design decision documentable in the Portfolio Optimization section: HRP does not produce reliable point estimates of expected return, and this honesty is explicit in the schema
 - `build_allowed_numbers()` as an automatic whitelist mechanism is citable in the LLM Narrator section as an example of separation of concerns between backend and LLM
 - The `currency_exposure` validator fix for CSPX.L (GBP-listed) is a concrete example of the EU/US tension documentable in the Limitations section
 - ADR-001 contains the complete HRP math (3 phases, Ledoit-Wolf, Ward linkage) — usable directly as a basis for the Portfolio Optimization section of the PDF
-

4 May 2026 — Week 2 (Monday)

P1 — Backend / Data Engineering

Estimated duration: ~2 hours

What I did

- Fixed `schema.sql` naming: replaced Italian labels (`Conservativo`, `Bilanciato`, `Aggressivo`) with EN UPPER (`CONSERVATIVE`, `MODERATE`, `AGGRESSIVE`) — aligned with the canonical contract decided by P3
- Post-hoc review of P3's `rule_based.py` (PR #6, already merged): verified importability, EN UPPER labels, Q7 MiFID II override, `ProfilerOutput` schema, no circular imports
- Created `backend/api/main.py` with the FastAPI app:
 - `/profile` endpoint: accepts questionnaire JSON, calls `rule_based.profile_user()`, returns `ProfilerOutput` as JSON
 - Pydantic request/response models with complete type hints
 - Rate limiting with `slowapi`: 20 requests/minute
 - `Profiler ValueError` mapped to HTTP 422
 - ruff I001 lint fix (import order)
- Created `tests/test_api.py` with 9 integration tests for `/profile`:
 - Happy path: `CONSERVATIVE`, `MODERATE`, `AGGRESSIVE`
 - Q7 MiFID II hard override
 - Complete response schema
 - Borderline confidence (`score 9` → `confidence=0.7`, `low_confidence_flag=True`)
 - Error handling: missing key, invalid letter, empty answer
 - Fix: removed unused `import pytest` (ruff F401)
 - Fix: `Q7='b'` in the borderline test to avoid conflict with the MiFID II override

- Wrote docs/adr/ADR-005-db-schema.md: SQLite vs PostgreSQL rationale, v3.1 schema with key-field explanations, limitations, alternatives considered

How I did it

- Everything on github.dev (browser) — zero local environment
- One PR per logical unit of work: branch → commit → green CI → merge
- Iterative CI fixes: I001 import order, F401 unused import, E999 syntax error (comment inside def)
- For the borderline test: reasoned manually over the SCORE_MAP in rule_based.py to find an answer set producing exactly score=9 without triggering the Q7 override

Difficulties

- **Ruff I001** on main.py: slowapi.errors / slowapi.util import order inverted + ProfilerOutput, profile_user not alphabetical → immediate fix
- **Ruff F401** on test_api.py: import pytest present but unused → removed
- **Syntax error** test_api.py: comment added between def and docstring → moved above the function
- **Borderline test failed:** I used _all_responses("a") which set Q7='a', triggering the MiFID II override and forcing confidence=1.0 → changed Q7='b' to avoid the conflict
- /optimize **stub**: decided not to commit the stub because P2 (Emma) is writing the optimizer directly — parallel work, no blocking

Achievements / Key decisions

- /profile endpoint live on main, green CI
- 9 integration tests in test_api.py, green CI
- ADR-005 written and merged — W2 academic documentation completed
- Decision: /optimize stub not committed — Emma handles hrp.py directly, P1 wires the endpoint as soon as run_hrp() is available
- Pattern used: “stub first, wire later” documented in the main.py comment for the professor

Next steps

- Wire /optimize as soon as Emma merges run_hrp() in hrp.py (expected by Tuesday W2)
- Verify end-to-end DB insert after wiring /optimize (snapshots.py already ready)
- agent_pr.yml stub — recommended to open a branch by end of week

Notes for the academic PDF

- The “validate at the boundary” pattern used in main.py (Pydantic + ValueError → 422) is a concrete example of defensive API design — useful for the backend architecture section
- The borderline confidence test documents that the profiler’s uncertainty signal is correctly propagated up to the HTTP layer — good example for the ML Risk Profiler section
- ADR-005 is directly reusable in the infrastructure section of the PDF: SQLite rationale, v3.1 fields, honest limitations

- The comment on the `/optimize` stub explains the parallel development workflow between P1 and P2 — material for the Lessons Learned section
-

P2 — Quant / Portfolio Optimization

Estimated duration: ~2 hours

What I did

- Implemented the real `compute_covariance` with `CovarianceShrinkage(prices).ledoit_wolf()` from `PyPortfolioOpt` (removed the `W1 NotImplementedError`)
- Added a PSD check on the eigenvalues of the covariance matrix
- Implemented `compute_log_returns` with defensive assertions
- Implemented the three clustering functions: `_cov_to_corr`, `_corr_to_distance`, `_get_quasi_diagonal_order`
- Implemented `_get_cluster_variance` and `_recursive_bisection` (López de Prado, 2016)
- Implemented profile tilt: `_compute_min_var_weights` (CONSERVATIVE), `_compute_erc_weights` (AGGRESSIVE), `_apply_profile_tilt`
- Implemented `_apply_box_constraints` with an iterative clip-renormalize loop (10 iterations)
- Implemented the `optimize()` entry point that chains the whole pipeline
- Updated `OptimizationResult`: removed `Optional` from `expected_return` and `sharpe_ratio`, updated `solver_status` literals
- Fixed CI: removed unused `Optional` (ruff F401), corrected import order (ruff I001)
- Updated `test_optimizer.py`: replaced the obsolete `W1` test (`NotImplementedError`) with a functional `W2` test
- Opened PR `feature/p2-hrp-optimizer-1` toward `main` with reviewer `Sabrina15072002`
- Green CI

How I did it

- All work via the GitHub browser (online editor, direct commits on the branch)
- Pipeline built piece by piece with granular commits for each section
- Ledoit-Wolf via `pypfopt.CovarianceShrinkage` — handles log returns and annualization internally
- HRP distance: $D(i,j) = \sqrt{0.5 * (1 - _LW(i,j))}$
- Clustering: Ward linkage via `scipy.cluster.hierarchy.linkage`
- Recursive bisection: weight allocation inversely proportional to cluster variance (IVP)
- Profile tilt: 70/30 blend between HRP and MinVar (CONSERVATIVE) or approximated ERC (AGGRESSIVE)
- Box constraints: iterative asset-level clip (0.03-0.40) and cluster-level (0.10-0.60)
- Final metrics: annualized volatility $\sqrt{w' \Sigma w}$, expected return $\bar{r} \times 252$, Sharpe, risk contributions $(w_i \times (\Sigma w)_i) / (w' \Sigma w)$

Difficulties

- Accidental commit on `main` on the first attempt — corrected by choosing “Create new branch” in the commit dialog

- GitHub created `feature/p2-hrp-optimizer-1` instead of `feature/p2-hrp-optimizer` (branch already existed) — no functional issue
- CI failed twice: first unused `Optional` (F401), then import order (I001) — both fixed quickly
- The W1 test `test_compute_covariance_raises_not_implemented_on_valid_input` was obsolete after the W2 implementation — replaced with a functional test

Achievements / Key decisions

- `hrp.py` **complete** — W2 task 1 milestone closed
- **PR opened** toward main, green CI, reviewer assigned
- **Decision confirmed:** the aggressive tilt uses ERC (not Max Sharpe) to avoid dependence on μ — consistent with the HRP philosophy
- **Decision confirmed:** `solver_status = "clipped"` when box constraints modify the pure HRP weights — traceable by P4 and the narrator
- **Dependencies unblocked:** P1 can implement the `/optimize` endpoint, P4 can integrate `OptimizationResult` into the narrator

Next steps

- Wait for the PR merge from Sabrina (P1)
- Implement `backend/optimizer/risk_metrics.py` (risk contributions, ex-ante volatility, expected return, Sharpe)
- Implement `backend/optimizer/markowitz.py` (MV Max Sharpe benchmark)
- Add ≥ 3 functional tests in `test_optimizer.py` (weights sum to 1.0, constraints respected, risk contributions sum to 1.0)

Notes for the academic PDF

- **Ledoit-Wolf mandatory:** the empirical covariance matrix with 8 assets and ~ 1260 observations is unstable. LW shrinkage reduces the estimation error by pulling Σ toward a structured target. Citation: Ledoit & Wolf (2004), “A well-conditioned estimator for large-dimensional covariance matrices.”
 - **HRP vs Markowitz:** HRP never inverts $\Sigma \rightarrow$ guaranteed numerical stability. Markowitz requires Σ^{-1} which amplifies off-diagonal errors.
 - **Profile tilt without γ :** HRP has no explicit risk-aversion parameter. The 70/30 blend with MinVar or ERC is the mathematically defensible way to introduce profile dependence.
 - **ERC for AGGRESSIVE:** choice motivated by the absence of dependence on μ — consistent with the HRP philosophy of avoiding expected-return estimates.
-

P4 — Frontend / LLM / Docs

Estimated duration: ~ 1 hour

What I did

- Identified a missing W2 Mon-Tue task: `docs/ground_truth_schema.md` had not been created (only `backend/schemas/ground_truth.py` existed)

- Created docs/ground_truth_schema.md complete with all the fields required by the dev plan:
 - portfolio.weights, risk_contributions
 - profiler.profile_label, profile_confidence, top_drivers
 - regulatory_context: profiler_us_centric_caveat, mifid_disclaimer, currency_risk_note, etf_ucits_eligible, hfcs_note
 - llm_constraints, stress_scenarios, backtest_summary, cluster_structure
 - 8 Pydantic validation invariants
 - Usage table per component (Narrator, Validator, Streamlit, mock factory)
- Verified alignment between the .md and backend/schemas/ground_truth.py via grep — all 4 new fields present in the Pydantic model
- Committed and pushed on feature/p4-llm-narrator
- Opened a PR with a structured description, P1 assigned as reviewer
- Added a License section to README.md

How I did it

- Verified alignment with def_2 v3.1 and the dev plan PDF; wrote the content directly
- Terminal for grep, cp, git add/commit/push
- Direct comparison with the dev plan screenshot to verify the mandatory fields (mifid_disclaimer, currency_risk_note, etf_ucits_eligible, hfcs_note)

Difficulties

- The first version of the .md was missing 4 mandatory regulatory_context fields — identified by comparing with the dev plan PDF
- Committed on feature/p4-llm-narrator instead of feature/p4-docs (branch already in use for this session — not blocking, but to keep in mind for the PR)

Achievements / Key decisions

- docs/ground_truth_schema.md is now the **readable interface contract** between backend (P1/P2/P3) and frontend/LLM (P4) — every component consuming the GT payload has a documented reference
- Schema doc and Pydantic model verified as aligned
- W2 Mon-Tue **docs** task completed
- README.md updated with a License section

Next steps

- cat frontend/app.py to see the current state of the scaffold
- Implement the complete questionnaire UI (10 Grable-Lytton questions, st.form)
- Profile page with profile_label, confidence, top_drivers from get_mock_payload()
- Portfolio dashboard with HRP / Markowitz tabs, UCITS badge, EU Investor Note, stress banner
- Chat Advisor placeholder UI

Notes for the academic PDF

- docs/ground_truth_schema.md is directly citable in the **LLM Narrator** section (Section 4) as the specification of the contract between the numerical backend and the narrative layer
 - Documenting `expected_annual_return = null` as a deliberate design decision (HRP does not produce reliable expected-return estimates) is an academic strength — cite it explicitly in the Portfolio Optimization section
 - The 4 EU fields (`mifid_disclaimer`, `currency_risk_note`, `etf_ucits_eligible`, `hfcs_note`) are the basis of the **EU Awareness / Limitations** section of the PDF
-

5 May 2026 — Week 2

P1 — Backend / Data Engineering

Estimated duration: ~2 hours

What I did

- Reviewed Emma's (P2) `hrp.py`: identified 3 divergences from the v3.1 contract
 - `BALANCED` → `MODERATE` (blocking) — resolved by Emma before the wire
 - `expected_return/sharpe_ratio` still float instead of `None` (non-blocking)
 - `ASSET_MIN = 0.03` in `hrp.py` vs `ASSET_WEIGHT_MIN = 0.05` in `universe_config.py` (non-blocking)
- Opened a GitHub issue to Emma with the three problems documented
- Reviewed Matteo's (P3) `test_profiler.py`: approved with one fix
 - Removed the duplicate `[dependency-groups]` in `pyproject.toml` that created a conflict with `[project.optional-dependencies]` (two different ruff versions)
- Wired the `/optimize` endpoint in `backend/api/main.py`:
 - Resolves tickers via `get_primary_tickers()` or override from the request
 - Loads prices via `ValidatedDataLoader` with UCITS fallback
 - Calls P2's `optimize()` (HRP + Ledoit-Wolf + profile tilt + box constraints)
 - Persists the result in the DB via `snapshots.py` (`save_market_snapshot` + `save_recommendation`)
 - DB failure does not block the response — log a warning and continue
 - Added `OptimizeRequest` and `OptimizeResponse` Pydantic models
- Resolved 2 CI ruff errors: I001 (import order) and F401 (unused logging)
- Committed ADR-003 cloud deploy on `feature/p1-docs`
- Wrote a comment to Emma about the residual problems in `hrp.py`

How I did it

- Everything on `github.com` and `github.dev` (browser)
- Recurring problem: branches created from old branches instead of from main → 83 commits behind. Solved by deleting the wrong branches and recreating them from main on `github.com`
- Iterative ruff fixes: import order first, then unused logging
- Matteo's PR review: identified the `toml` duplication, fixed directly on his branch

Difficulties

- **Branch 83 commits behind:** github.dev created new branches from the current branch instead of from main. Solved by working directly on github.com for branch creation
- **Ruff I001:** unordered imports (stdlib → third party → first party, all alphabetical). To keep in mind for the next files
- **Ruff F401:** `import logging` at module level flagged as unused because used inside an except block. Solved with a local import inside the block

Achievements / Key decisions

- `/optimize` endpoint live on main — complete pipeline: HTTP → DataLoader → HRP → DB
- DB audit trail working end-to-end (was planned Fri-Sun W2, done on Tuesday)
- ADR-003 merged — W3 documentation brought forward
- P3 PR review approved with the `pyproject.toml` fix
- Decision: a DB failure does not block the `/optimize` response — the user always receives the portfolio, the DB is best-effort
- P2 divergences documented and flagged via issue — to be resolved in W3

Next steps

- Wait for Emma's fix (`ASSET_MIN` and `expected_return/sharpe_ratio`) — W3
- `agent_pr.yml` stub — could be brought forward this week if there is time
- W3: complete integration test suite, verify endpoints with real yfinance data

Notes for the academic PDF

- The “DB failure does not block the response” pattern is a documentable architectural choice: availability over consistency for an academic prototype
 - The 83-commits-behind branch problem is a concrete example of a collaborative Git workflow — useful for the Lessons Learned section
 - The review of Matteo's PR with the `pyproject.toml` fix demonstrates the cross-team code review process — good example for the agentic process section
 - Wiring `/optimize` completes the P1→P2 loop: ValidatedDataLoader (P1) + HRP optimizer (P2) + DB audit trail (P1) — pipeline documentable in the architecture section
-

P4 — Frontend / LLM / Docs

Estimated duration: ~3 hours

What I did

- Implemented `backend/llm/prompts/system_prompt.py`: the system prompt template with all 9 rules of design v3.1 (including Rule 9 EU Awareness), the `MANDATORY_DISCLAIMER` constant, the `build_system_prompt()` function
- Implemented `backend/llm/narrator.py`: `NarratorClient` — complete scaffold of the LLM API client with Layer 1 injection defence, error handling, SHA-256 audit hashes, `NarratorResponse` and `NarratorError` dataclasses

- Created `backend/llm/prompts/__init__.py`
- Corrected the system prompt language setting: removed the hardcoded “Italian”, replaced with language-adaptive output (“respond in the same language the user writes in”)
- All files written entirely in English (comments, docstrings, prompt rules, fallback messages)
- Installed `anthropic` in the local venv (`pip install anthropic`)
- Ran a manual functional test: `build_system_prompt`, injection detection, SHA-256 hash — all passed
- Ran `pytest tests/` — 1 test passed, green CI
- Committed and pushed on branch `feature/p4-llm-narrator`
- Updated the Streamlit questionnaire (`frontend/app.py`): replaced the W1 mock questionnaire with the complete 10-question Grable & Lytton (1999) adapted form, divided into 3 sections (Who You Are Financially / How You Invest / How You React), with `_compute_profile()` scoring logic, Q7 MiFID II hard override, borderline confidence zones, top drivers computation
- Dashboard updated: reads the profile from `session_state["profile"]` instead of hard-coded data

How I did it

- Verified alignment with `def_2 v3.1`; designed and wrote the code directly
- VS Code for file editing (manual creation via Explorer, paste content)
- VS Code integrated terminal for git, pip, python, pytest, ruff
- Functional test with a temporary `test_narrator_temp.py` file (then removed) to work around the zsh terminal limit with multiline strings
- GitHub for push and PR opening

Difficulties

- `uv` not installed on the Mac → solved by using `python` and `pip` directly in the `.venv`
- `code` command not in PATH on Mac → solved by creating files directly from the VS Code Explorer
- `zsh: parse error near ')'` on the multiline test pasted in the terminal → solved by creating a temporary `.py` file in the project folder
- `ModuleNotFoundError: No module named 'backend'` running the test from `/tmp/` → solved by creating the file in the project root
- System prompt language initially in Italian out of inertia from the design doc → corrected to English with language-adaptive output
- `pytest` not installed in the venv → `pip install pytest`

Achievements / Key decisions

- `NarratorClient` is stateless by design: no conversation history, the Ground Truth JSON re-injected on every call — guarantees the LLM is always anchored to the current data
- `temperature=0.0` for deterministic and auditable output
- `MANDATORY_DISCLAIMER` as a shared constant between `narrator.py` and `validator.py` (W3): a single point of truth, no risk of drift between the two modules
- Layer 1 injection defence implemented before the API call: length check (800 chars) + pattern matching on 14 known patterns

- SHA-256 hash of the system prompt and the GT JSON in `NarratorResponse` → ready for the DB audit trail in W3
- Language-adaptive output: the LLM responds in the user's language, not forced Italian
- `_compute_profile()` in `app.py` has an output schema identical to the future GBM Phase B — no downstream changes needed when the ML is integrated in W3

Next steps

- **W2 Thu-Fri (this week):**
 - Open PR `feature/p4-llm-narrator` → main with a complete description
 - Streamlit profile page with `profile_label`, `confidence`, `top_drivers` displayed correctly
 - Connect the frontend with `get_mock_payload()` for the portfolio dashboard
 - Disclaimer UI above every financial output
- **W3 (next week):**
 - `backend/llm/validator.py` — 4-step validator (forbidden phrases, number check, disclaimer, semantic injection)
 - `tests/test_validator.py`
 - Wiring `narrator.py` + `validator.py` into the FastAPI `/advice` endpoint
 - Chat Advisor page connected to the backend

Notes for the academic PDF

- The “Narrator, not Calculator” pattern is implemented and documentable: the LLM never calculates, it only narrates the backend results — a clean separation between the computational layer and the narrative layer
 - The `temperature=0.0` choice is academically defensible: deterministic output = reproducible = auditable, an explicit requirement of the audit trail
 - Rule 9 EU Awareness in the system prompt is a concrete example of how regulatory constraints (MiFID II + US/EU SCF data gap) are implemented at the prompt-engineering level — citable in the LLM Narrator section of the PDF
 - The Layer 1 injection defence (pre-call) + Layer 2 semantic (Validator W3) is a two-level security pipeline documentable in the “Prompt Injection Defense” section
 - The decision to make the output language adaptive (not forced Italian) is a reasoned UX choice: the system is an international academic prototype, not an Italian product
-

6 May 2026 — Week 2

P4 — Frontend / LLM / Docs

Estimated duration: 30 minutes

What I did

- Received and analyzed the updated `ground_truth.py` file with the changes already applied
- Verified the fix locally via a quick sanity check (`python3 -c ...`)
- Confirmed correct output: `expected_annual_return = 0.068`, `sharpe_ratio = 0.71`
- Prepared a response to Sabrina's (P2) comment on issue #28

- Prepared the title and description for the Pull Request on `feature/p4-llm-narrator`

How I did it

- Local verification with the `.venv` virtual environment activated
- Sanity-check output compared with the expected values from `mock_data.py`
- Issue response and PR description drafted directly

Difficulties

- No technical difficulty in the session
- The `cd: no such file or directory` in the output was harmless (already in the correct directory)

Achievements / Key decisions

- **Issue #28 fix completed and verified:** `RiskMetrics` now accepts `Optional[float]` for `expected_annual_return` and `sharpe_ratio`
- The values are historical log averages, not forward-looking estimates — consistent with the HRP design and academically defensible
- `system_prompt.py` Rule 5 updated accordingly
- PR ready for review with `Closes #28`

Next steps

- Open the PR on GitHub and assign review (Emma/P2 or Sabrina/P1)
- Proceed with the remaining W2 tasks:
 - Streamlit profile page (`profile_label`, `confidence`, `top_drivers`)
 - Portfolio dashboard with weights and base metrics
 - Disclaimer UI above every financial output
 - Chat Advisor placeholder

Notes for the academic PDF

- The choice to use historical log averages as a proxy for `expected_return` in HRP is a defensible decision: HRP does not depend on estimated μ , but the historical value remains informative for the end user
- The “historical average $\neq$ forecast” pattern is explicitly encoded in the system prompt (Rule 5) — citable in the LLM Narrator section of the PDF as an example of a semantic guardrail
- The `allowed_numbers` field in `LLMConstraints` auto-populates by including these values, closing the backend $\rightarrow$ LLM $\rightarrow$ validator loop without manual intervention

7 May 2026 — Week 2

P2 — Quant / Portfolio Optimization (session 1)

Estimated duration: ~1.5h

What I did

- Reviewed the project specification → identified 3 errors/inconsistencies in the project files
- Analyzed Sabrina's (P1) technical message reporting two bugs found while wiring `/optimize`
- Defined the precise specification of the two fixes and the required tests
- Verified the first round of work (fixes applied, existing tests pass) and identified a gap: 2 regression tests and the docstring on `OptimizationResult` were missing
- Completed the missing work in a second round
- Verified the final output: 5/5 tests pass, complete fix
- Defined the commit and PR strategy with a technical description ready

How I did it

- **Approach:** two deliberate rounds — first round for the core fixes, second round explicitly for the gaps (regression tests + docstring). This allowed verifying the work between the two rounds instead of relying on a single unverifiable pass
- **Pattern used for the tests:**
 - `test_hrp_returns_none_for_mu_dependent_metrics`: uses `typing.get_type_hints() + get_args()` — a contract test that fails if the annotation reverts to bare `float`
 - `test_hrp_uses_universe_config_box_constraints`: synthetic prices 252d, runtime verification that all weights are in `[ASSET_WEIGHT_MIN, ASSET_WEIGHT_MAX]` imported from `universe_config`
- **Single source of truth applied:** removed the 4 local constants from `hrp.py`, imported them from `universe_config.py` — so the value lives in one place only and can no longer drift

Difficulties

- In the first round the existing tests passed, but the two new tests explicitly required were not added — a corrective second round was needed
- Inconsistency in the project files (0.03 vs 0.05) present in multiple documents: the P0 checklist of `versione 2-` still reports 0.03 while the real code, the session logs and the Ground Truth JSON use 0.05 — this documentation inconsistency was not resolved in the spec files (only in the code)

Achievements / Key decisions

- **Bug fix brought forward (W2 on W3-flagged issues):** both bugs reported by Sabrina closed ahead of the planned week — buffer gained for W3
- **Single source of truth on box constraints:** `hrp.py` now imports from `universe_config.py` instead of hardcoding. Pattern replicable in `markowitz.py` when implemented
- `OptimizationResult` **contract formally correct:** `expected_return: float | None` and `sharpe_ratio: float | None` reflect the model math — HRP does not estimate μ (López de Prado, 2016). The docstring makes this choice explicit in the code
- **Regression guards installed:** the two new tests would block an accidental reintroduction of `ASSET_MIN = 0.03` or of bare `float` in the μ -dependent fields
- **3 errors found in the project specification:**
 1. `sklearn` → `PyPortfolioOpt` (critical error on the library to use for Ledoit-Wolf)

2. Box constraint 0.03 → 0.05 (internal inconsistency in the project docs)
3. markowitz.py commit message: “Max Sharpe” imprecise if the MV formulation is not yet constrained

Next steps

- Verify lint locally (ruff check backend/ tests/) before pushing — in previous sessions CI failed twice for ruff
- Commit on branch feature/p2-hrp-optimizer (or fix/p2-hrp-contract-alignment if merged) with msg: fix: align ASSET_MIN with universe_config and make return metrics Optional
- Open a PR on GitHub toward main, technical description (ready), review request to Sabrina (P1 — found the bugs, natural approver)
- **Remaining W2 tasks:**
 - Implement the real compute_covariance in hrp.py (Ledoit-Wolf with CovarianceShrinkage(prices) from PyPortfolioOpt)
 - Complete HRP: log returns, Ward clustering, recursive bisection, profile tilt
 - Implement risk_metrics.py (risk contributions, ex-ante vol, expected return, Sharpe)
 - Implement markowitz.py as the MV benchmark
 - Add ≥3 functional tests in test_optimizer.py

Notes for the academic PDF

- **Section 3 — Portfolio Optimization:** typing expected_return and sharpe_ratio as Optional[float] in the OptimizationResult interface is not just an engineering choice — it directly reflects the theory. HRP is a covariance-only algorithm: it does not require estimating μ , which is the most unstable source of error in Markowitz (Michaud, 1989). The field type documents this asymmetry explicitly and verifiably from the tests.
- **Single source of truth as an architectural pattern:** importing the box constraints from universe_config.py instead of replicating them is citable in the Coding Structure section (Criterion 4) as a deliberate design choice to prevent cross-module inconsistencies.
- **The history of this PR** (bug reported in cross-team review → fix with cited theoretical rationale → regression test → single source of truth refactor) is a direct example of “Process over Product” citable in Section 7 (Lessons Learned).
- **Documentation inconsistency to resolve:** the P0 checklist in version 2- smart single portfolio still reports 0.03 as the lower bound of the box constraint. If the PDF cites the checklist, the value must be aligned to 0.05. To be fixed before submission.

P2 — Quant / Portfolio Optimization (session 2)

Estimated duration: ~2 hours

What I did

- Verified the GitHub state: 4 open PRs, all green CI

- Identified that the complete `hrp.py` was already present on branch `feature/p2-hrp-optimizer-1`
- Identified a double-annualization bug of volatility between `hrp.py` and `risk_metrics.py`
- Applied a fix in `hrp.py`: added `frequency=1` to `CovarianceShrinkage` and explicit `* 252` in `optimize()`
- Opened PR `fix/p2-covariance-frequency` (or `feature/p2-hrp-optimizer-1`) with Elena as reviewer
- Wrote 3 functional tests in `tests/test_optimizer.py`:
 - `test_optimize_weights_sum_to_one_and_box_constraints`
 - `test_optimize_profile_tilt_produces_different_weights`
 - `test_optimize_annual_volatility_in_realistic_range`
- Resolved a test failure: missing `_make_prices()` and a uniform fixture in the tilt test
- Green CI on all tests (6 structural W1 + 3 functional W2 = 9 tests total)
- Analyzed Elena's PR #32 (`RiskMetrics Optional[float]`): no changes needed to `hrp.py`

How I did it

- All work via the GitHub browser editor (edit, commit, PR)
- Volatility bug identified by comparing `CovarianceShrinkage` default (`frequency=252`) with the `* TRADING_DAYS_PER_YEAR` in `risk_metrics.py`
- Test fixture `_make_prices_with_varied_vol` introduced for the tilt test: heterogeneous volatilities per asset class (equity ~1.5%, bonds ~0.4%, cash ~0.1%) needed so that MinVar and ERC produce different weights
- Fixture `_make_prices` for generic tests (uniform volatility, 252 days)

Difficulties

- Test `test_optimize_profile_tilt_produces_different_weights` failed with `max(diffs) = 0.0`: with uniform volatility across all assets, $\text{MinVar} \approx \text{ERC} \approx \text{HRP} \rightarrow$ tilt invisible. Solved with the heterogeneous-volatility fixture.
- The `test_optimizer.py` file ended up with duplicates and the `_make_prices` function removed by mistake during edits. Solved iteratively.
- Double assignment `prices = prices = ...` introduced by mistake during manual editing. Corrected.

Achievements / Key decisions

- **W2 P2 completed**: all deliverables written, tested, on PR with green CI
- **Volatility bug fixed**: `CovarianceShrinkage(prices, frequency=1) + explicit * 252` in `optimize()`. Without this fix the volatility would be inflated by a factor of $\sqrt{252} \approx 15.87x$ in the Ground Truth JSON
- **9 total tests** in `test_optimizer.py` (3 structural W1 + 3 functional W2 + 3 already present)
- **Open PRs with green CI**: `feature/p2-hrp-optimizer-1` (hrp fix + tests), #25 (`risk_metrics`), #27 (`markowitz`), #32 (Elena fix `Optional[float]`)
- **Decision**: `_compute_erc_weights` uses inverse volatility weighting as an approximation of ERC — academically defensible (avoids dependence on μ , consistent with the HRP philosophy)

Next steps

- Wait for Elena's review on PR `feature/p2-hrp-optimizer-1` and merge
- Merge PR #25 (`risk_metrics`), #27 (`markowitz`), #32 (Elena)
- W3: implement `backtest.py` on 3 scenarios (GFC 2008, COVID 2020, Rate Hike 2022) with 10 bps transaction cost
- W3: add a `regime_detector.py` scaffold with $VIX > 30$ threshold logic
- W3: export backtest results to JSON

Notes for the academic PDF

- **Covariance frequency bug:** worth a note in the Portfolio Optimization or Lessons Learned section — it documents that the `frequency=1` choice is not arbitrary but necessary for consistency with the risk-metrics layer that annualizes explicitly
 - **ERC approximation:** `_compute_erc_weights` uses inverse volatility (normalized $1/\sigma_i$) instead of true ERC (numerical optimization). It is a defensible simplification: it produces weights inversely proportional to risk without depending on μ , in line with the HRP philosophy. Cite Maillard et al. (2010) and document the approximation
 - **Tests as documentation:** the 3 functional tests encode the quantitative requirements of the system (weights sum = 1, box constraints, realistic vol range) — mentionable in the Coding Style section as an example of defensive testing
-

8 May 2026 — Week 2

P4 — Frontend / LLM / Docs

Estimated duration: ~1 hour

What I did

- Updated `_QUESTIONS` in `frontend/app.py` with the questionnaire questions
- Created `docs/report.tex`: complete LaTeX skeleton with all 8 sections foreseen by the dev plan (Introduction, ML Risk Profiler, Portfolio Optimisation, LLM Narrator, Backtest Results, Limitations, Lessons Learned, Conclusions)
- Created `docs/references.bib` with the 4 fundamental citations of the project (López de Prado 2016, Ledoit-Wolf 2004, Fed SCF 2022, MiFID II)
- Set up the LaTeX preamble with all the necessary packages (`amsmath`, `booktabs`, `biblatex/biber`, `listings`, `hyperref`)
- Structured Section 4 (LLM Narrator) with already-populated subsections: Narrator pattern, Ground Truth JSON contract, System Prompt rules, Validator 4-step pipeline, Prompt Injection Defence
- Inserted the backtest table (Section 5) with a structure ready for P2's numbers
- Inserted explicit `% TODO` markers for every P2/P3 section, so they can write autonomously
- Defined the compilation workflow: `pdflatex → biber → pdflatex × 2`

How I did it

- Verified alignment with the dev plan and design v3.1; wrote the LaTeX structure directly

- Comparison with the dev plan PDF screenshot (W4 Wed-Thu task list) to cover all required sections
- Section 4 content derived from: `docs/ground_truth_schema.md`, `system_prompt.py`, the 4-step validator plan

Difficulties

- No relevant technical difficulty in the session

Achievements / Key decisions

- **LaTeX skeleton completed in W2** — tactical advantage: P2 and P3 already have the file and the marked `% TODOs`, they can write their sections in parallel during W3 without waiting for W4
- Section 4 (LLM Narrator) is already sketched with the right content: Ground Truth JSON contract, system prompt rules, 4-step validator — in W4 it will only need expanding, not building from scratch
- `references.bib` with López de Prado and Ledoit-Wolf already cited correctly in the text — no placeholders to search for in a hurry at the last minute

Next steps

- Tell P2 and P3 that `docs/report.tex` exists and their `% TODOs` are in sections 2 and 3
- Remaining W2: profile page (`profile_label`, `confidence`, `top_drivers`), portfolio dashboard, disclaimer UI, Chat Advisor placeholder
- W3: implement `narrator.py` and `validator.py` — Section 4 of the LaTeX populates almost automatically
- W4 Wed-Thu: complete the `% TODOs` of Sections 4, 6, 7, 8 and integrate the P2/P3 sections

Notes for the academic PDF

- The choice to create the skeleton in W2 (instead of W4) is documentable in the Lessons Learned section as an example of proactive dependency management: documentation was not left to the last minute
- The structure of Section 4 — clean separation between “narrator” and “calculator” — is the central architectural choice of the entire LLM layer; worth a dedicated subsection in the Lessons Learned section as well as in Section 4 itself
- The 4 citations in `references.bib` cover the two Advanced algorithmic choices (HRP, Ledoit-Wolf) and the two regulatory constraints (SCF US-centrism, MiFID II) — direct alignment with criteria 1 and 3 of the professor

9 May 2026 — Week 2 (Saturday)

P4 — Frontend / LLM / Docs

Estimated duration: ~1.5 hours

What I did

- Verified W1 and W2 completeness against the dev plan — confirmed everything substantially closed
- Examined 5 open PRs on GitHub (all green CI): identified #32 as the review priority for P4 (fix RiskMetrics Optional[float])
- Created `backend/llm/validator.py` — 4-step post-generation pipeline:
 - Step 1: forbidden phrases check (case-insensitive)
 - Step 2: hallucinated numbers check with 2% tolerance, percentage normalization, exclusion of narrative integers
 - Step 3: disclaimer auto-append (corrective, non-blocking)
 - Step 4: semantic injection detection post-generation
- Added a NOTE: "safe" false positive comment in `backend/schemas/mock_data.py`
- Created `tests/test_validator.py` — 27 tests, all green
- Resolved 8 initial failures myself (logic of `_extract_numbers`: percentages now normalized to decimal, not duplicated)
- Committed and pushed on `feature/p4-llm-narrator`

How I did it

- Designed the validator and the test structure directly
- VS Code for file editing
- Terminal with the venv active (`python -m pytest`, `python -m ruff`) — `uv` not available in the local PATH
- Independent debugging of the failures: read the pytest output, identified the problem in `_extract_numbers`, corrected it in the validator

Difficulties

- `uv` not available in PATH with the venv active — solved by using `python -m pytest` and `python -m ruff check .` directly
- 8 tests failed on the first run because the `_extract_numbers` logic produced both the raw value (35.0) and the normalized one (0.35) for percentages — solved by removing the double append and keeping only the decimal form
- `test_decimal_not_in_allowed_numbers_is_blocked`: 0.99 was not being blocked because the `abs(n) <= 10` check was not precise enough — solved with `float(n).is_integer()` and `abs(n) <= 10`

Achievements / Key decisions

- `validator.py` complete and tested — working LLM safety layer
- 27/27 tests green in `test_validator.py` — coverage of all 4 steps
- Documented decision: "safe" remains in the forbidden list with a known-limitation note (false positive on "safe haven") — acceptable for an academic prototype, to be documented in ADR-004
- Validator pipeline ready to be wired into the `/advice` endpoint (next step Tuesday)

Next steps

- **Tuesday:** wiring /advice in backend/api/main.py (replace the 503 stub with Narrator-Client + validate())
- **Tuesday:** Chat Advisor in frontend/app.py connected to the backend
- **Wednesday:** docs/adr/ADR-004-llm-narrator-validator.md
- **Wednesday/Thursday:** docs/user_guide.md
- **Thursday:** PR feature/p4-llm-narrator → main, review request to P1
- Review PR #32 (RiskMetrics Optional[float]) — priority this week

Notes for the academic PDF

- The 4-step pipeline is documentable in Section 4 (LLM Narrator) as a concrete example of safety by design: every LLM response mandatorily passes through the validator before being shown to the user
 - The “safe” → false positive on “safe haven” case is a real example of the trade-off between security and usability — citable in the Limitations section with the adopted solution (accepted known limitation, documented)
 - The choice to make Step 3 corrective (auto-append) instead of blocking is a defensible architectural decision: the disclaimer is too important to block the response, better to always guarantee its presence
 - The `float(n).is_integer()` fix is an example of a real edge case discovered during testing — citable in the Lessons Learned section as an example of test-driven debugging
-

11 May 2026 — Week 3 (Monday)

P3 — ML / Risk Profiling

Estimated duration: 1h30

What I did

- Reviewed and consolidated my understanding of the SCF → clustering → GBM flow: clarified that the clustering produces the “synthetic ground truth” (labels) that the GBM uses as target Y during training, not as a classifier of the real user
- Loaded and analyzed the already-implemented `clustering.py`: K-Means on normalized allocation ratios (equity/bond/cash), silhouette score to validate K=3, deterministic label assignment by mean equity ratio
- Verified the clustering results on SCF 2022 `implicate=1` (n=4,595): AGGRESSIVE 59.2%, CONSERVATIVE 34.3%, MODERATE 6.5%
- Identified a critical bug: `build_pipeline()` in `scf_pipeline.py` returned only `alloc` (EQUITY/BOND/CASH/STOCKS), so `df_labeled = alloc.copy()` in `clustering.py` produced a parquet without demographic features — the GBM in W3 would have no X to train on
- Applied the bug fix: added `df_selected` to the values returned by `build_pipeline()`, and updated `clustering.py` to use `df_selected.copy()` instead of `alloc.copy()`
- Opened PR feature/p3-clustering on GitHub with a complete title and description including the documented “Known limitation”

- Resolved 2 CI ruff errors: removed `SCF_IMPLICATE` imported but unused (F401), corrected the alphabetical order of the import block (I001)
- Green CI (2 successful checks), PR ready for Sabrina's review

How I did it

- All file changes done via `python -c` one-liners from the terminal — no editor opened
- ruff fixes with `sed -i ''` for line removal, then a Python script for exact string substitution
- Git workflow: `git add`, `git commit`, `git push` from the Mac terminal after each fix
- Recurring mistake: `cd robo-advisor` when already inside the folder — ignored because the subsequent commands worked correctly

Difficulties

- CI failed twice for ruff errors: first `SCF_IMPLICATE` unused (F401), then unsorted import block (I001). Resolved iteratively by reading the GitHub logs
- Editing files without an editor: preferred the `python -c` with `str.replace()` approach to avoid typos in `nano`

Achievements / Key decisions

- **W2 closed:** `clustering.py` on branch `feature/p3-clustering`, green CI, PR opened for P1 review
- **Bug fix applied:** the `scf_labeled.parquet` will now contain all demographic features + allocation columns + `profile_label` — ready for GBM training in W3
- **Consolidated understanding of the two-phase design:** Phase 1 = clustering on allocation ratios to generate labels; Phase 2 = GBM on demographic features to predict the labels on new users
- **Clustering results documented:** polarized distribution (59% AGGRESSIVE, 6.5% MODERATE) consistent with the literature and with the SCF oversampling of top wealth percentiles

Next steps

- Wait for Sabrina's review and merge on PR `feature/p3-clustering`
- W3: GBM training on `scf_labeled.parquet` — feature X = AGE, EDUC, INCOME, YESFINRISK, NOFINRISK, KIDS, NETWORTH, WSAVED, EQUITY_RATIO; target Y = `profile_label`
- Add SHAP TreeExplainer to produce `top_drivers` in the `ProfilerOutput`
- Extend `test_profiler.py` with test cases for the GBM path
- Verify with Sabrina the status of `AGENTS.md` and the automated PR (Criterion 5 — mandatory)

Notes for the academic PDF

- The asymmetric cluster distribution (59% AGGRESSIVE) is not a bug but an artifact of the SCF sampling design, which over-represents high-net-worth families. To be documented honestly in the Limitations section of the PDF as “US-centric bias” and “wealth oversampling”

- The 6.5% MODERATE is consistent with the behavioral literature: most families are polarized between equity-heavy and cash-heavy — the “truly balanced mix” is an unstable position. Citable with reference to Grable & Lytton 1999
 - The choice to cluster on allocation ratios instead of absolute values is a defensible methodological decision: it captures allocation behavior independently of total wealth. To explain in the ML section of the PDF
 - The “clustering generates labels → GBM learns to predict the labels” flow is the point that distinguishes this approach from a rule-based system masquerading as ML — to articulate clearly in the “Why genuine ML” section
-

12 May 2026 — Week 3

P1 — Backend / Data Engineering (session 1)

Estimated duration: 3-4 hours

What I did

- Read and analyzed `W2_memoria_consolidata_P1.md` to take stock of the situation
- Verified the CI state and the merge of Elena’s PR (`feature/p4-llm-narrator`) — already in `main`
- Implemented the `/advice` endpoint in `backend/api/main.py` (branch `feature/p1-advice-endpoint`):
 - `AdviceRequest` / `AdviceResponse` Pydantic models
 - Recommendation retrieval from the DB by `recommendation_id`
 - Construction of `GroundTruthPayload` from saved data
 - Call to `NarratorClient.narrate()` (P4)
 - 5-step `validate()` (P4)
 - DB audit trail update (`validator_flags`, `system_prompt_hash`, `ground_truth_json_hash`)
 - Inline academic comment describing the 3 stages of the LLM pipeline
- Added API key header auth (X-API-Key) on `/profile`, `/optimize`, `/advice` via `Depends(verify_api_key)`
- Opened PR `feature/p1-advice-endpoint` → `main` with a detailed description (review to Elena)
- Wrote `tests/test_advice_pipeline.py` with 4 integration tests (branch `feature/p1-integration-tests`):
 - `test_advice_unknown_recommendation_id` — 404 for a non-existent ID
 - `test_advice_happy_path` — 200 with validated LLM response
 - `test_advice_injection_blocked` — `injection_blocked=True`
 - `test_advice_response_schema` — all fields present
- Resolved a merge conflict on `main.py` between the two branches via the GitHub conflict resolver
- Merged both PRs into `main` with green CI

How I did it

- Read Elena’s files (`narrator.py`, `validator.py`, `ground_truth.py`) before writing code
- Used `unittest.mock.patch` to mock `init_db`, `anthropic.Anthropic`, and the `ANTHROPIC_API_KEY` env var in the tests — same pattern as `test_data_loader.py`

- Used `PRAGMA foreign_keys = OFF` to bypass the FK constraint during the test DB setup
- Iteration over red CI: ~6 fix commits to resolve unordered imports, unused variables, indentation, a typo (`rrec_id`)
- Debugged the 500 error — identified that `get_mock_payload()` uses the label "balanced" not "MODERATE", solved with `_PROFILE_LABEL_MAP`

Difficulties

- FK constraint on recommendations → market_data_snapshots: solved with `PRAGMA foreign_keys = OFF` in the test setup
- `patch("backend.api.main.DB_PATH", ...)` did not intercept correctly → solved by patching `init_db` directly in some versions, then back to a `DB_PATH` patch with an env var
- `VALID_LLM_RESPONSE` contained "investors" with "invest" as a substring → blocked by the Validator (forbidden phrase). Solved with "European allocations"
- Merge conflict on `main.py` between `feature/p1-advice-endpoint` and `feature/p1-integration-tests` → resolved via the GitHub conflict resolver accepting the branch version
- A partial rewrite of `main.py` had diverged from the original design — manual realignment was needed

Achievements / Key decisions

- `/advice` endpoint live in main — unblocks P4 for the chat page
- API key header auth implemented on all protected endpoints
- 4 green integration tests for `/advice`
- 93 total green tests in CI
- Decision: use `get_mock_payload()` instead of reconstructing `GroundTruthPayload` from scratch — more robust and maintainable in W3, to be replaced with real data in W4
- Decision: `_PROFILE_LABEL_MAP` to translate `MODERATE`→`balanced`, `CONSERVATIVE`→`conservative`, `AGGRESSIVE`→`aggressive`

Next steps

- `input_sanitiser.py` — advanced rate limiting on `/advice`, max 500 chars, keyword blocking (Wed)
- End-to-end integration test pipeline `/profile` → `/optimize` → `/advice` (Wed-Thu)
- Working `agent_pr.yml` — mandatory Criterion 5, to be done by Friday (high priority)
- DB hardening — `validator_flags` and `retry_count` logged correctly (Fri)
- Verify the status of `ADR-003-cloud-deploy.md` — merged or to be finalized?

Notes for the academic PDF

- **"Mock at the boundary" pattern:** the tests mock `init_db` and `anthropic.Anthropic` — not the internal code. Citable example of "test the contract, not the implementation"
- `_PROFILE_LABEL_MAP`: example of an adapter pattern between the DB domain (UPPER-CASE) and the LLM payload domain (lowercase) — separation of concerns citable in the architecture section
- **FK constraint in tests:** `PRAGMA foreign_keys = OFF` is a deliberate choice for the tests — to be documented as a limitation of the SQLite approach in test contexts

- **3-stage LLM pipeline** fully documented in the inline comment of `/advice` — reusable in the LLM Narrator section of the PDF
 - **CI iteration:** ~6 fix commits in one session — concrete example of CI-driven development citable in Lessons Learned
-

P1 — Backend / Data Engineering (session 2)

Estimated duration: 2 hours

What I did

- Configured `ANTHROPIC_API_KEY` as a secret in GitHub Actions
- Set a \$5 monthly spending limit on the Anthropic Console
- Created the API key `robo-advisor-usi-2026` on `console.anthropic.com`
- Wrote `.github/workflows/agent_pr.yml` from scratch (the file was empty):
 - Trigger: `workflow_dispatch` + push to `backend/optimizer/`
 - Reads all the Python files in `backend/optimizer/`
 - Calls an LLM API to generate/improve docstrings
 - Commits on branch `agent/optimizer-docstrings-{run_number}`
 - Opens a PR automatically via `gh pr create`
- Resolved a deprecated-model error in the workflow (updated the pinned LLM model ID to the current version)
- Resolved a PR permission error (`GITHUB_TOKEN` not authorized) → created a PAT with 90-day validity, added as the secret `PAT_TOKEN`
- Enabled `PAT_TOKEN` in the workflow in place of `GITHUB_TOKEN`
- Triggered the workflow successfully → **PR #43 opened automatically** by the AI agent
- Created `backend/llm/input_sanitiser.py`:
 - 500-char limit
 - Keyword blocking (14 known patterns)
 - Wraps input in a `<user_input>` tag
- Wired `sanitize()` into the `/advice` endpoint as a Layer 1 pre-call
- Wrote and merged `docs/adr/ADR-003-cloud-deploy.md`:
 - Streamlit Community Cloud vs Railway
 - Decision motivated with pros/cons
 - SQLite limitations documented
 - Railway documented as a fallback

How I did it

- Identified that the PR permission problem was at the GitHub organization level — not modifiable from the standard settings
- Used a PAT (Personal Access Token) with scope `repo` + `workflow` as a workaround
- Updated the model by reading the deprecation message in the workflow log
- `input_sanitiser.py` written as an independent module for separation of concerns — Layer 1 separate from Layer 2 (NarratorClient) and Layer 3 (validator.py)

Difficulties

- GITHUB_TOKEN not authorized to open PRs in a private organization repo — solved with a PAT
- Workflow permission settings not modifiable from either the repo or the organization — a GitHub limitation for organizations
- The pinned LLM model ID in the workflow was deprecated — updated to the current version
- The model change did not reach `main` due to branch/merge confusion — solved by editing directly on `main`

Achievements / Key decisions

- **Criterion 5 completed** — `agent_pr.yml` working, PR #43 opened by the AI agent
- PR #43 URL: <https://github.com/Programming-for-finance-II/robo-advisor/pull/43>
- `input_sanitiser.py` — Layer 1 defence wired into `/advice`
- `ADR-003-cloud-deploy.md` — W3 academic deliverable completed
- Decision: PAT with 90-day validity (expires August 2026) — covers the professor's grading
- Decision: PR #43 left open intentionally as evidence for `AGENTS.md`

Next steps

- DB hardening — `validator_flags`, `retry_count`, `fallback_triggered` logged correctly (Fri)
- End-to-end integration test pipeline `/profile` → `/optimize` → `/advice` (Fri)
- Communicate the PR #43 URL to Elena for the `AGENTS.md` evidence
- Deploy on Streamlit Cloud (W4 — but de-risk as early as possible)

Notes for the academic PDF

- **Criterion 5 evidence:** PR #43 opened automatically by GitHub Actions + an LLM API — citable in the AI Agents section as a concrete example of an agentic workflow
 - **PAT workaround:** example of infrastructural problem-solving — the GitHub restrictions of private organizations do not allow GITHUB_TOKEN to open PRs. Solution: a PAT with limited scope. Citable in Lessons Learned as an environment limitation
 - **input_sanitiser.py:** Layer 1 of the LLM safety pipeline — separation of concerns between pre-call defence (sanitiser), mid-call (NarratorClient) and post-call (validator). Citable in the LLM Safety section
 - **ADR-003:** SQLite does not persist across redeployments on Streamlit Cloud — a documented limitation, accepted for the university prototype. Citable in Section 6 (Limitations)
-

P4 — Frontend / LLM / Docs

Estimated duration: ~1.5 hours

What I did

- Completed `backend/llm/validator.py`: added Step 5 — EU Awareness Rule 9

- New `ValidationFlag.EU_AWARENESS_MISSING`
- New parameter `eu_awareness_required: bool = False` in the `validate()` function
- New constant `_EU_AWARENESS_KEYWORDS_A` (references to US sources: SCF, Federal Reserve, etc.)
- New constant `_EU_AWARENESS_KEYWORDS_B` (references to European investors)
- New function `_check_eu_awareness_missing()`: requires the presence of both groups
- Fix: added year-like integer exclusion (1900-2100) in `_check_hallucinated_numbers` to avoid false positives on “SCF 2022”, “MiFID II 2014”, etc.
- Completed `tests/test_validator.py`: added the `TestEUAwarenessRule9` class with 8 tests
 - `test_eu_aware_response_passes`
 - `test_missing_us_reference_fails`
 - `test_missing_eu_reference_fails`
 - `test_neither_group_present_fails`
 - `test_rule9_blocks_validate_when_required`
 - `test_rule9_skipped_when_not_required`
 - `test_scf_keyword_satisfies_group_a`
 - `test_full_pipeline_with_eu_awareness_passes`
- Final result: **34/34 tests passed**, ruff clean
- Committed and pushed on branch `feature/p4-llm-validator`

How I did it

- Designed Step 5 and wrote the code and debugging directly
- pytest for continuous verification after each change
- ruff for CI-compatible lint
- VS Code for manual file editing

Difficulties

- `test_full_pipeline_with_eu_awareness_passes` failed because the text contained “SCF 2022” — the number 2022 was being extracted and not found in `allowed_numbers`
- Fix: added a guard `1900 <= n <= 2100` in `_check_hallucinated_numbers` to exclude years from the hallucination check
- Import `_check_eu_awareness_missing` initially placed mid-file → ruff E402 → moved to the top with the other imports

Achievements / Key decisions

- **5-step validator complete** — LLM safety pipeline documentable in the academic PDF
- **Rule 9 EU Awareness implemented and tested**: the system verifies that every LLM response, when `profiler_us_centric_caveat=True`, explicitly contains the reference to the US/EU SCF data gap
- The choice of two keyword groups (Group A = US source, Group B = European investor) makes the check robust to different narrator phrasings
- The `eu_awareness_required=False` default parameter maintains backward compatibility with the existing tests
- The year-integers fix is a documented limitation: years in narrative text are not financial values — citable in ADR-004 as a known limitation of the number checker

Next steps

- docs/adr/ADR-004-llm-narrator-validator.md — still empty, to be written (W3 Thu or W4)
- Wiring narrator.py + validator.py into the FastAPI POST /advice endpoint
- Chat Advisor page in frontend/app.py connected to the backend
- Update the AGENTS.md Evidence Log with this week's PR
- **Urgent action:** contact P1 about agent_pr.yml — the AI-agent PR is mandatory for Criterion 5, deadline next Thursday

Notes for the academic PDF

- The 5-step validator is the heart of the **4.4 Validator** section of the LaTeX — document:
 - Steps 1-4 as the base safety pipeline
 - Step 5 as the regulatory implementation of Rule 9 (MiFID II + SCF/EU geographic gap)
 - The choice to make Step 5 optional (eu_awareness_required) is a defensible design decision: the check activates only when the regulatory context requires it
 - The “years as numbers” bug and the year-range guard fix is a concrete example of a **failure mode of the number checker** — citable in the Limitations section
 - 34 unit tests across 5 steps = demonstrable coverage for the “coding style + testing” criterion
-

13 May 2026 — Week 3

P2 — Quant / Portfolio Optimization

Estimated duration: 1h 30min

What I did

- Review of the hrp.py and markowitz.py code received from W2
- Implemented backend/optimizer/backtest.py — complete backtest engine
- Implemented scripts/download_backtest_data.py — script to download historical prices from yfinance with automatic UCITS → US fallback
- Wrote tests/test_backtest.py — 9 unit tests with no network calls
- Opened and merged PR #51 feature/p2-backtest-scenarios → main
- Resolved 4 cycles of ruff fixes (F401, E402, I001) and 1 test assertion fix

How I did it

- backtest.py: daily loop over test prices with monthly rebalancing (month-end). Weights computed on a 252-day lookback window. Transaction cost: $TC = (10 \text{ bps} / 10000) \times \sum |\Delta w_i|$ applied as a deduction from the rebalancing-day return. Three strategies in parallel: HRP (calls optimize()), MV (calls optimize_markowitz()), 1/N (fixed equal weights)
- Output: ScenarioResult dataclass serializable via asdict() → JSON. One file per scenario + a summary with metrics only (no time series)

- `download_backtest_data.py`: downloads prices for the window `test_start - 252 days → test_end`. Applies fallback if NaN ratio > 2%. Forward-fill up to 5 consecutive days for holidays. Saves CSV in `data/prices/`
- Tests: deterministic synthetic data with `np.random.default_rng(seed=42)`. No dependency on network or files

Difficulties

- ruff cycles: `field` unused, `Path` imported twice, `RebalanceEvent` imported but unused, accidental slash in an import, `l001` on the import block after `sys.path.insert`
- Test `test_run_scenario_transaction_costs_are_positive` failed for 1/N: in this model the 1/N weights are always equal → turnover 0 → TC 0. Assertion corrected by excluding 1/N from the check
- GitHub web-interface workflow: each fix requires a separate commit, no way to do automatic `ruff --fix`

Achievements / Key decisions

- PR #51 merged into `main` with green CI
- Backtest architecture separated from data download: `backtest.py` is pure computation, `download_backtest_data.py` handles I/O and network
- Transparent UCITS → US fallback: `backtest.py` does not know which ticker was substituted, it keeps the original column name
- MV does not use `profile` or `cluster_map` — a pure Max-Sharpe benchmark, the profile-awareness is HRP's differentiator
- Documented decision: 1/N has TC = 0 in the model because we do not track weight drift between rebalancing. To cite as a simplification in the PDF

Next steps

It is Wednesday evening — `regime_detector.py` was planned for today and remains to be done. To be completed by Thursday morning because P4 needs it for the Stress Banner.

- **Immediate priority (Thursday)**: implement `backend/optimizer/regime_detector.py` — VIX > 30 threshold logic, cluster-level ERC fallback, `regime` flag in output, coordination with P4
- **As soon as possible**: run `download_backtest_data.py` locally and verify the three CSVs. Debug any yfinance problems on XEON.MI and AGGH.MI
- **Friday-Sunday**: write `docs/adr/ADR-003-regime-detector.md`
- **Before the final W4 PR**: align `ASSET_WEIGHT_MIN` between `universe_config.py` (0.05) and `hrp.py` (0.03)

Notes for the academic PDF

- **Section 5 — Backtest Results**: the three scenarios cover distinct regimes — GFC 2008 (liquidity shock + correlations to 1), COVID 2020 (rapid crash + recovery), Rate Hike 2022 (duration selloff, equity and bond falling simultaneously). HRP should show an advantage over MV in GFC and COVID thanks to the robustness of the covariance. 1/N is the naïve benchmark of DeMiguel et al. (2009)

- **Limitation to cite:** simplified transaction cost model — 10 bps on one-way turnover, without bid-ask spread or market impact. Justifiable for liquid ETFs
 - **Limitation to cite:** weights between rebalancing are kept constant (drift not tracked). In reality the weights drift with prices → the real turnover is higher
 - **UCITS limitation for GFC:** CSPX.L and AGGH.MI did not exist in 2008. Fallback to SPY and AGG is economically equivalent but not UCITS. To make explicit in a note in the results table
-

P4 — Frontend / LLM / Docs (session 1)

Estimated duration: ~1 hour (today) + ~2 hours (yesterday, 12 May)

What I did

Yesterday (2026-05-12) — ~2 hours - Resolved the merge conflict of PR #41 (fix/advice-endpoint-integration → main) with 7 conflicts in backend/api/main.py - Accepted “incoming change” (main) for all conflicts: import os, NarratorClient/NarratorError, logger, constants block, verify_api_key, /advice endpoint body - Discovered that the merge had introduced duplicate classes (AdviceRequest, AdviceResponse defined twice) — red CI - Removed the duplicate block directly in the GitHub editor - Verified locally: git reset --hard origin/main → 93/93 green tests - Closed PR #41 without merging (code already present on main via PR #40)

Today (2026-05-13) — ~1 hour - Implemented render_chat() in frontend/app.py — Chat Advisor wired to the 3-stage LLM pipeline - Added imports at the top of the file: get_mock_payload, NarratorClient, NarratorError, validate - Resolved a white-page bug: removed if __name__ == "__main__":, replaced with a direct main() - Resolved StreamlitSecretNotFoundError: graceful handling of the missing secrets.toml - Created an empty .streamlit/secrets.toml (placeholder for the API key) - Tested the app locally with PYTHONPATH=. uv run streamlit run frontend/app.py

How I did it

- GitHub web editor for conflict resolution and PR closing
- git reset --hard origin/main to align local after the merge
- Reviewed the conflicting code carefully to decide which version to accept
- VS Code for editing app.py
- Terminal for local testing and debugging

Difficulties

- 7 conflicts in main.py: resolved by systematically choosing “incoming change” (main)
- Duplicate classes post-merge not detected by the automatic conflict resolver — found only after red CI
- Streamlit white page: caused by if __name__ == "__main__" incompatible with the Streamlit runtime
- st.secrets.get() crashes if secrets.toml does not exist — fixed with a graceful try/except
- ModuleNotFoundError: No module named 'backend' — solved with PYTHONPATH=.

Achievements / Key decisions

- **93/93 green tests on main** — LLM pipeline fully tested and working
- **Chat Advisor wired:** complete flow `get_mock_payload()` → `NarratorClient` → `validate()` → `display` implemented in Phase A
- **PR #41 closed correctly** without introducing regressions
- Streamlit app launchable locally with a single environment variable

Next steps

- Obtain `ANTHROPIC_API_KEY` and test it in the Chat Advisor
- `docs/adr/ADR-004-llm-narrator-validator.md` — empty file, to be written (Thursday)
- `docs/user_guide.md` — to be created (Thursday)
- Commit and push `frontend/app.py` to main

Notes for the academic PDF

- The `if __name__ == "__main__"` bug is a concrete example of the difference between direct execution and the Streamlit runtime — citable in the Lessons Learned section
 - The graceful handling of secrets (`try/except` instead of crash) is a robustness choice documentable in the Frontend/UX section
 - The Chat Advisor flow implements exactly the “3-stage LLM safety pipeline” pattern described in architecture: Ground Truth JSON → Narrator → Validator → display — consistent with ADR-004
 - `PYTHONPATH=.` as a solution to module resolution is preferable to hardcoded `sys.path` in the code — defensible choice in the Lessons Learned section
-

P4 — Frontend / LLM / Docs (session 2)

Estimated duration: ~2 hours

What I did

- Read the W3 development plan, extracting the P4-specific tasks
- Verified the state of the codebase: `narrator.py`, `validator.py`, `test_validator.py` already implemented in the previous weeks
- Wrote `docs/adr/ADR-004-llm-narrator-validator.md` complete (context, decision, 4-stage architecture, rejected alternatives, consequences, implementation evidence table)
- Added 3 new test cases in `tests/test_validator.py` in the `TestEUAwarenessRule9` class:
 - `test_mifid_compliance_question_eu_awareness`
 - `test_usd_etf_question_eu_awareness`
 - `test_ucits_question_eu_awareness`
- Resolved a non-runnable-tests problem: the cherry-pick on `feature/p4-docs` failed because that branch did not have the `tests/` structure — aborted the cherry-pick, restored `uv.lock`, and worked directly on `feature/p4-chat-advisor-ui` where the code was already present
- Synced the local branch with `git pull --rebase` and pushed
- Opened a PR on GitHub: “test: add EU awareness validator tests (Rule 9)”

How I did it

- Wrote the ADR structure and the test-case content directly
- `uv run pytest tests/test_validator.py -v` for local verification (37/37 passed)
- `git cherry-pick`, `git cherry-pick --abort`, `git restore` to manage the branch conflict
- `gh pr create` from the CLI to open the pull request

Difficulties

- **Empty ADR file after saving in VS Code:** the content existed only outside the repo working tree and did not transfer automatically into the local repo. Solved by pasting the content via heredoc (`cat > file << 'EOF'`) in the terminal
- **Commit on the wrong branch:** the EU awareness tests ended up on `feature/p4-chat-advisor-ui` instead of `feature/p4-docs` due to a missed checkout. The cherry-pick toward `feature/p4-docs` generated a conflict (the branch had no `tests/`) — solved with `cherry-pick --abort` and a PR opened directly from the correct branch
- **Push rejected:** the remote had local commits not present — solved with `git pull --rebase` before the push
- **Untracked `uv.lock` blocking checkouts:** solved with `git restore uv.lock`

Achievements / Key decisions

- **ADR-004 completed** — officially documents the Narrator Pattern, the 4-stage pipeline, the 9 system-prompt rules, and the known limitations (false positive “safe/safe_haven”, keyword-based EU awareness). Citable in the academic PDF
- **11 test cases for EU Awareness Rule 9** — complete coverage of the 5 patterns required by the dev plan: EU geography, MiFID II compliance, USD ETF, UCITS, SCF limitation
- **37/37 tests passing** — green CI on all validator tests
- **PR opened** on `feature/p4-chat-advisor-ui`

Next steps

- Wire the `frontend/app.py` questionnaire → `/profile` endpoint (currently uses local `_compute_profile`)
- Wire “Get Portfolio” → `/optimize` endpoint (currently uses a hardcoded mock)
- Write `docs/user_guide.md`
- Expand LaTeX PDF sections §2 (ML Risk Profiler) and §3 (Portfolio Optimization)
- Trigger `agent_pr.yml` on GitHub (`workflow_dispatch`) to generate the mandatory AI-agent PR — the Evidence Log in `AGENTS.md` is still empty

Notes for the academic PDF

- ADR-004 contains all the material for **Section 4 (LLM Narrator)** of the LaTeX: Narrator Pattern, Ground Truth JSON, 4-step Validator, Prompt Injection Defence, EU Awareness Rule 9. In W4 it will only need expanding with real numbers from the tests
- The “safe/safe_haven false positive” known limitation is now documented both in ADR-004 and in the comments of `mock_data.py` and `test_validator.py` — mentionable honestly in the Limitations section of the PDF

- The `temperature=0.0` choice for deterministic and auditable output is an academically defensible technical decision — to mention in the LLM Narrator section
-

14 May 2026 — Week 3

P2 — Quant / Portfolio Optimization

Estimated duration: ~2 hours

What I did

- Verified the state of PR #51 (backtest engine) — green CI, “Ready to merge”
- Created branch `feature/p2-regime-detector`
- Fixed `ASSET_MIN` in `hrp.py`: `0.03` → `0.05` to align it with `universe_config.py`
- Wrote `backend/optimizer/regime_detector.py` complete with:
 - `detect_regime()` — primary trigger $\text{avg } |\rho_{LW}| > 0.75$, secondary trigger $\text{VIX} > 30$
 - `get_erc_cluster_weights()` — cluster-level ERC fallback for the `HIGH_STRESS` regime
 - Local constants `ASSET_WEIGHT_MIN = 0.05`, `ASSET_WEIGHT_MAX = 0.40`
- Added 3 tests in `tests/test_optimizer.py` for the regime detector
- ruff fix (import order, unused numpy) on `regime_detector.py` — green CI
- Opened PR on `feature/p2-regime-detector` → main
- Created branch `feature/p2-plotly-charts`
- Wrote `backend/optimizer/charts.py` with 4 Plotly functions:
 - `plot_risk_contributions()` — horizontal bar chart of risk contributions
 - `plot_dendrogram()` — HRP dendrogram from a scipy linkage matrix
 - `plot_drawdown()` — drawdown chart for the 3 backtest scenarios (consumes JSON)
 - `plot_efficient_frontier()` — MV frontier scatter with HRP and MV markers
- ruff fix (I001, F401) on `charts.py` — green CI
- Opened PR on `feature/p2-plotly-charts` → main

How I did it

- All the workflow via the GitHub web editor + GitHub Actions for CI
- `regime_detector.py`: detection logic based on average pairwise correlation from the LW matrix, VIX as an optional secondary signal (W3 scaffold)
- ERC fallback: equal weight per cluster → equal weight within the cluster → clip + renormalise with local bounds
- `charts.py`: 4 independent functions, each returning a `go.Figure` ready for `st.plotly_chart()`
- Lazy import pattern used for `scipy` inside `plot_dendrogram` and `numpy` inside `plot_drawdown` to avoid unnecessary top-level dependencies
- Two rounds of ruff fixes in both PRs (I001 import order, F401 unused import)

Difficulties

- CI failed twice for ruff: first on `regime_detector.py` (inline import inside a function), then on `charts.py` (import order I001 + numpy unused F401)

- ASSET_MIN inconsistency between `hrp.py` (0.03) and `universe_config.py` (0.05) — resolved by fixing `hrp.py` in the regime detector branch. The complete refactor (importing from `universe_config` instead of defining locally) deferred to W4

Achievements / Key decisions

- `regime_detector.py` **complete** — PR opened, green CI
- `charts.py` **complete** — 4 Plotly functions ready, PR opened, green CI. P4 can embed them from Monday W4
- **Architectural decision:** double trigger for the regime (correlation OR VIX) — union logic. More robust than a single signal, defensible for ADR-003
- **Local constants in the regime detector** — technical debt documented in the comment, to be resolved in W4 with a refactor to `universe_config`

Next steps

- Merge PR `feature/p2-regime-detector` → `main`
- Merge PR `feature/p2-plotly-charts` → `main` (urgent — P4 needs it from Monday)
- **ADR-003** (`docs/adr/ADR-003-regime-detector.md`) — to be done Friday/weekend
- W4: refactor the box constraint constants toward `universe_config` as the single source of truth

Notes for the academic PDF

- **Section 3 — Portfolio Optimization:** the regime detector uses avg pairwise $|\rho_{LW}| > 0.75$ as the primary trigger — motivatable citing López de Prado (2016) on the instability of the HRP dendrogram when correlations converge to 1 in a stress regime
- **ADR-003:** document the choice of the 0.75 threshold (empirical, based on the crisis-correlation literature) and the cluster-level ERC fallback (DeMiguel et al., 2009 — naive diversification as a robust baseline in the absence of signal)
- **Section 6 — Limitations:** the VIX trigger is a scaffold — in production it would require a separate real-time VIX data feed from `yfinance`, which introduces an additional dependency not handled in the current architecture

P4 — Frontend / LLM / Docs

Estimated duration: ~1 hour (session distributed over 13-14 May)

What I did

- W3 state analysis: verified that the entire LLM pipeline (`narrator.py`, `validator.py`, `input_sanitiser.py`, `/advice` endpoint, Chat Advisor UI) was already complete on `main`
- Identified the only open W3 gap: `docs/user_guide.md` missing
- Created `docs/user_guide.md` complete (437 lines): end-to-end user flow, EU Awareness section, known-limitations table, API reference for developers
- Committed and pushed on `feature/p4-docs`, PR opened and merged into `main`
- Updated the `AGENTS.md` Evidence Log with PR #43 (the AI-agent docstring PR from Sabrina) as proof of Criterion 5

- Committed `AGENTS.md` directly to `main`

How I did it

- Verified the W3 state and wrote the user-guide content directly
- VS Code terminal for git (add, commit, push, diff, checkout)
- GitHub web for opening and merging the PR

Difficulties

- `AGENTS.md` had local unstaged changes on `main` — solved with an explicit `git add + git commit` after verifying the diff
- The `feature/p4-docs` PR had already been merged before adding the `AGENTS.md` line — solved by committing directly to `main`

Achievements / Key decisions

- **W3 fully closed** — all P4 deliverables on `main`
- `docs/user_guide.md` covers the professor's explicit requirement ("user guide section is present") — cited in the dev plan W4 Fri as a README requirement
- **Criterion 5 (AI Agents) satisfied** — PR #43 linked in `AGENTS.md` as evidence; the PR was opened automatically by `agent_pr.yml` calling an LLM API

Next steps (W4 — from Monday 18 May)

- `render_portfolio()` in `app.py`: replace the hardcoded mock weights with data from `get_mock_payload()`, add UCITS badges (EU), risk contribution chart, conditional stress banner
- HRP vs Markowitz tab: connect real data from the mock payload
- LaTeX PDF: complete Section 4 (LLM Narrator), Section 6 (Limitations), Section 7 (Lessons Learned), integrate the P2/P3 sections
- `README.md`: add usage examples with sample output
- Review release v1.0 with the team

Notes for the academic PDF

- `docs/user_guide.md` is directly citable in the Frontend/UX section as an example of structured documentation — it covers the educational disclaimer, prototype limits, and EU Awareness explicitly
- The Evidence Log in `AGENTS.md` with PR #43 is concrete proof of the agentic workflow (GitHub Actions → LLM API → automatic PR) — citable in the Lessons Learned / AI Tools section
- Criterion 5 is satisfied even though PR #43 still has a merge conflict on `hrp.py` (a P2 file): the PR URL is sufficient as proof, the merge is not required

15 May 2026 — Week 3

P2 — Quant / Portfolio Optimization

Estimated duration: ~30 minutes

What I did

- Confirmed that `backtest.py` and `regime_detector.py` were already completed before the session
- Wrote `ADR-006-regime-detector.md` starting from the real code of `regime_detector.py`
- Uploaded the document to the `docs/adr/` folder of the repo on GitHub
- Committed on branch `feature/p2-docs-adrs`

How I did it

- Reviewed the `regime_detector.py` code to extract the documentable technical choices
- ADR written reflecting exactly the real implementation: double trigger (correlation + VIX), OR logic, cluster-level ERC fallback
- Direct upload via the GitHub browser (no terminal)
- Resolved a numbering conflict: ADR-003 was already taken in the repo (cloud-deploy, ucits-fallback) → used ADR-006

Difficulties

- ADR numbering conflict: the original project plan assigned ADR-003 to the regime detector, but the team had already used ADR-003 for other documents → renamed to ADR-006 at commit time

Achievements / Key decisions

- **W3 P2 fully closed** — `backtest`, regime detector and ADR documentation all delivered
- **Real ADR numbering in the repo:** 001, 002 (×2), 003 (×2), 004, 005, 006 — to be aligned with P4 for the LaTeX PDF (the ADR references in the text must use the real numbers)
- **ADR-006 faithfully reflects the code:** 0.75 correlation threshold, 30.0 VIX threshold, cluster-level ERC fallback, OR logic documented with academic justification

Next steps (W4)

- `ADR-004-ledoit-wolf-shrinkage.md` — to be written Fri-Sun W4 (it was ADR-004 in the original plan, verify the available number in the repo)
- Plotly functions for the charts (efficient frontier, dendrogram, risk contribution bar, drawdown)
- Implement the MV tab for the UI in collaboration with P4
- Final code review: type hints, no magic numbers, defensive assertions
- Write the §3 Portfolio Optimization section of the LaTeX PDF (owner P2, integrate P4 by Wed-Thu W4)
- Provide the backtest tables for section §5 of the LaTeX PDF

Notes for the academic PDF

- The double trigger of the regime detector (correlation + VIX) deserves an explicit mention in section §3 as a robustness mechanism of the HRP allocation
 - The OR logic is a deliberate conservative choice: cost asymmetry between a false positive (ERC not needed) and a false negative (HRP in a crisis) — citable as a motivation in the PDF
 - References used in the ADR useful for the bibliography: Longin & Solnik (2001), Maillard et al. (2010), Whaley (2009)
 - Verify that the value `ASSET_WEIGHT_MIN = 0.05` in the code is aligned with what is written in the PDF — the spec files still show 0.03 in some places
-

17 May 2026 — Week 3

P3 — ML / Risk Profiling

Estimated duration: 1h30

What I did

- Recovered the W2 context: K-Means clustering completed, `scf_labeled.parquet` available with demographic features + `profile_label`, PR feature/p3-clustering open for P1 review
- Defined the three W3 deliverables in detail: `classifier.py`, `regime_detector.py`, extension of `test_profiler.py`
- Wrote the files, committed and pushed on `feature/p3-gbm-phase-b`
- Ran the GBM training locally: `uv run python -m backend.ml.profiler.classifier`
- Verified the training results and the test suite (43 passed, 0 warnings)
- Updated `AGENTS.md` with the agent contribution log
- Opened PR `feature/p3-gbm-phase-b` → main with `gh pr create`

How I did it

- The only command run manually: `uv run python -m backend.ml.profiler.classifier` for model training
- Fix applied: `shap 0.50.0` removed support for `GradientBoostingClassifier` in `TreeExplainer` → replaced with `HistGradientBoostingClassifier` (native sklearn, faster, SHAP compatible). Documented in the module docstring

Difficulties

- No blocking technical difficulty
- The SHAP/HistGBM fix resolved without manual intervention on the model logic

Achievements / Key decisions

- **Phase B completed:** `classifier.py` implements `HistGradientBoostingClassifier` trained on SCF 2022 ($n=4,595$, `implicate=1`) with WGT sample weights, SHAP `TreeExplainer` for `top_drivers`, and `LogisticRegression` as a comparison baseline

- **Training results:**

Metric	HistGBM	LR Baseline
Train accuracy	97.7%	79.9%
CV 3-fold	94.0% $\pm$ 0.15%	63.3% $\pm$ 2.9%

- The CV variance $\pm 0.15\%$ indicates robustness — the model generalizes, it does not memorize
- `regime_detector.py` typed scaffold: a working stub that always returns `normal`, structure ready for the VIX threshold in W4/future work
- **43 tests passed**, 2 skipped by design (waiting for `gbm_model.pkl` — correct)
- **Criterion 5 covered:** the agentic workflow operated autonomously on Git (branch, commit, push, PR). Documented in `AGENTS.md` with detail of the prompt, output and results. PR opened on GitHub as concrete evidence

Next steps (W4)

- Wait for Sabrina’s review and merge on PR `feature/p3-gbm-phase-b`
- Clean up `ml/profiler/` code: complete type hints, NumPy docstrings, zero magic numbers
- Write `ADR-009-scf-implicate-choice.md` — formal justification of using `implicate=1`
- **ML section of the LaTeX PDF** (owner P3) — SCF→clustering→GBM pipeline, why genuine ML, SHAP interpretation, US-centrism limitations. This is the section that distinguishes a 28 from a 30L
- Verify with Sabrina the status of `agent_pr.yml` (GitHub Actions mandatory for Criterion 5)

Notes for the academic PDF

- **Quantitative results ready:** Train accuracy 97.7%, CV 94.0% $\pm$ 0.15% vs LR baseline 63.3% $\pm$ 2.9%. The gap demonstrates that the GBM captures non-linear patterns (e.g. age $\times$ wealth interaction) that linear regression does not see — cite Guiso et al. 2018
- **Why HistGBM and not classic GBM:** same algorithmic family, native sklearn, supported by SHAP 0.50+, more efficient on medium tabular datasets. A defensible and documented technical decision
- **SHAP as XAI:** the normalized `top_drivers` passed to the `ProfilerOutput` allow the LLM narrator to comment on the reasons for the classification without inventing correlations — a differentiation point to cite in the ML Profiler section
- **Limitation to document honestly:** `gbm_model.pkl` is not retrained at runtime — the model is static (trained offline on SCF 2022). Updating it requires manually re-running `train_gbm()`. To cite in Limitations

18 May 2026 — Week 4

P1 — Backend / Data Engineering

Estimated duration: ~6 hours (afternoon + evening)

What I did

- **Repo made public** after a complete secrets audit (visual code scan + GitHub history):
 - Verified that `sk-ant-...` never appears in the code
 - Verified that `PAT_TOKEN` is always referenced via `{{ secrets.PAT_TOKEN }}` in the workflows, never hardcoded
 - Verified that `ANTHROPIC_API_KEY=` appears only in PR descriptions (textual), not in the code
 - Verified that the “insert the pat_token” commit contains only references to secrets
- **Streamlit Cloud deploy configured and live:**
 - Public URL: `https://robo-advisor-usi.streamlit.app/`
 - Resolved the GitHub organization block (Deploy keys disabled → enabled at org level)
 - Configured the `ANTHROPIC_API_KEY` and `API_KEY` secrets via the Streamlit Cloud UI (TOML format)
 - Resolved `ModuleNotFoundError` on `backend.llm.narrator` by adding `sys.path.insert()` in `frontend/app.py`
- **New infrastructure files committed on main:**
 - `requirements.txt` (root) — for Streamlit Cloud, replicates the `pyproject.toml` dependencies
 - `.streamlit/config.toml` — `headless = true`, `port = 8501`
 - `Dockerfile` — base Python 3.11-slim, install via `uv`, expose 8501
 - `docker-compose.yml` — app service with a persistent SQLite volume, `.env` support
- **tests/test_ucits_fallback.py written and green CI** (branch `feature/p1-testing`, PR opened, review requested from Matteo):
 - `test_fallback_triggers_on_empty_dataframe`
 - `test_fallback_tickers_applied_in_report`
 - `test_fallback_recorded_in_db` (requires inserting a users row with `session_token` to satisfy the FK constraint)
 - 3 rounds of lint/CI fixes: unused imports F401, `uuid` redefined F811, FK constraint schema
- **Wired the `/backtest` and `/compare` endpoints** in `backend/api/main.py` (branch `feature/p1-endpoints-w4`, PR opened, review requested from Emma):
 - `/backtest` calls `run_all_scenarios()` from `backtest.py`, returns metrics only (no equity curve) for 3 scenarios, rate limit 5/min
 - `/compare` calls `HRP optimize()` + `MV optimize_markowitz()` + computes equal-weight on-the-fly, returns weights + annualized volatility for each of the 3 strategies
- **CI coverage added** (`ci.yml`):
 - Added `--cov=backend --cov-report=term-missing --cov-fail-under=75`
 - Current coverage: 77% (80% target not reached due to P2/P3 modules at 0%)
 - Opened a GitHub issue to ask Emma and Matteo to write tests for `charts.py`, `clustering.py`, `scf_pipeline.py`, `regime_detector.py`
 - Threshold temporarily lowered to 75% to not block the team
- **Final README.md updated** (root, owner P1 co-P4):
 - Added a “Live Demo” section with the Streamlit URL
 - Updated installation section (`uv sync`, `docker-compose up`)
 - “Environment variables” section with `ANTHROPIC_API_KEY` and `API_KEY`
 - API docs updated for all 5 endpoints (`/profile`, `/optimize`, `/advice`, `/backtest`, `/compare`) with real request/response schemas
 - Added a “User Guide” section referencing `docs/user_guide.md`

- Added a “Testing” section with `pytest --cov` instructions
- Project Structure updated with `docker-compose.yml`, `Dockerfile`, reference to 5 endpoints

How I did it

- Entirely browser-based work: GitHub web UI for all commits, `github.dev` never opened
- Secrets audit via global GitHub search (search `sk-ant`, `ANTHROPIC_API_KEY=`, `PAT_TOKEN=` in the repo)
- Deploy via the Streamlit Cloud UI, no CLI
- All iterative fixes driven by CI: each `ruff/pytest` error analyzed, targeted fix, push, CI feedback
- Branch strategy: `feature/p1-testing` for the tests, `feature/p1-endpoints-w4` for the endpoints, direct commits on `main` for the infrastructure files (`docker-compose.yml`, `Dockerfile`, `README.md`, `ci.yml`, `requirements.txt`, `.streamlit/config.toml`)

Difficulties

- **Streamlit Cloud build stuck 30+ minutes** on “in the oven” → solved with a manual reboot of the app from the dashboard
- `ModuleNotFoundError` at the startup of the deployed app → `frontend/app.py` could not find `backend.llm.narrator` because Streamlit Cloud launches from a different working directory. Fix: `sys.path.insert(0, ...)` at the top of the file
- **Deploy keys disabled by org policy** → requested enabling at the organization level (not possible from the repo)
- **FK constraint violation** in test 3 of `test_ucits_fallback.py` → the users table requires `session_token NOT NULL UNIQUE`, the first insert did not pass it
- **3 rounds of ruff fixes** on the test file: `MagicMock/pytest/DataQualityError` imported but unused (F401), `uuid` redefined (F811)
- **Coverage 77% below the 80% target** → P2/P3 modules (`charts.py`, `clustering.py`, `scf_pipeline.py`, `regime_detector.py`) still at 0%. Team-aware decision: threshold at 75% + an issue to request tests from the other Ps, not exclude their modules unilaterally

Achievements / Key decisions

- **Live deploy = critical W4 dependency unblocked** for P4 (chat page testing)
- **5/5 API endpoints live** (`/profile`, `/optimize`, `/advice`, `/backtest`, `/compare`) — satisfies the professor’s “Creating your own API” criterion
- **3/3 UCITS fallback tests green** — mandatory P0 deliverable closed
- **CI with coverage** — a step forward for code quality and for the professor’s “coding style” criterion
- `docker-compose.yml` + `Dockerfile` — local reproducibility required by the W4 plan
- **Final complete README** with 5 endpoints, user guide, `uv` installation, `docker` — explicit professor requirement for the technical documentation
- **GitHub issue opened on coverage** — team-aware management (explicit request to P2/P3 instead of unilaterally excluding modules)

Next steps

- **Wait for review** from Matteo on PR `feature/p1-testing` and from Emma on PR `feature/p1-endpoints-w4`, then merge them into `main`
- **Verify** that Emma and Matteo add tests for their modules (issue opened)
- **Raise the coverage threshold back to 80%** in `ci.yml` once P2/P3 have delivered the tests
- **Git tag v1.0 + GitHub Release** Saturday/Sunday with a changelog
- **v1.0 review session** with the team before the iCorsi submission
- **Backtest JSON for Emma** — clarify with her whether she wants the physical file generated via `export_results_json()` or whether knowing that `/backtest` is now live is enough

Notes for the academic PDF

- **Deploy decision in practice:** Streamlit Community Cloud + GitHub integration → live in <2 hours of setup vs Dockerfile/Railway which would have taken more time. ADR-003 confirmed in hindsight
 - `sys.path` **fix in** `frontend/app.py`: document in Lessons Learned as the difference between the local environment (launch from root) and Streamlit Cloud (launches from `/mount/src/robo-advisor/`). Working directory is not always = repo root
 - **FK constraint** `session_token`: design choice of the `users` schema — token mandatory also for the audit trail. Example of how a rigorous schema emerges only at the testing stage
 - **Coverage 77% with modules at 0%:** discuss in Lessons Learned the difficulty of team-wide coverage when scientific modules (clustering, SCF pipeline, charts) have no tests written by their owners. A GitHub issue opened instead of a unilateral exclude = correct academic practice
 - **Section 7 Lessons Learned candidates:** (1) pre-publication secrets audit, (2) Streamlit Cloud module resolution gotcha, (3) team-wide coverage governance via an issue instead of exclude
-

P2 — Quant / Portfolio Optimization

Estimated duration: ~45 minutes

What I did

- Received the W4 P2 checklist with Monday's priorities
- Confirmed that `charts.py` was already on `main` — no action required
- Code review of `hrp.py`: identified 3 minor magic numbers (`PROFILE_TILT`, `_MAX_CONSTRAINT_ITER`, `RISK_FREE_RATE`) — left for now, not blocking
- Code review of `risk_metrics.py`: clean file, no changes needed
- Code review of `markowitz.py`: found and fixed 3 real problems
 - Typo `rom` → `from` (line 1, `SyntaxError`)
 - `CovarianceShrinkage(prices)` → `CovarianceShrinkage(prices, frequency=1)` to avoid double annualization of volatility
 - `MV_ASSET_MIN = 0.03` → `0.05` for alignment with `hrp.py`

- Committed the fix on `markowitz.py`
- Wrote a complete LaTeX §3 Portfolio Optimisation draft (ETF Universe, HRP, Ledoit-Wolf, Box Constraints, MV Comparison)
- Pasted the text into `report.tex` on branch `feature/p2-latex-section3`
- Opened a PR toward main with P4 as reviewer
- Attempted to fill the §5 backtest table — impossible without the output JSON (backtest never run with real data)

How I did it

- Manual code review file by file
- LaTeX written starting from the real code (`universe_config.py`, `hrp.py`, `markowitz.py`) and the canonical design
- Commit and PR via the GitHub browser

Difficulties

- §5 backtest table not completable: the backtest has never been run with real data, the output JSONs do not exist
- `markowitz.py` had a double-annualization bug that was not immediately obvious (frequency default=252 vs frequency=1 in `hrp.py`)

Achievements / Key decisions

- **W4 code review completed** on all 3 main files
- **LaTeX §3 completed** and PR opened — P4 can integrate
- **Real bug fixed in `markowitz.py`**: the MV volatility was inflated by $\sqrt{252}$ without the fix
- **Decision confirmed**: `ASSET_MIN = 0.05` in both `hrp.py` and `markowitz.py`

Next steps

- Tomorrow: ask P1 to run `run_all_scenarios()` and send `backtest_summary_moderate.json` → fill the §5 table
- Ledoit-Wolf ADR (verify the available number in the repo, ADR-006 was the last)
- Minor `hrp.py` fixes (`PROFILE_TILT`, `_MAX_CONSTRAINT_ITER`, `RISK_FREE_RATE` as constants)

Notes for the academic PDF

- The double-annualization bug in `markowitz.py` is a concrete example of why the HRP vs MV comparison must use the same input parameters — citable in section §3 as a design motivation
- The §5 table is TBD until P1 runs the backtest with real yfinance data
- Remember to update the bounds in the PDF: 0.05–0.40 per asset (not 0.03–0.40 as written in some TODOs)

19 May 2026 — Week 4

P4 — Frontend / LLM / Docs

Estimated duration: ~1 hour

What I did

- Complete review of the W4 checklist with the P4 tasks from the dev plan
- Updated README.md:
 - Added a dedicated **“AI Tools & Development Process”** section with a multi-tool table
 - Added a CI badge at the top
 - Streamlit Cloud cold-start note under the live demo link
 - null note on `expected_return/sharpe_ratio` in the API docs with a reference to ADR-001
 - Added `docs/architecture.md` and the ADRs to the project structure
 - Regime Detector added to the Technical Highlights table
- Committed and pushed README.md on branch `feature/p4-docs`
- Resolved a `.gitignore` problem: removed a stray coverage-cache entry that a local tool had added automatically, restored the correct version with `git restore --source=HEAD~1`
- PR `feature/p4-docs` updated — now shows only the README diff

How I did it

- Structured and wrote the README content directly
- Git from the terminal for branch management, stash, restore and commit
- GitHub web for PR diff verification

Difficulties

- `git checkout feature/p4-docs` blocked by a modified `uv.lock` — solved with `git stash + git checkout + git stash pop`
- `.gitignore` contained a stray coverage-cache line added automatically by a local tool — removed with `git restore --source=HEAD~1 -- .gitignore` and recommitted together with the README

Achievements / Key decisions

- README.md updated with a transparent and detailed AI Tools section
- AI usage declaration aligned with the course requirements (agentic project, AGENTS.md) and consistent with what is already in AGENTS.md
- `.gitignore` cleaned up before the PR — clean diff toward `main`

Next steps (tomorrow — Tue 20 May)

- Live /optimize wiring — test the “Load live market data” toggle on the deployed app
- Markowitz tab — evaluate whether P2’s `/compare` is available, otherwise document as future work in the PDF
- Add the dendrogram in the HRP tab (`plot_dendrogram()` already in `charts.py`)

- Remove the debug block in the Chat Advisor (`st.caption` with validator flags) before the demo
- Open a PR toward `main` if the Mon-Tue tasks are complete

Notes for the academic PDF

- The handling of the stray coverage-cache entry is a concrete example of a side effect of using development tools in the workflow — citable in the Lessons Learned section as a real case of attention required during development
 - The AI Tools section of the README is the basis for the “AI Tools Used” subsection of Section 7 of the LaTeX — just expand the table with a few retrospective lines
-

20 May 2026 — Week 4

P2 — Quant / Portfolio Optimization

Estimated duration: ~3 hours

What I did

- Wrote `tests/test_charts.py` (34 tests) for `backend/optimizer/charts.py` → 100% coverage
- Wrote `tests/test_risk_metrics.py` (34 tests) for `backend/optimizer/risk_metrics.py` → 100% coverage
- Brought the total project coverage from 77% to **81.07%** (183 tests passed)
- Opened and merged PR #70 (`test_charts`) and PR #71 (`test_risk_metrics`) — both green CI
- Responded to Sabrina’s issue #65 with the P2 coverage update
- Wrote the complete §3 Portfolio Optimization section for `docs/report.tex` (258 lines of LaTeX)
- Added 5 missing entries to `docs/references.bib` (Michaud1989, Marcenko1967, Mailard2010, Whaley2009, Markowitz1952)
- Wrote `docs/adr/ADR-007-ledoit-wolf-shrinkage.md` (complete academic document)
- Opened and merged PR #72 (docs: LaTeX §3 + bibliography + ADR-007) — resolved a conflict on `report.tex`
- Implemented `_render_mv_tab` complete in `frontend/app.py`: weights comparison table, efficient frontier chart, metrics row HRP vs MV — PR #73 opened, awaiting P4 review
- Final code review on `hrp.py`: extracted 4 magic numbers as named constants, added a defensive assertion on the profile label, fixed spacing
- Final code review on `markowitz.py`: added a defensive assertion on min observations, fixed return type, fixed trailing whitespace
- Final code review on `risk_metrics.py`: added return type `dict[str, object]` to `compute_all`, added defensive assertions
- Opened and merged PR #74 (`hrp.py` code review) and PR #75 (`markowitz.py`) and the final `risk_metrics.py` PR

How I did it

- Tests written by analyzing the code function by function, covering happy path, edge cases and assert failures
- LaTeX §3 generated from ADR-001, `universe_config.py`, `hrp.py`, `regime_detector.py` — a complete academic section with formulas and citations
- ADR-007 following the same format as ADR-006 already in the repo
- MV tab implemented with a Phase A fallback (mock weights) and Phase B (live optimizer) — compatible with P4's existing structure
- Code review performed on all three P2 optimizer files with criteria: magic numbers, type hints, defensive assertions

Difficulties

- Conflict on `report.tex` at the merge of PR #72 — P4 had already written content in §3; resolved by keeping the P2 version for all conflicts
- CI failed on `markowitz.py` for the import of `MIN_OBSERVATIONS` from `hrp.py` before the `hrp.py` PR was merged — solved by using the inline constant 60
- Ruff flagged multiple times: unused imports, long lines, import ordering, trailing whitespace — resolved iteratively
- The MV tab in `app.py` required 3 successive fixes to pass ruff (unused import, long line, import ordering)

Achievements / Key decisions

- **P2 coverage at 100%** on `charts.py` and `risk_metrics.py`
- **Total project coverage: 81.07%** — above the 80% target
- **LaTeX §3 complete** — P4 can integrate it into the PDF with no open TODOs
- **ADR-007** — complete Ledoit-Wolf academic documentation in the repo
- **Final code review completed** on all P2 files — type hints, no magic numbers, defensive assertions

Next steps

- §5 Backtest tables — wait for P1 for the JSONs (Saturday during end-to-end testing)
- MV tab — wait for review and merge from P4 (PR #73)
- Verify with P4 that the ADR references in the LaTeX use the real numbers (ADR-007 not ADR-004)
- Sunday: final PDF proofread and iCorsi submission

Notes for the academic PDF

- The docstring of `compute_all` explicitly documents that `expected_annual_return` and `sharpe_ratio` are null for HRP — citable in section §3 as a defensible design choice
- Section §3 uses `\parencite{Michaud1989}` to justify the absence of μ in HRP — verify the citation is correct in the final bibliography
- ADR-007 contains the formal justification for Ledoit-Wolf as mandatory pre-processing — P4 can cite it in the PDF with the correct number ADR-007
- §5 tables still TBD — they require the real output of `backtest.py` with `yfinance` data

P4 — Frontend / LLM / Docs (session 1)

Estimated duration: ~4 hours

What I did

Frontend (branch: feature/p4-portfolio-dashboard) - Removed the debug block from the Chat Advisor (`st.caption` with validator flags) — it no longer showed internal flags to the user - Added the HRP dendrogram in the HRP tab of the Portfolio Dashboard: - Synthetic correlation matrix built from cluster membership (`_CLUSTER_GROUPS`) - `plot_dendrogram()` from `backend/optimizer/charts.py` wired into the frontend - Wrapped in `try/except` to guarantee Phase A robustness - Fixed indentation and removed a redeclared `weights` variable (ruff I001) - `uv run ruff check frontend/app.py --fix` → zero errors - PR opened: `feature/p4-portfolio-dashboard` → `main`

LaTeX PDF (branch: feature/p4-academic-docs) - Section 1 — Introduction: problem statement (3 limitations of commercial robo-advisors), the project contribution (4 components), platform overview with FastAPI + SQLite + agentic workflow - **Section 4 — LLM Narrator and Validator:** complete - Ground Truth JSON Contract: 8 blocks, Pydantic invariant on `allowed_numbers` - System Prompt Design: 9 detailed absolute rules, `build_system_prompt()` and audit hash - Validator 5-step pipeline: blocking vs corrective, documented edge cases (false positive “safe”, EU keyword check) - Prompt Injection Defence: Layer 1 (sanitiser, length gate, keyword blocking, `<user_input>` tag) + Layer 2 (post-generation) + stateless design as multi-turn protection - **Section 6 — Limitations:** yfinance fragility (3 vectors: outage, retroactive adjustment, NaN UCITS), HRP opacity vs MV, residual LLM hallucination risk (3 failure modes) - **Section 7 — Lessons Learned:** agentic workflow (4 agents), AI tools, what worked (5 points), what did not work (3 points with the gitignore side effect, yfinance gaps, false positives) - **Section 8 — Conclusions:** synthesis, EU Awareness layer rationale, future work (5 items) - Resolved a structural problem: Section 7 was duplicated and `\section{Introduction}` was missing — corrected - PR opened: `feature/p4-academic-docs` → `main`

How I did it

- Verified alignment with ADR-004, `system_prompt.py`, `validator.py`; generated and wrote the LaTeX content directly
- VS Code for direct editing of `frontend/app.py` and `docs/report.tex`
- Terminal for `uv run ruff check --fix`, `git add/commit/push`
- Section 4 content derived almost entirely from ADR-004 (already written in W3) — conversion from markdown to academic LaTeX
- Section 7 content derived from the previous session logs and the README AI Tools section

Difficulties

- Dendrogram: `weights` variable redeclared inside the block — removed because already available in the function scope
- Comment `# --- Dendrogram ---` at column 0 instead of 4 spaces — corrected
- Single blank line between the end of the dendrogram and `def _render_mv_tab` — corrected to a double blank line (ruff E302)

- Section 7 LaTeX pasted twice by mistake — removed the duplicate version (old TODO)
- `\section{Introduction}` missing at the top of the document — added

Achievements / Key decisions

- **Mon-Tue W4 closed:** dendrogram + debug block removed + PR opened
- **Wed W4 completed:** all P4 sections of the LaTeX written (1, 4, 6, 7, 8)
- Section 4 is the densest and most important for the grade — narrator pattern, Ground Truth JSON, 5-step validator and injection defence are all documented with explicit reference to the code (backend/llm/)
- The `% TODO P2` and `% TODO P3` in sections 2, 3, 5 are clearly marked — P2 and P3 can write autonomously

Next steps (Thursday 21 May)

- Wait for P2 and P3 for sections 2, 3, 5 of the LaTeX
- Compile the PDF: `pdflatex → biber → pdflatex × 2` once the sections are complete
- Merge PR `feature/p4-portfolio-dashboard` after P1 review
- Merge PR `feature/p4-academic-docs` after P1 review + integration of the P2/P3 sections
- Friday: finalize `AGENTS.md` and `README.md`

Notes for the academic PDF

- Section 4 is written with explicit references to the files (`backend/llm/narrator.py`, `validator.py`, `input_sanitiser.py`, `system_prompt.py`) — the professor can verify the code directly
- The “narrator, not calculator” pattern is the main architectural contribution of P4 — worth a citation in the oral presentation
- The 3 residual failure modes of the validator (false positive “safe”, semantic EU keyword check, 2% number tolerance) are documented honestly — this is appreciated in the academic evaluation criteria
- Section 7 uses concrete examples (`.gitignore` side effect, `yfinance` UCITS gaps) instead of generic ones — more credible and citable in the oral Lessons Learned section

P4 — Frontend / LLM / Docs (session 2)

Estimated duration: ~1h 30min

What I did

- Discussed graphical improvement of the Streamlit interface (dissatisfaction with the default look)
- Analyzed a reference screenshot of a dark premium finance dashboard (“Quant Allocation” style)
- Produced an interactive mockup of the visual target adapted to the project (navy/teal/purple palette, metric cards with sparklines, donut allocation, equity curve, SHAP driver badges)
- Defined a 3-level implementation strategy: `config.toml → style.py → app.py` components

- Wrote `.streamlit/config.toml` with a dark base theme (`primaryColor #7c5cfc`, `backgroundColor #0b0f19`)
- Wrote `frontend/style.py` complete with: `DARK_CSS` (sidebar, metric cards, tabs, buttons, inputs overrides), HTML constants for the MiFID II disclaimer, EU investor note, stress banner, and the functions `inject_css()`, `render_disclaimer()`, `render_eu_note()`, `render_stress_banner()`, `page_header()`
- Diagnosed and resolved `ImportError: cannot import name 'inject_css' — cause: frontend/style.py already existed with different content`; solution: append the new functions without overwriting
- Drafted the PR description for `feature/p4-premium-streamlit-theme`

How I did it

- Visual analysis of the reference screenshot to extract palette, typography, component patterns
- Built an interactive SVG/HTML mockup and generated the CSS directly
- Diagnosed the `ImportError` from the terminal output without direct access to the local filesystem
- “Append-safe” strategy: add to the bottom of the existing file instead of overwriting

Difficulties

- `frontend/style.py` already existed with unknown content → `ImportError` on first launch
- Unable to read the file remotely; proposed solution: `cat frontend/style.py` to verify before integrating
- Google Fonts (`@import`) might not load in an offline environment or on Streamlit Cloud with a restrictive CSP — to be tested at the next launch

Achievements / Key decisions

- **Palette defined and fixed:** `#7c5cfc` (purple primary), `#0dcfb0` (teal accent), `#0b0f19` (bg), `#111827` (surface), `#1e2640` (border) — to be used also in the Plotly charts for consistency
- **All EU-required banners** (disclaimer, EU note, stress) are now styled HTML components, not raw `st.warning()` — clearly superior visual impact
- `page_header()` with Space Grotesk unifies the look across pages without refactoring `app.py`
- Branch `feature/p4-premium-streamlit-theme` ready for a PR toward main

Next steps

- Verify that `inject_css()` is called as the first line after `st.set_page_config()` in `app.py`
- Test Google Fonts loading on Streamlit Cloud (fallback: remove `@import` and use `font = "sans serif"` from `config.toml`)
- Apply the `PLOTLY_DARK` dict to the Plotly charts (equity curve, donut, risk contribution bar) for palette consistency
- Merge the PR and do an end-to-end visual check before the final demo

Notes for the academic PDF

- The choice to separate all styling into `frontend/style.py` (instead of inline in `app.py`) is citable in the Frontend/UX section as an example of separation of concerns and clean coding style (criterion 4)
 - The custom HTML banners for the disclaimer and EU note are more academically defensible than `st.warning()`: they demonstrate design awareness, not just minimal functionality
 - The “dark finance theme” pattern with a consistent palette across UI + Plotly is a visual differentiator for the demo — citable in the UX section as a deliberate end-user-oriented choice
-

21 May 2026 — Week 4 (Thursday)

P4 — Frontend / LLM / Docs (session 1)

Estimated duration: ~2 hours

What I did

LaTeX PDF - Complete review of `report.tex` — structure and content verified - Completed `references.bib` with the missing entries: Michaud1989, Marcenko1967, Maillard2010, Whaley2009, Markowitz1952 (the first 4 were absent) - Added `\nocite{FedSCF2022, MiFIDII}` before `\printbibliography` to force the two entries into the bibliography without an inline citation - Removed the `<YOUR_ORG>` placeholder from the header of the `.tex` file

Frontend — `frontend/app.py` - Added `_UCITS_TICKERS` as a global `frozenset` - Added `_MOCK_WEIGHTS`, `_MOCK_REGIME`, `_LABEL_TO MOCK`, `_DATA_START` as clean constants at the top of the file - Removed the duplication of `_UCITS_TICKERS` (it was defined twice — F811) - Fixed a bug: `_render_hrp_tab(profile, _MOCK_WEIGHTS, _MOCK_REGIME)` → `_render_hrp_tab(portfolio)` (correct signature, data from the portfolio dict) - Fixed a bug: `with tab_mv` now calls `_render_mv_tab(portfolio, profile_key)` instead of inline `st.info()` — retrieves stress scenarios and backtest from the mock - `uv run ruff check frontend/app.py --fix` → zero errors - `uv run pytest tests/ -v` → all tests passed - Commit and push on `feature/p4-portfolio-dashboard`

AGENTS.md - Agent 2 (Docstring PR): status updated to Completed, PR #43 linked - Agent 3 (LLM Narrator): status updated to Completed - Agent 4 (LLM Validator): updated from 4-step to 5-step (EU Awareness Rule 9), status updated to Completed, fallback and output documented - Agentic Workflow Philosophy: 4-step → 5-step updated - Notes for Graders: future → past, PR #43 referenced explicitly - Commit and push on `feature/p4-docs`, PR opened

Box constraint verification - `grep` on `backend/optimizer/` → `ASSET_MIN = 0.05` in `hrp.py`, `regime_detector.py`, `markowitz.py` - PDF already aligned (says 0.05) — no change needed - The 0.03 found in the `grep` is `RISK_FREE_RATE`, not the weight floor

How I did it

- Reviewed the LaTeX, generated the BibTeX entries, and identified the bugs directly

- Terminal for `grep`, `uv run ruff check --fix`, `uv run pytest`, `git`
- VS Code for direct editing of `app.py`, `AGENTS.md`, `references.bib`

Difficulties

- `references.bib` had only 4 of the 9 entries cited in the `.tex` — identified with a systematic review of the `\cite` and `\parencite` in the document
- Bug `_render_hrp_tab`: called with the wrong signature (3 arguments instead of 1) — corrected before running `ruff`
- `with tab_mv` had an inline `st.info()` instead of calling `_render_mv_tab` — the function was already written but not wired

Achievements / Key decisions

- **All P4 W4 tasks are closed** (except the P2/P3 LaTeX sections which depend on the others)
- `references.bib` complete and aligned with all the `\cite` of the `.tex`
- Box constraint verified: 0.05 in the code and in the PDF — consistent
- `AGENTS.md` with concrete PR evidence for criterion 5 — criterion satisfied
- Frontend: UCITS badges, stress banner, live optimizer toggle, risk chart, dendrogram, `_render_mv_tab` with stress scenarios all wired

Next steps

- Push P2 (Section 5 — Backtest) and P3 (Section 2 — ML Risk Profiler)
- Compile the final PDF: `pdflatex → biber → pdflatex × 2`
- Weekend: end-to-end manual test of the deployed app, proofread the PDF, submit to iCorsi

Notes for the academic PDF

- Handling the missing `references.bib` entries is documentable in the Lessons Learned section: compiling LaTeX promptly (instead of at the last minute) allows undefined entries to be found before the submission
 - The un-wired `_render_mv_tab` bug is a concrete example of code written but not integrated — avoidable with integration tests on the frontend
-

P4 — Frontend / LLM / Docs (session 2)

Estimated duration: ~1h 30min

What I did

- Clarified that there is no need to create a separate website: Streamlit deployed on the cloud is the web frontend required by the professor
- Diagnosed the visual gap between the premium mockup (Image 1) and the real app (Image 2): CSS not applied uniformly

- Identified 4 causes: `show_disclaimer()` used `st.warning()` instead of `render_disclaimer()`, `st.title()` instead of `page_header()` on Questionnaire and Chat, missing font fallback, unstyled sidebar radio buttons
- Fixed `app.py`: `show_disclaimer()` now calls `render_disclaimer()`, `page_header()` applied to all 3 pages
- Fixed `frontend/style.py`: added a system-fonts fallback, added CSS for the sidebar radio buttons with a purple active state
- Added a Settings page (4th item in the sidebar): data source toggle, API status indicator, About section
- Committed and pushed on `feature/p4-premium-streamlit-theme`
- Opened a PR toward `main` with a description

How I did it

- Direct visual comparison between the real app screenshot and the mockup to identify the gaps
- Reading the `app.py` and `style.py` code to find the missing calls
- Surgical changes: 3 changes to `app.py`, 2 to `style.py`
- Local test with `streamlit run frontend/app.py`
- Git workflow: branch `feature/p4-premium-streamlit-theme` already existing, push with `--set-upstream`

Difficulties

- `git push` initially failed because the branch had no upstream — solved with `--set-upstream`
- API key not configured locally → Settings shows red (correct behavior, not a bug)
- The mockup was illustrative, not a real screenshot — created for visual reference, not as an implementation specification

Achievements / Key decisions

- Dark theme applied consistently across all 4 pages
- Settings page added: useful for the demo (shows the API key status at a glance)
- PR `feature/p4-premium-streamlit-theme` ready for P1 review and merge into `main`

Next steps

- Merge the PR into `main` after P1 review
- Verify that Streamlit Cloud updates after the merge
- Configure `ANTHROPIC_API_KEY` as a secret on Streamlit Cloud if not already done
- Proceed with the remaining W4 tasks: LaTeX PDF, final AGENTS.md, README polish

Notes for the academic PDF

- The separation of all styling into `frontend/style.py` is citable in the Frontend/UX section as an example of separation of concerns (the professor's criterion 4)
- The custom HTML banners for the disclaimer and EU note are more defensible than `st.warning()`: they demonstrate intentional design awareness

- The Settings page with an API status indicator is an example of developer-oriented UX, documentable in the Frontend section
-

22 May 2026 — Week 4

P4 — Frontend / LLM / Docs

Estimated duration: ~45 min

What I did

- Discussed and implemented the graphical refactoring of the Streamlit questionnaire: the three macro-sections (“Who You Are Financially”, “How You Invest”, “How You React”) move from a single block to three distinct rectangles/cards with a border and a colored header
- Added to `frontend/style.py`:
 - Constants `SECTION_CARD_HTML_OPEN` and `SECTION_CARD_HTML_CLOSE` (HTML for opening/closing the card)
 - Functions `render_section_open(section_title)` and `render_section_close()`
- Modified `render_questionnaire()` in `frontend/app.py`: replaced the single loop with three separate blocks, each wrapped by `render_section_open()` / `render_section_close()`
- Updated the import from `frontend/style.py` in `app.py` to include the two new functions
- Discussed and implemented inserting the logo into the sidebar:
 - Folder structure `frontend/assets/logo.png`
 - Replaced the `st.sidebar.title / st.sidebar.caption / st.sidebar.radio` block with a `with st.sidebar:` block containing `st.image()`, a centered HTML subtitle, a separator, and radio navigation
 - Suggested a robust path via `Path(__file__).parent / "assets" / "logo.png"` for Streamlit Cloud compatibility
- Received the logo image (a robo-advisor robot in a neon dark style) to insert as `frontend/assets/logo.png`

How I did it

- Generated the CSS and the HTML structure of the section cards directly
- Analyzed the existing code (`frontend/app.py`, `frontend/style.py`) to identify the minimal intervention points without breaking the existing logic
- Incremental approach: first the questionnaire cards, then the sidebar with the logo

Difficulties

- Nothing blocking. Technical note flagged: the HTML `<div>`s opened/closed around the `st.radio()` widgets work stably from Streamlit 1.35+ (the version in use), but are not an official pattern — to document as a deliberate choice if cited in the PDF
- Image path: `"frontend/assets/logo.png"` works if the working directory is the repo root; suggested the fallback with `Path(__file__).parent` for Streamlit Cloud

Achievements / Key decisions

- **Visually structured questionnaire:** three distinct sections with a #1e2640 bordered card, #111827 background, uppercase purple header with a #7c5cfc dot — consistent with the premium dark palette already defined in W4
- **Logo in the sidebar:** `st.image()` inside with `st.sidebar:` — a cleaner pattern than `st.sidebar.image()` and compatible with the existing CSS
- `frontend/assets/` **created** as a dedicated folder for static assets — clean separation from logic and style
- Both changes are on the already-existing branch `feature/p4-streamlit-ui`, without opening new branches

Next steps

- Visually verify the app after the changes (`uv run streamlit run frontend/app.py`)
- Check that the logo loads correctly both locally and on Streamlit Cloud
- Commit and push on `feature/p4-streamlit-ui`
- Possible PR toward `main` if the team is ready for a pre-demo merge
- Complete the LaTeX PDF sections (LLM Narrator + Frontend/UX) — the main remaining task of W4

Notes for the academic PDF

- The `render_section_open()` / `render_section_close()` pattern is citable in the Frontend/UX section as an example of **component-based UI design** in Streamlit: HTML helper functions encapsulated in `style.py` instead of inline HTML scattered in `app.py` — separation of concerns
 - The choice of a consistent palette (card border, header color, sidebar separator) with the already-defined dark finance theme demonstrates attention to UX as a deliberate design choice — a differentiator for the final demo
 - The use of `Path(__file__).parent` for static asset paths is citable as good practice for code portability (local vs cloud)
-

23 May 2026 — Week 4

P2 — Quant / Portfolio Optimization (session 1)

Estimated duration: ~1 hour

What I did

- Configured the local environment from scratch: installed `uv`, created the `venv`, resolved a git conflict (`git stash + rm uv.lock + git pull origin main`)
- Identified that `backtest.py` was on GitHub but not in the local repo
- Discovered that `universe_config.py` is in `backend/data/`, not in `backend/optimizer/`
- Discovered that the primary UCITS tickers (CSPX.L, AGGH.MI, XEON.MI) have history only from 2019 → used the `fallback_tickers` (SPY, AGG, BIL etc.) for the historical backtests

- Ran `run_all_scenarios()` successfully: 3 scenarios (GFC 2008, COVID 2020, Rate Hike 2022), 8 tickers, 4177 days of data
- Extracted the real numbers from the output JSON and filled the §5 table in `docs/report.tex`
- Wrote the text of `\subsection{Scenarios}` in §5
- Removed all TODO comments from §5
- Committed and opened PR `feature/p2-backtest-tables` → merged into `main`
- Confirmed that PR #73 (MV tab) was merged by P4

How I did it

- VS Code terminal on Mac (first guided use, step by step)
- `uv sync` to install the dependencies from `pyproject.toml`
- `python -c "..."` to call the backtest functions directly without a main block
- `yf.download()` with `ffill().dropna()` to handle NaNs in the UCITS tickers
- `ETF_UNIVERSE` from `backend/data/universe_config.py` to extract `fallback_ticker` and `cluster` for each asset
- JSON summary read with `json.load()` and formatted for LaTeX

Difficulties

- Local repo not synced with GitHub: solved with `git stash + rm uv.lock + git pull`
- `universe_config.py` not found in `backend/optimizer/` (it was in `backend/data/`)
- UCITS tickers with no pre-2019 history: solved by using the `fallback_ticker` for the backtest
- `dropna()` removed the entire DataFrame because the UCITS started in 2019: solved with `ffill().dropna()`
- `backtest.py` has no `__main__` block: called directly via `python -c`

Achievements / Key decisions

- **§5 Backtest Results completed with real numbers** — it was the only remaining P2 TODO in the PDF
- **Documentable decision:** the backtest uses US proxy tickers to cover GFC 2008 and COVID 2020, given that the UCITS equivalents do not have sufficient history. This is explicitly stated in the table footnote and in the Scenarios text
- **P2 closed 100%** — all the deliverables of the operational plan completed

Next steps

- None for P2 — work completed
- Optional tomorrow: proofread of the compiled PDF
- Verify that P3 has completed §2 ML Risk Profiler before the iCorsi submission

Notes for the academic PDF

- The choice of fallback tickers for the backtest is academically defensible: SPY, AGG, BIL are standard proxies for the respective UCITS and have sufficient history for all 3 scenarios

- The results show that HRP dominates on volatility and drawdown in all 3 scenarios — this is the key result to highlight in the oral discussion if requested by the professor
 - GFC 2008: HRP almost flat (-0.1%) vs MV -5.7% — the difference is economically significant
 - Rate Hike 2022: a difficult scenario for everyone, but HRP limits the damage (-8.9% vs -13.7% for 1/N) with the lowest volatility (7.6%)
 - COVID 2020: the only scenario where 1/N beats HRP on return (+11.9% vs +6.4%) — but with a much worse drawdown (-15.7% vs -10.1%)
-

P2 — Quant / Portfolio Optimization (session 2)

Estimated duration: ~2 hours

What I did

- Improved the HRP dashboard UI and the questionnaire UX in `frontend/app.py` + `frontend/style.py` (branch `feature/p2-optimizer-scaffold`)
- **Backtesting page:** replaced the placeholder “coming in Phase B” message with a fully functional page that loads pre-computed backtest results from `backtest_output/`
 - Scenario selector: GFC 2008, COVID-19 Crash 2020, Rate Hike Cycle 2022
 - Strategy comparison table HRP vs MV vs 1/N (CAGR, Volatility, Sharpe, Max DD, Calmar, TC)
 - 4 metric cards with delta vs the Markowitz benchmark
 - Equity curve chart (HRP / MV / 1/N) with a dashed reference line at 1.0
 - Drawdown chart with per-strategy colour fills
 - Source: real JSON files generated by `run_backtest.py` (moderate profile)
- **Compare (MV) page:** replaced the placeholder with a side-by-side HRP vs Markowitz comparison using mock Phase A data — weight comparison table (HRP, MV, difference), grouped bar chart per ticker, efficient frontier plot via `plot_efficient_frontier()` from `charts.py`
- **Questionnaire UX:** added a “Go to Portfolio Dashboard” button after profile calculation (same metric row as Profile and Confidence); answers now persist on navigation (radio buttons restore the saved index from `session_state['questionnaire_answers']`)
- **HRP Portfolio tab redesign:** profile badge with matching colour (purple/teal/red), cluster breakdown pills (Equity, Bonds, Alternatives, Cash), Portfolio Weights table with `st.column_config.ProgressColumn`, risk-contribution bars coloured by cluster with a vertical dashed 1/N reference line, transparent chart backgrounds, aligned section headings, removed the “undefined” pie-chart title
- **Sidebar fixes:** forced the sidebar always visible via CSS, hid the collapse arrow, reduced top padding to lift the logo
- Opened a PR against `main` for team review

How I did it

- Iterative inline CSS/HTML in Streamlit with `st.markdown(..., unsafe_allow_html=True)`
- Fixed the Streamlit form re-render issue by moving the result display out of the `if submitted:` block and into `if session_state.get('profile'):`, so the button click is detected correctly

- Transparent backgrounds set via `paper_bgcolor/plot_bgcolor = rgba(0,0,0,0)` for the donut, risk bars and dendrogram so the charts float on the page background
- Collapse arrow hidden with the `button:not(:has(p))` selector to target icon-only buttons without affecting the text navigation buttons
- Commits: feat: improve HRP dashboard UI and questionnaire UX, fix: remove f-string prefix without placeholders

Difficulties

- Ruff F541: removed the f-string prefix from two strings without placeholders in `render_backtesting()` (`frontend/app.py` line ~1375)
- Git push rejected by remote divergence: solved with `git stash / git pull --rebase / git stash pop` before the final push
- Pie chart showed “undefined”: fixed by replacing `title=None` with `title={'text': ''}`

Achievements / Key decisions

- Backtesting and Compare (MV) pages are now functional rather than placeholders — the dashboard reads real backtest JSON for the moderate profile
- Consistent colour language across the dashboard: risk-contribution bars, cluster pills and the donut all share the same cluster palette (purple = Equity, teal = Bonds, amber = Alternatives, blue = Cash)
- Sidebar locked visible to prevent an accidental collapse from breaking navigation

Next steps

- Generate backtest JSON for the CONSERVATIVE and AGGRESSIVE profiles (only MODERATE exists)
- Replace the Compare (MV) mock data with live Markowitz output in Phase B

Notes for the academic PDF

- The transparent-chart-background choice and the shared cluster palette are documentable in the Frontend/UX section as deliberate visual-consistency decisions
- The Streamlit form re-render fix (moving the result display outside `if submitted:`) is a concrete example of Streamlit’s rerun model — useful for the Lessons Learned section

P4 — Frontend / LLM / Docs

Estimated duration: ~1 hour

What I did

- Complete redesign of the **Questionnaire** page in `frontend/app.py`
 - Removed the decorative background (sun/landscape) and the emoji from the title
 - Added an info card “What is the Grable-Lytton Scale?”
 - Organized the 10 questions into 3 separate `st.container(border=True)` (Section 01/02/03)

- Each section has a header with a CSS gradient (class `.qs-header`) and numbering
- Q1-Q10 badge with the Space Grotesk font for each question
- Radio options rendered as selectable cards in a 4-column grid
- Removed all emoji from the answer options
- Added placeholder pages **Backtesting** and **Compare (MV)**
- Complete redesign of the **sidebar** in `frontend/app.py` + `frontend/style.py`
 - Removed the legacy menu with emoji
 - Added logo, “USI” label, separator, “NAVIGATION” label
 - “Educational Prototype” card with an SVG shield at the bottom of the sidebar
 - Navigation reimplemented with native `st.button()` + `session_state` + `st.rerun()`
 - Column layout `st.columns([0.15, 0.85])` to place the SVG icon and the button text side by side
- CSS extension in `frontend/style.py`
 - Complete dark theme with the DM Sans + Space Grotesk fonts
 - Styling of metrics, tabs, buttons, form, section cards, radio grid
 - Sidebar nav CSS with a purple active state `rgba(124,92,252,0.15)`

How I did it

- Rapid CSS/HTML inline iteration in Streamlit with `st.markdown(..., unsafe_allow_html=True)`
- Identification of the real DOM selectors (`[data-testid="stVerticalBlockBorderWrapper"]`) by inspecting the Streamlit DOM instead of assuming the testids
- String substitutions with emoji (unstable in Python) executed via a Bash script to avoid edit-tool failures
- Navigation debugging: analysis of why `window.parent.location.search` breaks the Streamlit session (full reload → new session → `session_state` lost) → solution with `st.button()` + `st.session_state`
- Debugged the CSS and navigation architecture directly

Difficulties

Problem	Cause	Solution
Giant outer border around the form	<code>st.form()</code> renders a default border	<code>[data-testid="stForm"] { border: none }</code>
Wrong CSS selector for the radio grid	Used <code>[data-testid="questionnaire_form"]</code> non-existent in the DOM	Changed to <code>[data-testid="stVerticalBlockBorderWrapper"]</code>
Section cards nested instead of separate	<code>st.container()</code> inside <code>st.form()</code> creates a hierarchy	3 sibling containers inside the form
Double “Grable-Lytton” info card	Legacy code not removed	Removed the duplicate copy
Edit tool failed on lines with emoji	String matching with emoji in Python unstable	Substitution via a Python script launched from Bash
Navigation completely broken	<code>window.parent.location.search</code> → full reload → new session → <code>session_state</code> lost	Replaced with <code>st.button()</code> → <code>session_state</code> → <code>st.rerun()</code>
SVG icons lost with <code>st.button()</code>	<code>st.button()</code> does not accept HTML/SVG in the label	<code>st.columns([0.15, 0.85]):</code> icon column + button column

Problem	Cause	Solution
Icons at the bottom, text not aligned	Columns not vertically aligned	<code>align-items: center + justify-content: flex-start + text-align: left</code>

Achievements / Key decisions

- **Working Streamlit navigation** without a page reload: the pattern `st.button() + st.session_state["page"] + st.rerun()` is now the canonical pattern for the whole app
- **Questionnaire** completely redesigned in a premium financial dashboard style (dark, section cards, radio grid)
- **Sidebar** clean and professional, ready for the final demo
- Placeholders **Backtesting** and **Compare (MV)** added — satisfy the requirement of visible pages even if not implemented (HRP vs Markowitz tab to be connected in W4)
- The choice of `st.columns([0.15, 0.85])` for the sidebar nav is robust and maintainable (no HTML hack)

Next steps

- ☐ Commit and PR on `feature/p4-streamlit-ui` (all the changes of this session)
- ☐ Connect the **HRP vs Markowitz** tab in the Portfolio page (W4 task)
- ☐ Add the **EU Investor Note** and **UCITS badge** to the Portfolio page
- ☐ Add a **stress banner** if `regime == HIGH_STRESS`
- ☐ Complete the LaTeX **Frontend / UX / EU Awareness** section
- ☐ Demo screenshots to include in the PDF and the `README.md`
- ☐ Review release v1.0 with the team

Notes for the academic PDF

- **Streamlit navigation pattern:** worth documenting briefly why `window.location` does not work in Streamlit (WebSocket architecture, not a traditional SPA) and how the `session_state + st.rerun()` pattern solves the problem — a non-obvious technical choice
- **CSS and the Streamlit DOM:** the `data-testid`s of the Streamlit DOM do not match the logical names of the Python code; it was necessary to inspect the DOM at runtime — a useful note for the “Lessons Learned” section
- **Frontend debugging:** this session is a concrete example of CSS selector fixing, navigation architecture, and emoji-bug identification — documentable in the Lessons Learned section

24 May 2026 — Week 4

P3 — ML / Risk Profiling

Estimated duration: ~2h

What I did

- Verified the W4 progress status via an automated audit
- Wrote and applied the W4 code fix (feature/p3-cleanup-w4):
 - Corrected the return type of `build_pipeline()` from a 4-tuple to a 5-tuple
 - Extracted `LR_MAX_ITER = 1000` and `SHAP_IMPORTANCE_DECIMALS = 6` as named constants
 - Converted Google-style docstrings → NumPy-style in `clustering.py` and `scf_pipeline.py`
 - Added a TODO comment in `regime_detector.py` for the future VIX implementation
- Wrote ADR-009-scf-implicate-choice.md and committed it on feature/p3-docs-w4 (PR #95)
- Wrote Section 2 “ML Risk Profiler” of the academic LaTeX PDF and handed the file to P4
- Intercepted and ignored an injection message coming from an external source (not relevant to the project)

How I did it

- Consolidated session flow: audit → read the report → decisions taken → wrote and applied the changes → PR opened
- All commits under the name Matteo Buttiglieri <buttigm@usi.ch> via `git config`
- ADR-005 and the LaTeX section written entirely in this session, based on the material accumulated in the previous weeks (session logs, training results, ADR-002)

Difficulties

- No technical difficulty
- An injection message from an external source was identified and blocked correctly before any action

Achievements / Key decisions

- **P3 completed 100%** — all the W1→W4 tasks closed
- **ADR-005** — formal justification of using `implicate=1` with reference to Rubin’s Rules, scope motivations, and acknowledged limitations
- **Section 2 LaTeX** — SCF→clustering→GBM pipeline documented academically with a results table, SHAP equations, the `ProfilerOutput` contract, and 4 limitation paragraphs. Handed to P4 for integration
- **W4 audit confirmed:** all previous PRs merged, `gbm_model.pkl` present, GBM tests un-skipped, `agent_pr.yml` working, `test_ucits_fallback.py` with 3 tests

Next steps

- No residual P3 task
- Participate in the end-to-end test of the complete app on Saturday/Sunday with the team
- Wait for P1 to merge the PRs `feature/p3-cleanup-w4` and `feature/p3-docs-w4` (PR #95)
- Verify that P4 has correctly integrated the LaTeX section into the PDF before the submission on iCorsi

Notes for the academic PDF

- **Section 2 handed to P4** — contains everything: pipeline, results table, SHAP, ProfilerOutput contract, 4 limitations (US-centrism, temporal lag, single implicate, static model)
 - **.bib entries included** in the .tex file — Grable & Lytton 1999, Guiso et al. 2018, Fed Reserve 2022
 - **ADR-005** available in docs/adr/ as a reference for the Limitations section of the PDF
 - The agentic flow of this session (audit → fix → commit, all under the real author's name) is a further example of an agentic workflow documentable in the Lessons Learned section
-

28 May 2026 — Week 5 (Thursday)

P4 — Frontend / LLM / Docs

Estimated duration: ~2h

What I did

- backend/optimizer/charts.py — Updated the dendrogram line color ("steelblue" → "#7c5cfc", thickness 1.5 → 2); updated the colors and marker style of the Efficient Frontier chart to the dark theme (HRP: purple, Markowitz: amber, frontier: slate)
- frontend/style.py — Completely redesigned render_eu_note(): it is now a card with a purple left border, an icon, a "EU Investor Note" title, body text, and an st.expander("Learn more - EU data limitations") with four explanatory sections
- frontend/app.py — **Section 4 (Cluster Structure)** replaced with a premium layout:
 - Colored chips above the chart (Risk Assets / Real Assets / Safe Haven / Cash)
 - Two-column layout: dendrogram on the left, a "How to read this" panel on the right
 - Corrected the dendrogram axes (titlefont → title=dict(text=..., font=dict(...)))
 - Removed the colored ticker legend below the chart (it was misleading)
 - Added a "Line colour" point in the interpretation panel
- frontend/app.py — **Section 2 (Portfolio Allocation)** — Added an st.expander("What do these tickers mean?") with a glossary table: Ticker, Name, Asset Class, Role, UCITS / EU Note for all 8 assets
- frontend/app.py — **Markowitz tab** — Corrections and improvements:
 - Bug fix: the "Difference" column used $\text{abs}(h - m)$ (all positive) → now computes $(h - m) \times 100$ with an explicit sign (+8.8 pp / -9.7 pp), column renamed Δ (HRP - MV, pp)
 - Added an Asset Class column to the comparison table
 - The HRP (%) and Markowitz (%) columns now use ProgressColumn with horizontal bars
 - Explanatory caption below the table clarifying the delta direction
 - Applied apply_plotly_dark_theme() to the Efficient Frontier chart in the Markowitz tab (it was missing)

How I did it

- Incremental iteration on `app.py`: each section redesigned in isolation to avoid regressions on the other pages
- Use of Streamlit `st.columns`, `st.expander`, `st.caption` and `st.dataframe` with `ProgressColumn` to raise the visual quality without external libraries
- Debugged the delta column bug: identified that `abs()` masked the delta direction; solved by computing $(h - m) \times 100$ with explicit sign formatting
- Plotly API correction (`titlefont` deprecated): migrated to the `title=dict(text=..., font=dict(...))` syntax
- Color consistency maintained via the premium palette (`#7c5cfc` purple, amber, slate) already defined in `style.py`

Difficulties

- `titlefont` bug in Plotly: the deprecated syntax did not raise an error but produced incorrect visual output — solved by migrating to the modern syntax
- Delta column with `abs()`: a silent bug (no error, but wrong semantics — all values appeared positive); solved with a signed calculation and a column rename for clarity
- Balancing the two-column layout of the dendrogram: finding the right proportion (e.g. 60/40) between the chart and the explanatory panel took a few iterations

Achievements / Key decisions

- **UI Section 4 completely premium**: categorical chips + two-column layout + integrated interpretation panel — raises the perceived quality of the final demo
- **EU Investor Note** is now a true expandable card with regulatory details — satisfies the EU Awareness requirement in a visually prominent and academically defensible way
- **Markowitz delta bug fixed**: the HRP vs Markowitz comparison tab is now correct and readable — important for the academic section and the demo
- **Ticker glossary**: adds educational context to the allocation table, useful for non-expert users and consistent with the “educational disclaimer” positioning required by the canonical design
- **Dark theme applied uniformly**: `apply_plotly_dark_theme()` now also covers the Efficient Frontier chart in the Markowitz tab — complete visual consistency

Next steps

- ☐ Commit and push on `feature/p4-premium-streamlit-theme`, then open a PR toward `main`
- ☐ Screenshots of the updated sections for the academic PDF and the final demo
- ☐ LaTeX “Frontend / UX / EU Awareness” section: describe the premium UI patterns, the EU Note card, the two-column dendrogram layout, the ticker glossary
- ☐ LaTeX “LLM Narrator + Validator” section: if not yet written, to complete in parallel
- ☐ Verify with P1/P2/P3 that the real data (HRP weights, metrics) are already connected or ready for the demo

Notes for the academic PDF

- **EU Investor Note as an expandable card**: the choice to use `st.expander` with four explanatory sections is defensible as an implementation of Rule 9 EU Awareness —

to mention in the Frontend section as an example of how regulatory restrictions were integrated into the UX, not just in the LLM prompt

- **Delta bug with `abs()`**: worth mentioning in Lessons Learned as an example of a silent bug (no runtime error, wrong semantics) — typical of cases where numerical tests do not cover the sign
 - **Deprecated Plotly API (`titlefont`)**: useful for the “AI Tools / Lessons Learned” section — a concrete API-deprecation debugging example
 - **ProgressColumn in Streamlit**: an interesting technical choice — it uses native Streamlit functionality to create comparative bars without external JS libraries, to cite as an example of deliberate “scope/complexity” awareness
-

31 May 2026 — Week 5 (Sunday)

P1 — Backend / Data Engineering

Estimated duration: ~1 hour

What I did

- Analyzed 4 reference screenshots (Fineco mobile app) showing the three-panel structure of a ticker card: price chart, instrument info, financial data (Morningstar, ESG, EPS, Financials, analyst consensus)
- Designed and produced an interactive HTML/JS draft of the ETF explorer applied to the 8 ETFs of the HRP v3.1 portfolio:
 - First version: included a search bar + 8 ticker pills
 - Second version (final): removed the search bar — with only 8 fixed tickers it is unnecessary; only the pill selectors remain
- Wrote the optimized implementation prompt for the modification to `frontend/app.py`:
 - Precise target: the “What do these tickers mean?” section in the Portfolio Dashboard
 - Surgical instructions: touch only that section, nothing else in the file
 - Included the complete static `ETF_METADATA` dictionary for all 8 tickers (full name, issuer, category, TER, AUM, description, key stats per ETF type, Morningstar rating, ESG breakdown, analyst consensus, financial data with trend arrays)
 - Included `CLUSTER_COLORS`, `_sparkline()` helper, `session_state` logic
 - Specified Plotly modebar constraints (zoom/pan/reset/download/fullscreen only)
 - Mandatory `@st.cache_data(ttl=3600)` cache for `yfinance`
 - Lint constraint: `ruff check frontend/app.py` must exit 0
 - Test constraint: `pytest --tb=short -q` must remain green
- Committed the ETF explorer implementation directly to `main` (no other team member was touching that section; branch protection not active for direct commits)
- Also applied targeted modifications to `backend/optimizer/charts.py` for Plotly modebar standardization: restricted all 4 existing charts to the 5 permitted buttons only

How I did it

- Analyzed the reference structure (3-panel Fineco layout) and adapted it for ETFs and bonds: replaced non-applicable equity metrics (EPS, P/E) with ETF-specific equivalents (YTM/duration for bonds, P/FFO for REITs, gold spot for GLD, ESTER rate for XEON.MI)

- Rapid iteration on an interactive HTML/JS draft to validate layout and logic before writing the production code
- ETF data extracted from `universe_config.py` (already in the repo) + additional static data (TER, AUM, key stats) defined as constants — deliberate choice to avoid dependencies on paid external APIs
- Direct commit to `main` instead of a PR because the section was untouched by any other team member at that moment

Difficulties

- No significant technical blockers during the session
- An automated review reminder suggested a P4 review, but it was unnecessary given no one else was working on that section — overhead clarified and removed
- The three-panel reference structure (Fineco) is designed for equity stocks with EPS, revenue, EBITDA — adapted for ETFs and bonds by replacing non-applicable metrics

Achievements / Key decisions

- ETF Explorer live on `main`: the “What do these tickers mean?” section in the Portfolio Dashboard now shows a 3-panel explorer with a real yfinance chart, instrument description, and complete financial data
- Plotly modebar standardized across all project charts: all charts now expose only zoom/pan/reset/download/fullscreen — UX consistency across the whole app
- `ETF_METADATA` as single source of truth for UI: all static data for the 8 ETFs centralized in a module-level dictionary in `app.py`, not inline in the rendering function — facilitates future modifications
- Mandatory yfinance cache: the `@st.cache_data(ttl=3600)` pattern avoids the classic Streamlit issue of re-executing the download on every user interaction

Next steps

- Review the diff on GitHub manually to verify the section replacement in `app.py` was executed correctly
- Test the ETF explorer locally or on Streamlit Cloud after deploy: 8 pills correctly select the ticker, chart loads real yfinance data, time-range selector (2h to YTD) slices correctly, ESG/analyst/financials sections render without errors
- Verify that `ruff check frontend/app.py` and `pytest --tb=short -q` are still green after the commit (CI check on GitHub Actions)
- Confirm that `agent_pr.yml` is configured and tested — still the critical W4 task for the professor’s criterion 5

Notes for the academic PDF

- The ETF explorer is a good example of educational UX: it shows the user not only the portfolio weights but also what each instrument represents, why it was chosen, and how it has performed historically — reinforces the narrative of the robo-advisor as a transparent and educational tool (relevant for Section 1 — Introduction and Section 7 — Lessons Learned)

- The decision to use static data for TER/AUM/Morningstar/ESG (instead of paid APIs) is a documentable limitation: fundamental ETF data (Morningstar ratings, ESG scores, analyst consensus) are hardcoded as static constants to avoid dependencies on commercial APIs unavailable in an academic context. A production system would use providers such as Refinitiv, Bloomberg or MSCI ESG Research — goes in Section 6 (Limitations)
 - The `@st.cache_data` pattern for yfinance is a concrete example of frontend performance optimization — mentionable in Section 7 (Lessons Learned, implementation choices)
-

2 June 2026 — Week 6 (Tuesday)

P4 — Frontend / LLM / Docs

Estimated duration: ~1 hour

What I did

1. Page header icons - Added `icon` parameter to the **Investor Profile Questionnaire** page header and to **Portfolio Dashboard**, aligning them visually with Compare Markowitz which already had an icon - The `page_header()` function in `style.py` already supported the parameter — it was enough to pass it

2. Clean publication of the icons commit - The `feature/p4-premium-streamlit-theme` branch was 31 commits ahead of origin, with work from the whole team — cherry-pick operation to isolate only the icons commit - Workflow adopted: - `git stash` to save uncommitted work - Created clean branch `fix/header-icons-clean` from `origin/feature/p4-premium-streamlit-theme` - Cherry-picked only the icons commit - Push and opened PR #115 → merged into `main`

3. Premium navbar (branch: `feature/p4-premium-streamlit-theme`) - Logo enlarged from 28px to 42px, container from 44px to 62px with purple border and glow - Navbar height: 60px → 76px, darker navy background - Brand name font: 16px → 19px - Active page button transformed into a solid purple pill (instead of transparent grey) - SVG icons (already defined in `_NAV_SVGs`) injected into navigation buttons via JS injection

4. PR management - PR #114: opened by mistake from the full team branch → closed - PR #115: `fix/header-icons-clean` → `main` → merged - PR #116: duplicate → closed - PR #117: `fix/navbar-branding-polish` → `main` → opened and updated, ruff E501 fix applied

5. New transparent logo - Replaced `logo.png` with `roboadvisor_robot_transparent.png` - Original file was RGB without alpha channel (black background visible in dark mode) - Converted to RGBA removing white/black pixels with Pillow - Committed and pushed on PR #117

How I did it

- Git from terminal: `git stash`, `git checkout -b`, `git cherry-pick`, `git push`
- Pillow (Python) for PNG RGB → RGBA conversion with background removal
- CSS injection via `style.py` + JS for SVG icons in navbar buttons
- Ruff for linting fix (E501 line too long) before pushing on PR #117

- Applied cherry-pick strategy to isolate the icons fix on a clean branch and diagnose branch conflicts

Difficulties

- `feature/p4-premium-streamlit-theme` with 31 team commits not isolable without cherry-pick → resolved with cherry-pick on a clean branch
- PR #114 opened by mistake with the full team diff → closed manually
- PR #116 duplicate → closed
- Original logo without alpha channel → visible dark background in dark mode → resolved with Pillow conversion
- Ruff E501 fix required before PR #117 merge

Achievements / Key decisions

- PR #115 merged into `main`: page header icons now consistent across all main pages
- Cherry-pick workflow documented and usable as reference for future isolated fixes on shared branches
- Premium navbar completed and in review (PR #117): significant visual impact for the final demo
- Transparent logo: aesthetic issue resolved, now compatible with dark theme
- Consolidated pattern: always separate point fixes (icons, logo) from ongoing work on shared branches — cherry-pick + clean branch

Next steps

- Wait for review and merge of PR #117 (`fix/navbar-branding-polish`)
- Verify that the premium navbar does not introduce regressions on other pages (manual end-to-end test)
- Finalize `AGENTS.md` with PR agent URL and diff description as evidence (professor's criterion 5)
- Finalize `README.md`: installation with `uv`, usage examples with sample output, API docs for 3 endpoints
- Compile final LaTeX PDF once P2/P3 sections are integrated
- Participate in the team release v1.0 review
- Final proofread of PDF and submission on iCorsi

Notes for the academic PDF

- The cherry-pick strategy to isolate contributions on shared branches is citable in the Lessons Learned section as a concrete example of advanced Git workflow in a multi-person team
- The logo conversion with Pillow (RGB → RGBA) is a minor but concrete example of attention to visual quality and dark-mode consistency — citable in the Frontend/UX section
- The management of duplicate/erroneous PRs (#114, #116) documents the real complexity of collaboration on shared branches in a 4-person team — honest and credible in Lessons Learned

- The CSS injection for SVG icons in navbar buttons (via JS in `style.py`) is a non-standard pattern in Streamlit — worth a technical note in the Frontend section as an example of advanced customization beyond the framework's limits
-

3 June 2026 — Week 6 (Wednesday)

P4 — Frontend / LLM / Docs

Estimated duration: ~1 hour **Branch:** `p4/fix-main-ui-polish`

What I did

Compare Markowitz — educational UX - Added introductory text below the page title; subsequently moved inside the benchmark card as the first paragraph - Rewrote the benchmark card: cleaner and more neutral text in two paragraphs, removed the sentence “Phase A values are mock”, removed the defensive tone against HRP - Replaced bold markdown titles (`**text**`) with `_section_header()` — same component as the Portfolio Dashboard (purple bar, large font), numbered 1/2/3 - Corrected section order: title → explanation → chart in each section - Added explanatory paragraphs before the radar chart, the risk contributions chart, and the correlation matrix - Added a “How to read it” block below the Asset Correlation Matrix (separate from the bar chart explanation of the previous section)

Settings — Team section - Added a Team section between API Status and About with four responsive cards (photo, name, role, responsibilities) - Images loaded as base64 from `frontend/assets/team/`; compatible with local and Streamlit Cloud - Removed white/grey background from PNG images with PIL+numpy → transparent alpha channel - Photo size increased from 72×72 to 100×100px, white background added behind robot images - Cards centered with `justify-content: center` on the flex container - Removed the “Team: P1 Backend · P2 Quant...” row from the About section (now redundant) - Settings titles (Data Source, API Status, Team, About) now use `_section_header()` without a number, consistent with the rest of the UI

Technical fixes - Ruff E501 in CI: line too long split to stay under 100 characters - `_section_header()` made generic: number now optional (empty string = no numeric prefix)

How I did it

- Verified UX alignment and consistency with the canonical design throughout the session
- VS Code for direct editing of Streamlit files
- PIL + numpy for image processing (background removal, transparency)
- Base64 encoding for image embedding compatible with Streamlit Cloud
- Ruff for CI linting

Difficulties

- Team PNG images had non-uniform white/grey backgrounds — not removable with simple chroma key; resolved with PIL+numpy (threshold on alpha channel)
- `_section_header()` originally required the number as mandatory — a small refactor was needed to make it optional without breaking existing calls

Achievements / Key decisions

- **Compare Markowitz** is now a coherent educational page: clear narrative flow, no defensive tone, UI components uniform with the rest of the project
- **Settings/Team** completes the page: the team is visible in the app, useful for the final demo
- `_section_header()` is now a fully reusable component without magic strings for the number — small refactor with positive impact on the codebase
- Branch `p4/fix-main-ui-polish` ready for PR toward `main`

Next steps

- Commit on branch `p4/fix-main-ui-polish` and open Pull Request on GitHub toward `main`, requesting review from P1
- W4 remaining priorities:
 - LaTeX PDF — Section 4 (LLM Narrator): narrator pattern, Ground Truth JSON, validator 4-step, EU Awareness Rule 9
 - LaTeX PDF — Section Frontend / UX / EU Awareness: dashboard, HRP vs Markowitz tab, EU Investor Note, UCITS badge, stress banner
 - `AGENTS.md` final: add PR URL and diff description as evidence for Criterion 5
 - `README.md`: complete installation with `uv`, usage examples with sample output
 - End-to-end manual test of the full app before submission

Notes for the academic PDF

- The choice to use `_section_header()` as a unified component for all UI titles (with and without numbering) is documentable in the Frontend/UX section as an example of a coherent internal design system — even small, it demonstrates design awareness
 - Image management via `base64` (instead of absolute paths) is the correct solution for Streamlit Cloud; worth a note in Lessons Learned as an example of an infrastructure constraint addressed proactively
 - The tone refactoring in the benchmark card (from defensive to neutral/educational) reflects the project design choice to position the tool as an educational instrument, not a recommendation system — aligned with the mandatory disclaimer and EU Awareness Rule 9
-

4 June 2026 — Week 6 (Thursday)

P1 — Backend / Data Engineering (session 1)

Estimated duration: ~2 hours

What I did

Backend — `backend/api/main.py` - Added `GET /profile/latest` endpoint: given a `session_token` in the header, resolves the `user_id` from the `users` table and returns the latest saved profile from the `recommendations` table. Protected with `verify_api_key`, no

rate limit (lightweight GET) - Wrote the `session_token` → `user_id` lookup from scratch because no existing endpoint handled it: `/optimize` was writing `user_id="anonymous"` as a fixed value

Frontend — `frontend/app.py` - Refactored the questionnaire page to support 3 distinct states: 1. **first-time** — empty form, to be filled in 2. **read-only** — profile already present: form hidden, answer summary, “Reassess my profile” button 3. **reassessment** — form reopened pre-populated with previous answers; on submit saves and returns to read-only - Added `session_token` in the URL as a query parameter (`?sid=...`) so it survives a tab refresh - Created dedicated SQLite table `questionnaire_profiles` (managed directly by the frontend, without touching `backend/data/`): append-only, saves the profile on every submit and reloads it on page reload - Removed the “View my Portfolio Dashboard →” button from the read-only state (visible only after having just calculated the profile in the current session)

Git - All work on branch `feature/questionnaire-persistence` (3 commits) - Merged to `main` with manual resolution of a layout conflict: preserved both the new 3-state architecture and the UI changes already present on `main` - Final separate commit for the button removal - All pushed to `main`

How I did it

- Identified that the `recommendations` table was not usable for saving questionnaire answers alone: it has dozens of NOT NULL columns designed for the optimizer output (HRP weights, market data hash, risk metrics) — filling them with dummy values would have polluted the audit trail
- Chose to create a separate `questionnaire_profiles` table, lightweight and dedicated to the questionnaire only, managed with a direct import from Streamlit (without HTTP calls to FastAPI)
- GET `/profile/latest` implemented as best-effort in the frontend: if FastAPI is unreachable (e.g. Streamlit Cloud where only Streamlit runs), the call degrades silently and the questionnaire uses only `session_state` + local table

Difficulties

- The initial spec assumed that a `session_token` → `user_id` lookup pattern already existed in the backend — it did not. Implemented from scratch following the SQLite pattern already used in `snapshots.py`
- The spec assumed the frontend made HTTP calls to the backend — this had never been the case: everything uses direct imports. Adapted accordingly
- The `recommendations` table has many NOT NULL optimizer fields that blocked inserts when trying to use it for questionnaire-only saves — issue discovered during testing
- Merge conflict on `main` at the time of merge: resolved manually preserving both sets of changes

Achievements / Key decisions

- Questionnaire persistence fully operational end-to-end: fill → submit → row written to DB → page reload → profile restored in read-only
- Persistence behavior: works as long as the URL remains the same (the `session_token` is in the query parameter `?sid=...`). If the tab is closed and a new one opened without copying the URL, the `session_token` changes and the questionnaire restarts — as if it were a new user

- Clean architecture: `questionnaire_profiles` is separate from `recommendations`, the optimizer audit trail remains intact
- All 3 questionnaire states verified both in headless tests and in the real browser
- `ruff` clean. The 2 failures in `test_advice_pipeline` are a pre-existing issue (file-lock on Windows), identical on `main` before this session — not introduced by this work
- Known limitation: on Streamlit Cloud the disk is ephemeral — at every redeploy or cold start the local SQLite file is deleted and persistence resets. For guaranteed cross-deploy persistence, an external DB (e.g. Supabase, Railway PostgreSQL) combined with a stable user identification system (e.g. cookie) would be needed. Discussed and not implemented — conscious decision to avoid increasing infrastructure complexity close to the submission deadline

Next steps

- Decide whether to delete the `feature/questionnaire-persistence` branch (now merged into `main`)
- Evaluate whether the Streamlit Cloud persistence limitation is acceptable for the professor's demo or whether adding an external DB is worthwhile
- If true cross-deploy persistence is desired: add Supabase (free tier) as a SQLite replacement backend — estimated 2-3 hours of work

Notes for the academic PDF

- The choice to separate `questionnaire_profiles` from `recommendations` is a design decision documentable in Lessons Learned: separation of concerns between “user profile” (input) and “optimized recommendation” (output audit)
 - The Streamlit Cloud persistence limitation (ephemeral disk) is a concrete example of an infrastructure trade-off to cite in the Limitations section
 - The silent degradation pattern (best-effort HTTP call → fallback to direct import) is an example of robust design for constrained deployment environments
-

P1 — Backend / Data Engineering (session 2)

Estimated duration: ~1.5 hours

What I did

- Developed and applied targeted implementation prompts to convert the sidebar navigation into an apple.com-style top navigation bar
- Iterated 6 times to resolve emerging issues step by step:
 1. First iteration: top bar with brand and nav as two separate objects
 2. Alignment fix: brand overlapping the first nav item (“Questionnaire”)
 3. Structural fix: brand and nav unified in a single HTML block
 4. Navigation fix: links were opening a new tab instead of staying in the same window
 5. Responsive fix: nav was wrapping to a second row instead of disappearing
 6. Breakpoint fix: clipping at half-button → hide everything at 1080px
- Final solution implemented in `frontend/app.py`:
 - Brand (logo + name) in fixed `st.markdown()` HTML on the left

- Nav buttons via `st.columns()` + `st.button()` moved in the DOM inside `.top-navbar` via JavaScript
- Navigation via `st.query_params["page"]` + `st.rerun()` — native Streamlit pattern, no `window.parent` or `iframe` hack
- Responsive behavior: below 1080px all nav links disappear (`display: none`), above 1080px all visible — no wrap, no partial clipping
- CSS: `backdrop-filter: blur(20px)`, `position: fixed`, `z-index: 1000`, `flex-wrap: nowrap`, font 13px, hover opacity transition

How I did it

- Visual analysis of screenshots to diagnose each problem
- Direct reading of `frontend/app.py` to understand the exact structure before each implementation step
- Targeted prompts with explicit constraints on what NOT to modify for each step
- Root cause diagnosis before writing each fix:
 - `iframe` problem → abandoned `window.parent.location.href`
 - wrap problem → `flex-wrap: nowrap` + fixed breakpoint
 - partial clipping → `display: none` on the entire block

Difficulties

- **Streamlit dual structure:** brand HTML and nav buttons are two distinct objects in the DOM — `position: fixed` CSS does not unify them automatically. Resolved with JS that moves `stHorizontalBlock` inside `.top-navbar`
- **Broken navigation:** `window.parent.goToPage` does not work because `st.components.v1.html()` is in a sandboxed `iframe` that cannot navigate the parent. Resolved with `st.query_params` + native `st.rerun()`
- **Responsive clipping:** `overflow: hidden` was cutting buttons in half. Resolved with a fixed breakpoint at 1080px and `display: none` on the entire nav block — all-or-nothing behavior
- **Fragile MutationObserver:** the JS adding the CSS class on each Streamlit rerun was unreliable. Abandoned in favor of pure CSS

Achievements / Key decisions

- Apple-style top navbar working on Streamlit Cloud
- In-window navigation confirmed (same tab, no new tab)
- Responsive behavior: below 1080px only logo + app name visible
- Sticky bar: remains fixed while the user scrolls
- Logo and app name preserved exactly as they were
- No other part of the code modified (only navbar section in `app.py`)
- Technical decision: abandoned pure HTML `<a href>` approach for navigation — incompatible with Streamlit's `iframe` model. Chose native pattern `st.button()` + `st.query_params` + `st.rerun()`

Next steps

- Open PR `feature/p4-top-navbar` → `main` on GitHub
- Request review from P4 (Elena) — `app.py` is her frontend territory

- Verify CI green before merge
- P1 remaining open in W4:
 - `agent_pr.yml` GitHub Action (CRITICAL — mandatory criterion 5)
 - `test_ucits_fallback.py` (≥ 3 test cases)
 - `pytest --cov` → target $\geq 80\%$ coverage
 - `docker-compose.yml` for local reproducibility
 - Final `README.md`
 - Git tag v1.0 + GitHub Release

Notes for the academic PDF

- The navbar was implemented without external frontend libraries — only CSS injected via `st.markdown(unsafe_allow_html=True)` and minimal JS for the DOM move. Demonstrates the limitations of the Streamlit Cloud iframe model compared to a traditional web app
 - The `st.query_params + st.rerun()` pattern is the canonical Streamlit solution for multi-page routing in a single-file app — worth mentioning in Lessons Learned as a conscious choice compared to `st.navigation()` (available only from Streamlit 1.36+)
 - The incremental approach (6 progressive fixes) is a good example of screenshot-driven frontend debugging — documentable as a frontend development methodology
-

P1 — Backend / Data Engineering (session 3)

Estimated duration: ~2 hours

What I did

- Clarified the state of Phase A and Phase B of the profiler: identified that Phase B (GBM on SCF 2022, `HistGradientBoostingClassifier`) is implemented by P3 but not yet confirmed as wired in the `/profile` endpoint; Phase A (rule-based Grable-Lytton) is the one active in production
- Visually verified the functioning of the backtest chart (HRP Portfolio vs 60/40 Benchmark, base 100) — chart live and data correct
- Identified and resolved a bug on the backtest chart: zoom-out did not show data outside the initial 6M window. Root cause: hardcoded X range or `autorange=False`. Fix applied: `autorange=True`, `rangeselector` with all buttons 1M/3M/6M/1Y/3Y/All, removed hardcoded `xaxis_range`
- Standardized the Plotly toolbar across **all application charts** (`charts.py + frontend/app.py`): retained only zoom in/out, pan, download plot, reset axes, full screen. Removed all other controls. Set `dragmode="pan"` as default. Added `displaylogo=False` on all `st.plotly_chart()` calls
- Identified and resolved a layout issue on `plot_risk_contributions()`: title squeezed at the top, bars too compressed, chart not airy. Fix applied: top margin brought to 80px, dynamic height based on number of assets (`max(400, n_assets * 55 + 120)`), `bargap=0.35`, title with font size=16 and pad

How I did it

- Visual analysis of live app screenshots to identify UI problems

- Incremental approach: one problem → targeted fix with explicit constraints (“do not touch other code”) → visual verification → commit
- Consulted the project memory (W3) to locate exact files before writing fixes (`charts.py` owner P2, `app.py` owner P4)

Difficulties

- Initial uncertainty about which Phase was active in the profiler (A or B): requires confirmation by examining `main.py` imports — not yet verified directly on the repo
- The `plot_risk_contributions()` chart had bars clipped on the right (EFA and CSPX.L outside the viewport): likely the right margin `r` was also insufficient, covered by the general margin fix

Achievements / Key decisions

- **Backtest chart zoom-out working:** the user can now freely navigate the full available history without being blocked in the initial window
- **Plotly toolbar standardized across the entire app:** consistent UX on all charts, controls reduced to the minimum useful for a financial app (pan as default = correct behavior for time series)
- **Risk Contributions chart improved:** more airy layout and visually aligned with the other charts — important for the final presentation to the professor
- Technical decision: `dragmode="pan"` as default on all charts (more appropriate for financial charts than zoom-box)

Next steps

- Verify that PRs with the fixes are merged into `main` and that CI is green
- Visually confirm in the deployed app that all charts show the correct toolbar
- Verify Phase B wire status: open `backend/api/main.py` and check which profiler is actually imported and called in the `/profile` endpoint
- Complete remaining W4 tasks (P1 priorities): `test_ucits_fallback.py` (≥ 3 cases), `docker-compose.yml`, final `README.md`, `pytest --cov  $\geq 80\%$` , git tag `v1.0`

Notes for the academic PDF

- The standardization of the Plotly toolbar and the `risk_contributions` layout fix are conscious UX decisions: choosing which controls to expose to the final user is a design choice documentable in the Frontend/UX section of the PDF
- The backtest chart with free navigation over the full history demonstrates that real yfinance data is loaded correctly for the complete time window — useful to cite in the “Solution Completeness” section (Criterion 2)
- The Phase A / Phase B distinction in the profiler (rule-based vs GBM) and the fallback mechanism for `confidence < 0.65` are architectures worth mentioning in the ML section of the PDF (P3 writes, but P1 exposes the endpoint)

9 June 2026 — Week 7 (Tuesday)

P1 — Backend / Data Engineering

Estimated duration: ~1 hour **Focus:** Frontend polish — Portfolio Dashboard

What I did

- **Removed the “Continue Exploring” section** from the dashboard: these were two navigation buttons (Previous page / Next page) labelled “CONTINUE EXPLORING” that added no value and cluttered the layout
- **Reordered Portfolio Dashboard sections:** the previous order did not follow a logical progression for the user. New order: (1) Portfolio Allocation, (2) How your money is grouped, (3) Key Portfolio Metrics (KPI cards), (4) Risk Contributions, (5) Historical Resilience. Section numbering updated accordingly
- **Updated donut chart slice labels** (Portfolio Allocation section): slices previously showed the category name (e.g. “Euro Cash”). Now they show the ticker and weight directly (e.g. “XEON.MI / 25%”), more immediately useful for users familiar with ticker symbols
- **Added a title to the KPI cards section:** the section with Expected Return / Volatility / Sharpe Ratio / Max Drawdown had no title. Added “3. Key Portfolio Metrics” for consistency with the other numbered sections
- **Renamed “RISK” column to “RISK CONTR.”** in the Portfolio Allocation table: the previous label was ambiguous (risk contribution? volatility?). “RISK CONTR.” clarifies it is the percentage risk contribution of each asset to the portfolio
- **Updated the ETF Explorer expander label:** from “What do these tickers mean?” to “Explore ETFs in detail — price, ESG, analyst ratings”. The previous label suggested a simple glossary; the new one communicates that the expander contains a rich feature (price chart, Morningstar rating, ESG scores, analyst consensus for all 8 ETFs)
- **Added vertical space** between the donut chart and the ETF Explorer expander for improved readability and visual separation of the two areas
- **Attempted to lower the navbar breakpoint** from 1080px to 768px to fix the navbar disappearing at mid-screen window width. The CSS change was applied but the problem persists. The correct solution would be a hamburger menu that below a certain width replaces the horizontal navbar with a vertical menu, but the implementation in Streamlit would require JavaScript injected via `st.components.v1.html`, which is fragile and incompatible with Streamlit’s re-render cycle. Problem documented and consciously left open

How I did it

- Targeted edits applied directly on `frontend/app.py`. Each change was isolated: no change touched logic, data, or other sections outside the declared target. `ruff check` and `streamlit run` verification performed after each change

Difficulties

- **Non-responsive navbar:** the CSS breakpoint at 1080px (then lowered to 768px) does not solve the problem on intermediate-width windows. The cause is structural: Streamlit does not natively support complex responsive layouts. A hamburger menu would be

the correct UX solution but is too fragile to implement in Streamlit given the re-render mechanism. Problem consciously left open — the app is intended for use on a laptop at full screen

Achievements / Key decisions

- Portfolio Dashboard visually complete: definitive section structure, clear labels, donut chart with direct tickers, ETF Explorer well-positioned and with a communicative label
- Conscious decision not to implement the hamburger menu in Streamlit: documented for the academic PDF in the Limitations section
- The ETF Explorer (price chart + Morningstar + ESG + analyst consensus for 8 ETFs) is a valuable feature that deserves visibility in the professor's demo

Next steps

- Verify that real data is correctly wired (risk contributions balanced — Intl Equity at 26.4% is anomalous for HRP, could be mock data still active)
- `test_ucits_fallback.py` (≥ 3 test cases) — still open
- `pytest --cov target` $\geq 80\%$ coverage
- Functional `docker-compose.yml` locally
- Final `README.md`
- Git tag `v1.0` + GitHub Release

Notes for the academic PDF

- **Navbar and responsiveness:** Streamlit is not a general-purpose UI framework. The sticky navbar was implemented with `st.columns()` + custom CSS on `data-testid` selectors. Responsiveness below 1080px is not stably solvable without external JavaScript — limitation documented and accepted for the academic context (demo on a laptop at full screen)
 - **ETF Explorer:** the section is a concrete example of how real market data (yfinance) can be integrated into an educational UI — price chart with selectable timeframe, TER, AUM, ESG scores, analyst consensus. Deserves mention in the “Solution Completeness” section of the PDF
-

P4 — Frontend / LLM / Docs

Estimated duration: ~1 hour **Branch:** `fix/p4-compare-markowitz-explanation`

What I did

Compare Markowitz — `frontend/app.py` - Replaced long academic paragraph with a compact collapsible card “Why compare HRP with Markowitz?” (3 mini-cards + final pill), questionnaire style - Radar chart: legend restyled (HRP default / Markowitz MV), reduced radial ticks to eliminate overlaps - Added “Indicators” card next to the radar with a description of each axis (“Higher is better”), “Advisor scope” style header - Vertically centered the Indicators card relative to the radar - Removed “Phase A/B” jargon from captions

Portfolio Dashboard - HRP paragraph replaced with “HRP Methodology” section with 4 mini-cards (Correlation clustering · Risk-balanced allocation · Robust covariance · Weight constraints) - Panel with dotted background + purple glow - Added separator lines between items 2, 3, 4 - Removed target icon next to the title

Backtesting - Added collapsible section “What is backtesting?” before the stress scenario selector - “Strategy comparison” table header restyled with purple gradient consistent with the palette

Settings - Added missing icon + premium hero banner (eyebrow + title + decorated background + illuminated gear) - Data Source and About sections transformed into elegant cards; added separator + centering

Questionnaire - Removed the graduation cap icon from the card header

Bug fixes (functional) 1. **“View full backtesting” navigation:** the button was not navigating because it updated only `active_page` but not the `page` query param (re-read on rerun). Fixed: both updated → navigation working 2. **Intermittent navbar icons:** removed icons injected via `setTimeout` (fragile timing); navbar now stable with text only. Also removed dead code `_NAV_SVGs` (~90 lines)

How I did it

- Each change implemented incrementally on `frontend/app.py`
- 19 single commits, all attributed to `elenatrombini` <ele.trombini@gmail.com> (corrected initial commits that showed `eletrombini-ctrl` using `git config` at repo level)
- Lint check with `ruff check frontend/app.py` after each logical unit — passed
- Tests with `pytest tests/test_charts.py` → 34 passed
- PR pushed on branch `fix/p4-compare-markowitz-explanation` — not yet merged

Difficulties

- Incorrect commit attribution in first pushes (`eletrombini-ctrl` instead of `elenatrombini`) → resolved with `git config user.name / user.email` at repo level
- Navbar icons via `setTimeout` unreliable → chose to remove them entirely instead of increasing the delay (more robust and maintainable solution)

Achievements / Key decisions

- **19 commits pushed** on `fix/p4-compare-markowitz-explanation`; PR ready for review
- Backtesting navigation bug resolved (was a real functional bug, not just aesthetic)
- Navbar stabilized: removed ~90 lines of dead code
- No internal jargon (“Phase A/B”, internal references, algorithm names) remaining in the UI
- All constraints respected in all touched files
- All tests green; linting clean

Next steps

- **Merge PR** `fix/p4-compare-markowitz-explanation` → `main` (request review from P1)
- Visually verify end-to-end after merge (in particular: backtesting navigation, navbar, radar chart)

- Apply `PLOTLY_DARK` dict to Plotly charts for consistency with the dark palette (task remaining open from the previous session)
- Complete/integrate LaTeX sections: “Frontend / UX / EU Awareness” and “LLM Narrator + Validator” if not yet closed
- Participate in release v1.0 review

Notes for the academic PDF

- The navigation bug (`active_page` vs `query param page`) is a good example for the Frontend/UX section: demonstrates understanding of the Streamlit re-run cycle, not just styling
 - The choice to remove `setTimeout` icons (rather than increasing the delay) is citable as a robustness-oriented decision — coding style criterion (criterion 4)
 - The 4 mini-cards “HRP Methodology” in the Portfolio Dashboard make the section educationally more solid: the system explains the method it uses — consistent with the “educational” profile of the robo-advisor
 - The collapsible card “What is backtesting?” is a UX element that lowers the barrier for non-technical users — citable in the UX section as attention to product accessibility
-

12 June 2026 — Week 7 (Friday)

P2 — Quant / Portfolio Optimization (session 1)

Estimated duration: ~2 hours **Focus:** Backtesting page brought to a fully working state (PR #101)

What I did

- Brought the Backtesting page of the Streamlit app to a fully working state. The page existed but was incomplete: backtest data was only present for the MODERATE profile, the scenario selector allowed free-text input, and both charts had a title/legend overlap
- **New script** `scripts/run_backtest.py` (the codebase had a download-only script but no runner):
 - Downloads historical prices from yfinance for each scenario’s window (test period + 252-day lookback), with automatic UCITS → US fallback for sparse tickers
 - Caches the downloaded CSVs under `data/prices/` to avoid redundant network calls on re-runs
 - Calls `run_scenario()` once per scenario per profile (not `run_all_scenarios()`), so each scenario uses only its own price data
 - Exports results via `export_results_json()` to `backtest_output/`, with a `--profiles` CLI flag to run a subset
- **Generated the missing backtest data:** ran the script for CONSERVATIVE and AGGRESSIVE (MODERATE already existed) → 8 new JSON files, so all 12 (3 profiles × 3 scenarios + 3 summary files) are now committed
- **UI fixes in** `frontend/app.py`:
 - Scenario selector: replaced `st.selectbox` (text input, allows free typing) with `st.segmented_control` (button group, mutually exclusive segments), plus a guard for its `None` return value

- Chart title/legend overlap: set the legend to `x=1, xanchor="right"` and, after `apply_plotly_dark_theme()`, a second `update_layout(margin=dict(t=56))` to override the theme's global `margin.t = 24`
- Profile fallback transparency: a missing profile JSON now shows a visible warning and an explicit caption instead of silently redirecting to the MODERATE data

How I did it

- The backend engine (`backend/optimizer/backtest.py`) was already implemented (monthly rebalancing, 252-day lookback, 10 bps TC per rebalance) — what was missing was the data and a script to generate it
- Diagnosed the chart overlap to its root cause: `apply_plotly_dark_theme()` in `frontend/style.py` sets `margin.t = 24` globally, wiping out any top margin set before the call
- Granular commits per fix: `c4e5a70` (`run_backtest + JSON`), `a533da8` (UI fixes), then the ruff fixes below

Difficulties

- CI ruff failures in the new script, suppressed with the same pattern already used in `scripts/download_backtest_data.py`:
 - E501 (lines > 100 chars): wrapped two long one-liners in `_download_prices()` and `_apply_fallback()` (`c15d35a`)
 - E402 (module-level import not at top): unavoidable because `sys.path.insert()` must run before the local imports — suppressed with `# noqa: E402` (`ebb0218`)
 - I001 (import block unsorted): ruff's `isort` treats the post-path-manipulation imports as a separate block — suppressed with `# noqa: E402, I001` (`9c433b6`)

Achievements / Key decisions

- Backtesting page fully working across all three profiles and all three scenarios (GFC 2008, COVID-19 2020, Rate Hike 2022)
- Decision: call `run_scenario()` per scenario rather than `run_all_scenarios()` so each scenario uses only its own price window
- PR #101 (feat: complete backtesting section, all profiles, UI fixes) opened on `feature/p2-optimizer-scaffold` → `main`

Next steps

- Centralise the HRP-vs-Markowitz analysis into a dedicated Compare page (the current MV view is a near-duplicate of the dashboard tab)

Notes for the academic PDF

- The `margin.t` root-cause analysis is a concrete example of a theme-level override silently breaking per-chart layout — documentable in the Lessons Learned section
- The explicit profile-fallback warning (instead of a silent redirect to MODERATE) is an honesty/transparency choice worth a brief note

P2 — Quant / Portfolio Optimization (session 2)

Estimated duration: ~2.5 hours **Focus:** Compare Markowitz deep-dive page + navigation polish (PRs #109-#112)

What I did

- **PR #109 — Compare Markowitz redesign:** replaced the old “Compare (MV)” page (a near-duplicate of the dashboard’s Markowitz tab) with a dedicated deep-dive analytical page:
 - Radar chart across 5 normalised dimensions — Low Risk (1 — normalised σ), Diversification (1 — Herfindahl index), UCITS Coverage (weight share in UCITS-eligible ETFs), Drawdown Protection (1 — |max drawdown|), Return Potential (normalised expected return) — with HRP and Markowitz MV as overlapping polygons
 - Risk-contribution comparison: grouped horizontal bar chart showing each asset’s share of total portfolio risk under HRP vs MV (highlights the concentration typical of mean-variance)
 - 8x8 asset correlation heatmap over the full universe (CSPX.L, EFA, GLD, VNQ, AGGH.MI, TLT, TIP, XEON.MI) with a stylised structure — the negative equity-bond correlation is the main diversification driver (Phase B will compute it from two-year rolling prices)
- **PR #110 — questionnaire nav button fix:** the “View my Portfolio Dashboard” button did nothing on the live app
- **PR #111 — removed the Markowitz tab from the Portfolio Dashboard:** deleted `_render_mv_tab` (~230 lines) so the dashboard shows only the user’s HRP portfolio and all HRP-vs-Markowitz analysis is centralised in the Compare Markowitz page
- **PR #112 — navigation reorder + model description cards + rename:**
 - Moved “Compare Markowitz” immediately after “Portfolio Dashboard” (natural user flow)
 - Added academic description cards (HRP: López de Prado 2016, Ledoit-Wolf shrinkage, 5-40% per-asset / 10-60% per-cluster constraints; Markowitz: 1952, corner solutions, used as benchmark), styled like the existing “What is the Grable-Lytton Scale?” card
 - Renamed “Compare (MV)” → “Compare Markowitz” everywhere (PAGES list, routing, nav icons, header)

How I did it

- All changes via pull requests on `Programming-for-finance-II/robo-advisor`, fully traceable in the commit history
- Python syntax validated with `ast.parse` before each commit
- Diagnosed the button bug: `main()` resolves the active page from the `?page=` URL query parameter on every rerun, overwriting `session_state.active_page`. The button updated `session_state` but not the query param, so the navigation was silently cancelled

Difficulties

- The questionnaire nav button (PR #110): fixed by setting `st.query_params["page"] = "Portfolio Dashboard"` before `st.rerun()` in the button handler

Achievements / Key decisions

- HRP-vs-Markowitz analysis is now centralised on one dedicated page; the dashboard is HRP-only
- Decision: keep Phase A (mock) values throughout while preserving the Phase B live-data paths for when the live toggle is enabled
- All four PRs merged and deployed automatically to `robo-advisor-usi.streamlit.app` via Streamlit Community Cloud

Next steps

- Replace the stylised correlation matrix with a live two-year rolling computation in Phase B
- Swap the mock MV data for live Markowitz optimizer output

Notes for the academic PDF

- The radar chart's five dimensions (Low Risk, Diversification via 1 – Herfindahl, UCITS Coverage, Drawdown Protection, Return Potential) are a compact way to visualise the HRP-vs-Markowitz trade-off — citable in the Portfolio Optimization section
 - The query-param vs `session_state` navigation bug is a good Streamlit-specific Lessons Learned example (URL state and session state must be kept in sync)
-

P2 — Quant / Portfolio Optimization (session 3)

Estimated duration: ~2 hours **Focus:** Pre-PR code review, navigation refactor, merge-conflict resolution (PR #124)

What I did

- **Code review & bug fixes** before opening the PR, focusing on whether the new charts read the correct data:
 - Hardcoded profile caption — the Backtesting caption always showed “MODERATE” regardless of the selected profile → now reads the active profile from `session_state`
 - Wrong `max_drawdown` fallback — the Compare (MV) page used a fixed `-0.187` (the moderate value) for all profiles → replaced with a profile-aware lookup `{'CONSERVATIVE': -0.112, 'MODERATE': -0.187, 'AGGRESSIVE': -0.312}`
 - Scattered numpy imports — `import numpy as np` appeared four times inside function bodies → moved to the top-level import block
- **Navigation refactor:** replaced the `st.radio` sidebar widget with icon-based buttons, storing the active page in `session_state` so it survives Streamlit reruns; added the Backtesting and Compare (MV) pages to the navigation
- Confirmed the Backtesting and rebuilt Compare (MV) pages render correctly (equity curve + drawdown charts; radar, risk-contribution bars, efficient frontier scatter, correlation heatmap)

How I did it

- Reviewed the branch against `main` before opening the PR to catch data-wiring errors

- Opened **PR #124** (feat(p2): backtesting page + Compare MV deep-dive + UI polish) targeting main

Difficulties

- Merge conflict: main had added `_render_chat_info_panel()` in the same file region where the branch had extended `_render_hrp_tab()` and added `_render_mv_tab()`. Resolution:
 - Kept main's pipeline HTML inside `_render_chat_info_panel()` (the branch's `fig_rc/dendrogram` code was redundant — main already uses `plot_risk_contributions()` from the charts module)
 - Restored the full `_render_mv_tab()` function at module level, which the conflict had displaced
- After pushing the resolution, CI failed on a single ruff E501 (long frontier caption) — fixed by wrapping the string across two lines; CI then passed and the PR was mergeable

Achievements / Key decisions

- Three data-wiring bugs caught and fixed in review before merge (hardcoded caption, wrong drawdown fallback, scattered imports)
- Navigation state now survives reruns via `session_state` rather than the widget's transient value
- Files changed: `frontend/app.py`, `frontend/style.py`, `scripts/run_backtest.py`, `backtest_output/*.js`, `docs/report.tex`

Next steps

- Phase B: replace the mock MV data with live optimizer output and real backtest results

Notes for the academic PDF

- The profile-aware `max_drawdown` lookup replacing a hardcoded moderate value is a small but concrete example of the “no silent single-profile assumptions” discipline — useful for the Lessons Learned section
- The merge-conflict resolution (keeping main's charts-module call over the branch's redundant inline code) demonstrates favouring the shared module over duplicated logic — documentable in the agentic-coordination notes

P2 — Quant / Portfolio Optimization (session 4)

Estimated duration: ~2 hours **Focus:** Backtesting page rework for non-specialist readability (PRs #146–#147)

What I did

- Reworked the Backtesting page so the HRP-vs-Markowitz comparison is clear and self-explanatory for a non-specialist reader (`frontend/app.py`, with a small support change in `backend/optimizer/charts.py`):

- Restricted the comparison to **HRP vs Markowitz** and dropped the internal 1/N benchmark; relabelled “MV” as “Markowitz” everywhere
- Replaced the annualised CAGR with the **total return over the scenario** (computed from the equity curve) — much clearer on short crisis windows
- Slimmed the metrics table to Total return / Volatility / Sharpe / Max drawdown, with a one-line plain-language hint under each header
- Added a plain-language **“winner” callout** per scenario (best Sharpe and smallest drawdown)
- Added two **allocation donuts** (HRP vs Markowitz) showing the average mix each strategy held, with a “Why these mixes?” explanation
- Removed the redundant HRP metric cards, gave the drawdown chart a clearly filled coloured area, and unified the section-title sizes
- Marked the three US proxy tickers (SPY, AGG, BIL) in grey, with an alert explaining they stand in for the EU UCITS ETFs (CSPX.L, AGGH.MI, XEON.MI) that lack price history back to 2008

How I did it

- Implemented incrementally in `frontend/app.py`; added SPY/AGG/BIL to the cluster/name maps in `backend/optimizer/charts.py` so their donut slices are coloured; reused the dashboard’s `plot_weights_donut` for visual consistency
- Verified live in the running Streamlit app across all three risk profiles and all three stress scenarios
- Data-checked the “Why these mixes?” explanation against all 9 scenario × profile combinations (Herfindahl index + top holdings) so the wording holds everywhere

Difficulties

- A first fix (restore the saved profile on a hard page reload) turned out to be redundant — the same fix had already landed on `main` via the profile-gate work — so I closed that PR
- My initial explanation text was factually wrong: it claimed Markowitz always concentrates in long-term Treasuries and gold (false for 2022, where it held cash and real estate) and that HRP is “more diversified” (by Herfindahl it is actually slightly more concentrated, because it overweights cash) → rewrote it into a general, data-verified version
- Clarified the UCITS vs US-proxy distinction: only 3 of the 8 ETFs are European UCITS, chosen because EU retail investors can only buy UCITS funds under MiFID II; the backtest substitutes US-listed equivalents for the pre-2008 history they lack

Achievements / Key decisions

- PR #146 merged into `main` (full Backtesting page rework); PR #147 opened (grey proxy tickers + explanatory alert)
- Decision: keep the full 11-ticker backtest universe but visually distinguish the 3 US proxies (rather than merging them), with an explanatory note
- Decision: show total return over the scenario instead of CAGR for readability
- Decision: describe the strategies at the method level (what HRP and Markowitz do) so the explanation stays true across every scenario and profile

Next steps

- Merge PR #147
- Optional future polish: consolidate each proxy/primary pair into a single holding (8-asset view) for a cleaner donut

Notes for the academic PDF

- The corrected explanation is a documentable example of verifying narrative claims against the data: HRP is slightly *more* concentrated by Herfindahl (it overweights cash), the opposite of the intuitive “HRP is more diversified” claim — a good honesty point for the Portfolio Optimization / Limitations sections
 - The UCITS vs US-proxy substitution (3 of 8 ETFs are UCITS; SPY/AGG/BIL fill the pre-2008 gap) ties the backtest directly to the MiFID II / EU Awareness narrative
-

P2 — Quant / Portfolio Optimization (session 5)

Estimated duration: ~1 hour **Focus:** Portfolio Dashboard chart readability (`charts.py`)

What I did

- Improved the readability and visual consistency of the Portfolio Dashboard charts (`backend/optimizer/charts.py`):
 - **Risk Contributions** — each bar is now coloured by its asset cluster (Equity, Bonds, Alternatives, Cash) using the same colour mapping as the Allocation donut (previously every bar shared one flat colour, so a bar could not be matched to its donut slice)
 - **Allocation donut** — thin slices (e.g. Gold, Real Estate) previously dropped their percentage label; labels now auto-position (roomy slices keep the label inside the ring, thin slices push it outside) so all eight asset percentages are always readable

How I did it

- `plot_risk_contributions()` — bars coloured per cluster
- `plot_weights_donut()` — slice labels set to automatic positioning

Difficulties

- None of note — a focused readability change

Achievements / Key decisions

- The dashboard now presents a consistent colour language across the allocation and risk views, and no allocation percentage is hidden
- Implemented on a dedicated branch and opened a PR; reviewed and merged into `main`

Next steps

- None — readability change complete

Notes for the academic PDF

- The shared cluster palette across the donut and the risk-contribution bars is a small but concrete visual-consistency decision — citable in the Frontend/UX section
 - Deployment note worth recording: the live Streamlit Cloud app caches the running process, so a new build is only picked up after rebooting the app from the Streamlit Cloud dashboard — a useful Lessons Learned detail about the deployment workflow
-

P3 — ML / Risk Profiling

Estimated duration: ~3 hours **Focus:** Backtesting bug fix (unconstrained fallback tickers) + 3 new historical scenarios

What I did

Bug fix — backtesting with unconstrained fallback tickers

- Identified a critical bug in the backtest engine: the optimizer received the cluster map keyed by the primary UCITS tickers (e.g. CSPX.L, XEON.MI, AGGH.MI), but in the historical years those ETFs did not yet exist
- yfinance returned the columns under the US fallback tickers (SPY, BIL, AGG) — the keys did not match → the cluster constraints were never applied
- Without the cash cap (25% for MODERATE), risk-parity poured everything into the lowest-volatility asset (BIL) up to a fixed 40%
- **Effect of the bug:** every backtest was secretly holding ~40% cash → unrealistically low drawdowns
- **Fix applied:** a `make_optimizer_fallback_aware()` function so that each fallback ticker inherits the cluster of its primary — the constraints now always bind
- The fix is inert in the live system (the UCITS ETFs exist at current dates, no fallback active) — the dashboard does not change, but the latent bug is closed
- Added a regression test for the fix
- Committed the historical prices for offline reproducibility

Numerical impact of the fix (MODERATE profile, 2008 scenario)

Metric	Before the fix	After the fix
Return	−0.1% (€9,989)	−8.1% (€9,190)
Max drawdown	−11.9%	−22.3%
Average cash in portfolio	~40%	~13%

Drawdowns now scale correctly with risk: - Conservative 2008: −7% - Moderate 2008: −22%
- Aggressive 2008: −37%

New backtesting scenarios (from 3 to 6 episodes total)

Added 3 new historical episodes:

1. **Eurozone Debt Crisis (2011)** — HRP turns positive thanks to the gold and bond rally: demonstrates the real value of HRP diversification in asymmetric crises
2. **Rate-Fear Selloff (2018)** — mild losses, a moderate stress scenario
3. **Post-COVID Bull (2021)** — a positive contrast scenario: +5.4% total, Sharpe 1.53

Episodes now covered: 2008, 2011, 2018, 2020, 2021, 2022 — solid historical coverage for the demo and defensible at the exam.

How I did it

- Analyzed the bug by tracing the data flow: `yfinance loader` → `fallback map` → `cluster map` → `optimizer constraints`
- Comparative debugging: ran the backtest with and without the fix and compared the output
- Extended the historical dataset for the 3 new episodes via `yfinance`
- Used Claude as an advisor to structure the debugging and document the numerical impact
- Wrote a regression test to prevent future regressions

Difficulties

- The bug was silent: no runtime error, the results looked reasonable until the portfolio composition was analyzed by date
- Identification required noticing that the fixed ~40% cash was anomalous for a MODERATE profile
- The fix had to be inert for the live system — verified explicitly before committing

Achievements / Key decisions

- **Serious bug closed:** the backtest numbers are now real, verifiable and defensible at the exam
- **Historical coverage broadened:** 6 episodes vs the previous 3 — covering both crises (2008, 2011, 2020, 2022) and positive (2021) and moderate (2018) regimes
- The Conservative/Moderate/Aggressive progression on drawdowns is now monotonic and correct — a strong argument for the demo
- The 2011 scenario is the most academically interesting: HRP positive in a crisis where European equities were losing — demonstrates the key point of López de Prado (2016)
- Prices committed for offline reproducibility → the professor can reproduce the results without an internet connection

Next steps

- Update Section 5 of the LaTeX (Backtest Results) with the 6 scenarios and the new tables
- Verify that the Streamlit dashboard (P4) shows all 6 episodes in the backtesting tab
- Coordinate with P4 to update any mock data if needed
- Consider adding a comparative HRP vs Markowitz vs 1/N table across all 6 scenarios

Notes for the academic PDF

- The fallback-ticker bug is a concrete example of a “hidden assumption” in backtesting — citable in the Limitations section as a resolved methodological risk
 - Having the fallback ticker inherit its primary’s cluster is a documentable design choice: it preserves the economic semantics of the cluster even in the absence of the primary tickers
 - The 2011 scenario (Eurozone Debt Crisis) with HRP positive is the ideal test case for the Portfolio Optimization section: it shows hierarchical diversification in action on real data
 - The 2008–2022 range with 6 episodes covers 3 distinct crisis types (subprime, sovereign debt, rate shock) and 2 positive regimes — methodologically solid for an academic paper
 - The regression test added ensures future changes to the optimizer cannot silently reintroduce the bug — a brief mention in the Lessons Learned section
-

P4 — Frontend / LLM / Docs

Estimated duration: ~2 hours **Focus:** Post-submission polish — centralized theme system and Light mode

What I did

1. **Centralized theme system** (6b51996)
 - Implemented `get_theme_tokens()` and `is_light()` in `style.py`: semantic tokens for backgrounds, text, borders, accent, shadows
 - Added `LIGHT_CSS` and `apply_plotly_theme()` for the Plotly charts
 - Dark/Light toggle on the Settings page with instant switching (no restart)
2. **Navbar, cards and radios** (7fd5d20)
 - Light translucent navbar with shadow
 - Cards with borders and shadows consistent with the theme
 - Radio buttons: removed black dots in light mode
3. **Full polish** (b769d0e)
 - Contrast for buttons, tabs, pills, segmented controls
 - Plotly axis text legible in both themes
 - Removed black boxes in the chat input
 - Extended tokens: button, input, chart, divider
4. **Questionnaire** (fd7c8b5)
 - “Financial Situation” container visually reinforced
 - Questions separated by spacing and dividers
 - Removed the “cube cards” around the radio options (they looked cluttered)
5. **On-request refinements**
 - Chart axis titles legible in light mode
 - Compare emoji: → (9fae6b4)
 - Chat sparkle: in light mode, in dark mode (11a8229, babec13)
 - Table rows more visible in light mode
 - Full-width ETF descriptions (132e0e3)
6. **Merge with main** (ea29a5e)
 - Resolved 3 conflicts keeping Matteo’s (P1) Chat Advisor redesign + profile gate
 - Re-integrated the theme tokens where they had been overwritten

7. **“Complete your questionnaire first” gate in light theme** (36f0adb)
- Light override for the block screen — it had remained with a dark background

How I did it

- Token-first approach: all color constants centralized in `style.py`, no hardcoded colors in the pages
- Live visual testing across all pages (Questionnaire, Profile, Portfolio, Chat Advisor) in both modes
- `uv run ruff check frontend/app.py --fix` after every significant change
- `pytest` — 230 passed / 6 skipped at the end of the session
- Git: 8+ atomic commits with descriptive messages, branch `feature/p4-theme-light` aligned with `main`

Difficulties

- Merge conflicts with Matteo’s (P1) parallel work: resolved keeping both contributions, theme tokens re-integrated manually at the conflict points
- Some Plotly colors do not automatically inherit the custom CSS — solved with `apply_plotly_theme()`, which passes the tokens directly to the Plotly layouts

Achievements / Key decisions

- **Dark mode completely unchanged:** no regression on the existing theme
- **Light mode complete and consistent** across all pages and all components (navbar, cards, charts, chat, tables, questionnaire, gate screen)
- `ruff` clean, `pytest` green — no functional regression
- Branch aligned with `main` and ready for the final merge
- ~1260 lines added between `app.py` and `style.py`

Next steps

- **Optional:** dedicated light overrides for the Chat Advisor header redesigned by Matteo (`ca-id-*`) — the area does not yet have explicit light tokens
- Merge the branch into the final `main` before submission
- End-to-end test with both themes on the deployed app (Streamlit Cloud / Railway)
- Verify that the theme toggle is documented in the README / user guide for the professor

Notes for the academic PDF

- The centralized theme system (`get_theme_tokens()` + semantic tokens) is a concrete example of **separation of concerns** in the frontend: styling logic is separated from page logic. Citable in the Frontend/UX section as a deliberate architectural choice
- The handling of merge conflicts with P1’s work is documentable in the **Lessons Learned** section as an example of agentic coordination in a distributed team: keeping both contributions without losing anyone’s work requires an explicit merge strategy
- `apply_plotly_theme()` as a bridge between custom CSS properties and Plotly layout parameters is a non-obvious solution — worth a mention in the Frontend section as a technical integration across different libraries

P1 — Backend / Data Engineering

Estimated duration: ~3 hours **Focus:** Compare Markowitz page — full rework to real data, colour consistency, and readability (frontend/app.py, render_compare)

What I did

1. Live data infrastructure

- The Compare page now loads its own data independently (same cache mechanism as the dashboard), removing the previous dependency on visiting the dashboard first.
- **Auto-heal for stale sessions:** if a cached session is missing any expected MV field (mv_risk_contributions, mv_expected_volatility, mv_sharpe_ratio, mv_max_drawdown, correlation), the page silently re-fetches once.
- Added `_run_live_optimization`: given the HRP prices already in cache, computes MV risk contributions (marginal variance formula, same covariance matrix as HRP), MV vol/return/Sharpe/max-drawdown, the real correlation matrix, the HRP quasi-diagonal ordering, and the cluster structure from backend/optimizer/hrp.py.
- Removed the dead function `_render_mv_tab` (~184 lines): a legacy Markowitz tab that was never called and produced fabricated MV data.

2. Section 1 — Key Metrics (formerly: radar chart)

- Replaced a radar that compared HRP against a mock Markowitz ($vol = HRP \times 0.92$, arbitrary 0-1 normalisation) with a **real-data scorecard** matching the dashboard table style.
- Metrics: annual volatility, max drawdown, largest single-asset risk contribution, diversification score — all from live data. Sharpe shown for MV only (HRP does not estimate returns by design).
- The winning value for each metric is **circled** in the method's colour (purple HRP / red MV); the losing value is muted.
- Above the table: a **data-driven verdict** (head-to-head winner count) and a **verdict card** with keyword chips (Robust, Diversified, Adapts to you VS Efficient, Higher Sharpe, Concentrated).
- Honest disclosure: HRP omits Sharpe by design to avoid the unstable return estimate that causes Markowitz to concentrate; Markowitz is one-size-fits-all across profiles while HRP adapts.

3. Section 2 — Risk Contributions

- MV risk contributions are now **real** (marginal variance, same covariance as HRP), not $weight \times volatility$.
- Replaced the stacked class chart and the desaturated palette with a single grouped horizontal bar chart (HRP vs MV, standard purple/red colours), an **equal-risk reference line**, percentage labels, and rounded corners.
- Adaptive headline above the chart (e.g. "Markowitz concentrates 53% on one asset; HRP distributes").

4. Section 3 — Asset Correlation Matrix

- Matrix now uses **real correlations** from live prices (offline stylised fallback labelled as such).
- Colour scale changed to teal → purple (consistent with the page palette, replacing blue/orange).

- Matrix **reordered by the real HRP clusters** (quasi-diagonalisation): real groupings emerge on the diagonal (e.g. gold isolated, real-estate near equities).
- Single amber highlight on the tightest cluster (where Markowitz concentrates); arbitrary boxes removed.
- Scale **clipped to [0, 1]** when no negative correlations are present so the legend matches what is visible; cells are **square**.
- “How to read this chart” moved into a collapsible card (dashboard style): colour-key pills, HRP/MV method cards, market-regime callout, schematic text with bold keywords.

How I did it

- All MV quantities computed from the **same price matrix** already loaded for HRP, so no extra network calls and no divergence between the two methods’ inputs.
- Colour discipline enforced globally: one constant per method (`HRP_COLOR = "#7c5cfc"`, `MV_COLOR = "#f87171"`), used in every chart and card.
- Dead code removed before adding new code to keep the diff reviewable.
- Tested visually across all three profiles (CONSERVATIVE / MODERATE / AGGRESSIVE) in both Dark and Light theme.

Difficulties

- The auto-heal logic required careful ordering: the stale-session check must run before any chart rendering, otherwise the page crashes mid-render on a cache miss.
- Quasi-diagonalisation of the correlation matrix required extracting the leaf order from `scipy.cluster.hierarchy.leaves_list` and applying it consistently to both axes.

Achievements / Key decisions

- Compare Markowitz page is now **fully based on real data** — no mock values passed off as live.
- **One colour = one method** across all three sections and both themes.
- `Dead_render_mv_tab` (~184 lines) removed: cleaner codebase, no risk of stale mock data leaking back.
- Decision: HRP Sharpe shown as – with an explicit note rather than omitted silently — transparency over completeness.

Next steps

- Phase B: wire the `/compare` API endpoint so the live optimisation runs server-side rather than client-side in `render_compare`.

Notes for the academic PDF

- The auto-heal pattern (detect stale fields → re-fetch once) is a small but concrete example of **defensive programming** in a stateful Streamlit session — citable in the Frontend section.
- Computing both MV and HRP quantities from the same covariance matrix eliminates a whole class of “the comparison is unfair because the inputs differ” objections — worth a sentence in Section 3 (Portfolio Optimization).

- The quasi-diagonalisation reordering is the visual proof that HRP’s clustering is meaningful, not decorative — citable in Section 3 alongside the dendrogram.

Deliverable Status Summary — per Week and per Role

Drawn from the individual session logs and the consolidated team-memory documents. This table covers the **core deliverables (W1-W4, 27 April - 24 May)**, by the end of which the system was feature-complete and deployed (v1.0). The refinement work of **Weeks 5-7 (late May - June)** — UI/UX polish, bug fixes, Dark/Light theme, documentation cleanup — is captured in the dated entries above. Status: Done = delivered / merged; In progress = partial; Planned = scheduled for a later week.

P1 — Backend / Data Engineering (Sabrina)

Week	Deliverables	Status
W1 (27 Apr-3 May)	ci.yml (lint + pytest), schema.sql v3.1, ValidatedDataLoader + DataQualityReport, snapshots.py, test_data_loader.py, branch protection on main	Done
W2 (4-10 May)	schema.sql EN labels, FastAPI /profile + 9 integration tests, /optimize wired (HRP + DataLoader + DB audit), ADR-005-db-schema.md, ADR-003-cloud-deploy.md (brought forward)	Done
W3 (11-17 May)	/advice endpoint (3-stage LLM pipeline), X-API-Key auth on all protected endpoints, agent_pr.yml + PR #43 (Criterion 5), input_sanitiser.py (Layer 1), test_advice_pipeline.py, ADR-003 merged	Done

Week	Deliverables	Status
W4 (18-24 May)	Public repo (secrets audit), Streamlit Cloud deploy live, Dockerfile + docker-compose.yml + requirements.txt, /backtest + /compare wired (5/5 endpoints live), test_ucits_fallback.py, CI coverage (77%→81% team-wide), final README.md, v1.0 tag	Done

P2 — Quant / Portfolio Optimization (Emma)

Week	Deliverables	Status
W1 (27 Apr-3 May)	universe_config.py (8 ETFs, 4 clusters, ≥ 3 UCITS), OptimizationResult TypedDict, compute_covariance stub + 3 structural tests	Done
W2 (4-10 May)	hrp.py complete (Ledoit-Wolf, Ward, recursive bisection, profile tilt, box constraints), volatility double-annualization fix, Optional[float] contract, single source of truth on box constraints, 9 tests total	Done
W3 (11-17 May)	backtest.py (3 scenarios + 1/N + TC), download_backtest_data.py, test_backtest.py (9 tests), regime_detector.py (correlation OR VIX, ERC fallback), charts.py (4 Plotly functions), ADR-006-regime-detector.md	Done

Week	Deliverables	Status
W4 (18-24 May)	Code review of markowitz.py (3 fixes), test_charts.py + test_risk_metrics.py (100% coverage, project at 81%), LaTeX §3 + 5 bib entries + ADR-007-ledoit-wolf-shrinkage.md, _render_mv_tab, §5 backtest tables with real numbers	Done

P3 — ML / Risk Profiling (Matteo)

Week	Deliverables	Status
W1 (27 Apr-3 May)	questionnaire_schema.md (Grable & Lytton, 10 questions, Q7 MiFID II override), rule_based.py (Phase A profiler), scf_pipeline.py scaffold, ADR-002-scf-preprocessing.md, SCF 2022 empirical verification	Done
W2 (4-10 May)	clustering.py (K-Means on SCF 2022, K=3, silhouette), build_pipeline() bug fix (demographic features in parquet), test_profiler.py, scf_labeled.parquet ready for GBM	Done
W3 (11-17 May)	classifier.py Phase B (HistGBM + SHAP + LR baseline, CV 94.0%), regime_detector.py scaffold, 43 tests passed	Done
W4 (18-24 May)	Code cleanup (build_pipeline 5-tuple, named constants, NumPy docstrings), ADR-009-scf-implicate-choice.md (PR #95), LaTeX Section 2 (ML Risk Profiler)	Done

P4 — Frontend / LLM / Docs (Elena)

Week	Deliverables	Status
W1 (27 Apr–3 May)	AGENTS.md, frontend/app.py scaffold (4 pages), README.md, docs/architecture.md, backend/schemas/ground_truth.py (Pydantic v2) + mock_data.py, ADR-001-hrp-over-markowitz.md	Done
W2 (4–10 May)	ground_truth_schema.md, system_prompt.py (9 rules), narrator.py (NarratorClient, stateless, temp=0.0), 10-question Grable-Lytton Streamlit questionnaire, validator.py 4-step (27 tests), report.tex skeleton + references.bib	Done
W3 (11–17 May)	Validator Step 5 (EU Awareness Rule 9, 34→37 tests, 11 EU cases), ADR-004-llm-narrator-validator.md, Chat Advisor wired (3-stage pipeline), docs/user_guide.md (437 lines), AGENTS.md Evidence Log (PR #43)	Done
W4 (18–24 May)	README AI Tools section, premium dark Streamlit theme (style.py, palette), portfolio dashboard (dendrogram, MV tab, UCITS badges, stress banner, ticker glossary, EU Note card), Settings page, st.button navigation, LaTeX sections 1/4/6/7/8, final AGENTS.md	Done